

User Manual

for S32K1_S32M24X MCL Driver

Document Number: UM2MCLASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	6
3.4 Deviations from Requirements	7
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	8
3.6.1 MCAL MCL DMA migration guide to RTD MCL	8
3.6.1.1 Introduction	8
3.6.1.2 TRESOS Configuration	8
3.6.2 Channel State Machine	11
3.6.2.1 MCL Driver	11
3.6.2.2 DMA Driver	12
3.7 Runtime errors	14
3.8 Symbolic Names Disclaimer	15
3.9 Usage in non AUTOSAR context	15
4 Configuration Parameters	18
4.1 Mcl	19
4.1.1 Module Mcl	23
4.1.2 Container MclGeneral	23
4.1.3 Parameter MclEnableDemReportErrorStatus	24
4.1.4 Parameter MclEnableDevErrorDetect	24
4.1.5 Parameter Mcl_VersionInfoApi	25
4.1.6 Parameter MclEnableUserModeSupport	25
4.1.7 Parameter MclEnableMultipartitionSupport	25
4.1.8 Parameter MclEnableVirtualAddressMappingSupport	26
4.1.9 Container MclDma	26
4.1.10 Parameter MclEnableDma	26
4.1.11 Container MclCache	27
4.1.12 Parameter MclEnableCache	27
4.1.13 Parameter MclCacheTimeoutValue	28
4.1.14 Parameter MclCacheTimeoutMethod	28
4.1.15 Container MclTrgMux	28
4.1.16 Parameter MclEnableTrgMux	29
4.1.17 Container MclFlexioCommon	29
4.1.18 Parameter MclEnableFlexioCommon	30
4.1.19 Container MclFtmCommon	30
4.1.20 Parameter Mcl_FtmCommonTimebase	30
4.1.21 Parameter Mcl_FtmSplitCommonTimebase	31
4.1.22 Container MclConfig	31
4.1.23 Container MclVirtualMemorySection	32
4.1.24 Parameter MclVirtualAddressStart	32
4.1.25 Parameter MclVirtualAddressEnd	32
4.1.26 Parameter MclPhysicalAddressStart	33

4.1.27	Parameter	MclPhysicalAddressEnd	33
4.1.28	Container	MclDemEventParameterRefs	34
4.1.29	Reference	MCL_E_TIMEOUT_FAILURE	34
4.1.30	Container	dmaLogicInstance_ConfigType	35
4.1.31	Parameter	dmaLogicInstance_IdName	35
4.1.32	Parameter	dmaLogicInstance_hwId	35
4.1.33	Parameter	dmaLogicInstance_enDebug	36
4.1.34	Parameter	dmaLogicInstance_enRoundRobin	36
4.1.35	Parameter	dmaLogicInstance_enHaltAfterError	37
4.1.36	Parameter	dmaLogicInstance_enChLinking	37
4.1.37	Reference	dmaLogicInstance_EcucPartitionRef	38
4.1.38	Container	dmaLogicChannel_Type	38
4.1.39	Parameter	dmaLogicChannel_LogicName	39
4.1.40	Parameter	dmaLogicChannel_HwInstId	39
4.1.41	Parameter	dmaLogicChannel_HwChId	40
4.1.42	Parameter	dmaLogicChannel_InterruptCallback	40
4.1.43	Parameter	dmaLogicChannel_ErrorInterruptCallback	41
4.1.44	Parameter	dmaLogicChannel_EnableGlobalConfig	41
4.1.45	Parameter	dmaLogicChannel_EnableTransferConfig	41
4.1.46	Parameter	dmaLogicChannel_EnableScatterGather	42
4.1.47	Reference	dmaLogicChannel_EcucPartitionRef	42
4.1.48	Container	dmaLogicChannel_ConfigType	43
4.1.49	Container	dmaLogicChannel_GlobalConfigType	43
4.1.50	Container	dmaLogicChannelConfig_GlobalRequestType	44
4.1.51	Parameter	dmaGlobalRequest_enDmaRequest	44
4.1.52	Container	dmaLogicChannelConfig_GlobalInterruptType	44
4.1.53	Parameter	dmaGlobalInterrupt_enDmaErrorInterrupt	45
4.1.54	Container	dmaLogicChannelConfig_GlobalPriorityType	45
4.1.55	Parameter	dmaGlobalPriority_LevelPriority	45
4.1.56	Parameter	dmaGlobalPriority_enPreemption	46
4.1.57	Parameter	dmaGlobalPriority_disPreempt	46
4.1.58	Container	dmaLogicChannel_TransferConfigType	47
4.1.59	Container	dmaLogicChannelConfig_TransferControlType	47
4.1.60	Parameter	dmaLogicChannelConfig_enDmaMajorInterrupt	47
4.1.61	Parameter	dmaLogicChannelConfig_enDmaHalfMajorInterrupt	48
4.1.62	Parameter	dmaLogicChannelConfig_disDmaAutoHwReq	48
4.1.63	Parameter	dmaLogicChannelConfig_bandwidthControl	49
4.1.64	Parameter	dmaLogicChannelConfig_ScatterGatherAddressType	49
4.1.65	Parameter	dmaLogicChannelConfig_DestinationStoreAddressType	49
4.1.66	Container	dmaLogicChannelConfig_TransferSourceType	50
4.1.67	Parameter	dmaLogicChannelConfig_SourceAddressType	50
4.1.68	Parameter	dmaLogicChannelConfig_SourceSignedOffsetType	50
4.1.69	Parameter	dmaLogicChannelConfig_SourceLastAddressAdjustmentType	51
4.1.70	Parameter	dmaTransferConfig_TransferSizeType	51
4.1.71	Parameter	dmaLogicChannelConfig_SourceModuloType	52
4.1.72	Container	dmaLogicChannelConfig_TransferDestinationType	52
4.1.73	Parameter	dmaLogicChannelConfig_DestinationAddressType	52
4.1.74	Parameter	dmaLogicChannelConfig_DestinationSignedOffsetType	53
4.1.75	Parameter	dmaLogicChannelConfig_DestinationLastAddressAdjustmentType	53
4.1.76	Parameter	dmaTransferConfig_TransferSizeType	54
4.1.77	Parameter	dmaLogicChannelConfig_DestinationModuloType	54
4.1.78	Container	dmaLogicChannelConfig_TransferMinorLoopType	55
4.1.79	Parameter	dmaLogicChannelConfig_enSourceOffset	55
4.1.80	Parameter	dmaLogicChannelConfig_enDestinationOffset	55

4.1.81	Parameter dmaLogicChannelConfig_OffsetValueType	56
4.1.82	Parameter dmaLogicChannelConfig_enMinorLoopLinkCh	56
4.1.83	Parameter dmaLogicChannelConfig_MinorLoopSizeType	56
4.1.84	Reference dynamic_dmaLogicChannelConfig_MinorLoopLinkChValueType	57
4.1.85	Container dmaLogicChannelConfig_TransferMajorLoopType	57
4.1.86	Parameter dmaLogicChannelConfig_enMajorLoopLinkCh	57
4.1.87	Parameter dmaLogicChannelConfig_MajorLoopCountType	58
4.1.88	Reference dynamic_dmaLogicChannelConfig_MajorLoopLinkChValueType	58
4.1.89	Container dmaLogicChannelConfig_ScatterGatherConfigType	59
4.1.90	Container dmaLogicChannelConfig_ScatterGatherArrayType	59
4.1.91	Container dmaLogicChannelConfig_ScatterGatherElementConfigType	59
4.1.92	Parameter dmaLogicChannelConfig_ScatterGatherElementNameType	60
4.1.93	Parameter dmaLogicChannelConfig_LastElementLink_ScatterGatherType	60
4.1.94	Parameter dmaLogicChannelConfig_enScatterGatherConfig	61
4.1.95	Reference dynamic_dmaLogicChannelConfig_BasicElementLink_ScatterGatherType	61
4.1.96	Container dmaLogicChannelConfig_TransferControlType	61
4.1.97	Parameter dmaLogicChannelConfig_enStart	62
4.1.98	Parameter dmaLogicChannelConfig_enDmaMajorInterrupt	62
4.1.99	Parameter dmaLogicChannelConfig_enDmaHalfMajorInterrupt	62
4.1.100	Parameter dmaLogicChannelConfig_disDmaAutoHwReq	63
4.1.101	Parameter dmaLogicChannelConfig_bandwidthControl	63
4.1.102	Parameter dmaLogicChannelConfig_ScatterGatherAddressType	64
4.1.103	Parameter dmaLogicChannelConfig_DestinationStoreAddressType	64
4.1.104	Container dmaLogicChannelConfig_TransferSourceType	64
4.1.105	Parameter dmaLogicChannelConfig_SourceAddressType	65
4.1.106	Parameter dmaLogicChannelConfig_SourceSignedOffsetType	65
4.1.107	Parameter dmaLogicChannelConfig_SourceLastAddressAdjustmentType	66
4.1.108	Parameter dmaTransferConfig_TransferSizeType	66
4.1.109	Parameter dmaLogicChannelConfig_SourceModuloType	66
4.1.110	Container dmaLogicChannelConfig_TransferDestinationType	67
4.1.111	Parameter dmaLogicChannelConfig_DestinationAddressType	67
4.1.112	Parameter dmaLogicChannelConfig_DestinationSignedOffsetType	68
4.1.113	Parameter dmaLogicChannelConfig_DestinationLastAddressAdjustmentType	68
4.1.114	Parameter dmaTransferConfig_TransferSizeType	68
4.1.115	Parameter dmaLogicChannelConfig_DestinationModuloType	69
4.1.116	Container dmaLogicChannelConfig_TransferMinorLoopType	69
4.1.117	Parameter dmaLogicChannelConfig_enSourceOffset	70
4.1.118	Parameter dmaLogicChannelConfig_enDestinationOffset	70
4.1.119	Parameter dmaLogicChannelConfig_OffsetValueType	70
4.1.120	Parameter dmaLogicChannelConfig_enMinorLoopLinkCh	71
4.1.121	Parameter dmaLogicChannelConfig_MinorLoopSizeType	71
4.1.122	Reference dynamic_dmaLogicChannelConfig_MinorLoopLinkChValueType	72
4.1.123	Container dmaLogicChannelConfig_TransferMajorLoopType	72
4.1.124	Parameter dmaLogicChannelConfig_enMajorLoopLinkCh	72
4.1.125	Parameter dmaLogicChannelConfig_MajorLoopCountType	73
4.1.126	Reference dynamic_dmaLogicChannelConfig_MajorLoopLinkChValueType	73
4.1.127	Container trgmuxInstaceList	73
4.1.128	Parameter trgmuxHardwareInstance	74
4.1.129	Container trgmuxLogicGroup	74
4.1.130	Parameter trgmuxLogicGroupHardwareInstance	74
4.1.131	Parameter trgmuxLogicGroup_Name	75
4.1.132	Parameter trgmuxLogicGroup_Lock	75
4.1.133	Container trgmuxLogicTrigger	76
4.1.134	Parameter trgmuxLogicTrigger_Name	76

4.1.135	Parameter	trgmuxLogicTrigger_Output	77
4.1.136	Parameter	trgmuxLogicTrigger_Input	78
4.1.137	Reference	trgmuxLogicTrigger_EcucPartitionRef	79
4.1.138	Container	FlexioCommon	80
4.1.139	Parameter	FlexioMclInstances	80
4.1.140	Parameter	FlexioDebugEnable	80
4.1.141	Container	FlexioMclLogicChannels	81
4.1.142	Parameter	FlexioMclChannelId	81
4.1.143	Parameter	FlexioMclPinId	82
4.1.144	Parameter	FlexioMclAddPinEnable	82
4.1.145	Parameter	FlexioMclAddPinId	82
4.1.146	Parameter	FlexioMclAddChannelEnable	83
4.1.147	Parameter	FlexioMclAddChannelId	83
4.1.148	Container	CommonPublishedInformation	84
4.1.149	Parameter	ArReleaseMajorVersion	84
4.1.150	Parameter	ArReleaseMinorVersion	84
4.1.151	Parameter	ArReleaseRevisionVersion	85
4.1.152	Parameter	ModuleId	85
4.1.153	Parameter	SwMajorVersion	86
4.1.154	Parameter	SwMinorVersion	86
4.1.155	Parameter	SwPatchVersion	87
4.1.156	Parameter	VendorId	87
4.2	DMA_IP		87
4.2.1	Module	Dma_Ip	88
4.2.2	Parameter	MclEnableDmaDevErrorDetect	89
4.3	CACHE_IP		89
4.3.1	Module	Cache_Ip	89
4.3.2	Parameter	MclEnableCacheDevErrorDetect	90
4.4	TRGMUX_IP		90
4.4.1	Module	Trgmux_Ip	90
4.4.2	Parameter	MclEnableTrgMuxDevErrorDetect	91
4.5	FLEXIO_MCL_IP		91
4.5.1	Module	Flexio_Mcl_Ip	92
4.5.2	Parameter	MclEnableFlexioDevErrorDetect	92
4.6	FTM_MCL_IP		92
4.6.1	Module	Ftm_Mcl_Ip	93
4.6.2	Parameter	MclFtmEnableUserModeSupport	93
4.6.3	Parameter	MclFtmEnableDevErrorDetect	93
5	Module Index		95
5.1	Software Specification		95
6	Module Documentation		96
6.1	CACHE IP Driver		96
6.1.1	Detailed Description		96
6.1.2	Enum Reference		96
6.1.2.1	Cache_Ip_Type		97
6.1.2.2	Cache_Ip_BusType		97
6.1.3	Function Reference		97
6.1.3.1	Cache_Ip_Enable()		97
6.1.3.2	Cache_Ip_Disable()		97
6.1.3.3	Cache_Ip_Invalidate()		98
6.1.3.4	Cache_Ip_Clean()		98
6.1.3.5	Cache_Ip_InvalidateByAddr()		99

6.1.3.6	Cache_Ip_CleanByAddr()	99
6.2	MCL Driver	100
6.2.1	Detailed Description	100
6.2.1.1	MISRA-C:2012 violations	104
6.2.2	Data Structure Documentation	104
6.2.2.1	struct Mcl_DmaChannelGlobalListType	104
6.2.2.2	struct Mcl_DmaChannelTransferListType	105
6.2.2.3	struct Mcl_DmaChannelScatterGatherListType	105
6.2.2.4	struct Mcl_DmaInstanceStatusType	106
6.2.2.5	struct Mcl_DmaChannelStatusType	107
6.2.2.6	struct Mcl_ConfigType	108
6.2.3	Macro Definition Documentation	108
6.2.3.1	MCL_DET_INIT	108
6.2.3.2	MCL_DET_DMA_INSTANCE_COMMAND	109
6.2.3.3	MCL_DET_DMA_INSTANCE_STATUS	109
6.2.3.4	MCL_DET_DMA_CHANNEL_COMMAND	109
6.2.3.5	MCL_DET_DMA_CHANNEL_STATUS	109
6.2.3.6	MCL_DET_DMA_GLOBAL	109
6.2.3.7	MCL_DET_DMA_TRANSFER	110
6.2.3.8	MCL_DET_DMA_SCATTER_GATHER_LIST	110
6.2.3.9	MCL_DET_DMA_INFORMATION	110
6.2.3.10	MCL_DET_DMA_SCATTER_GATHER_CONFIG	110
6.2.3.11	MCL_DET_DEINIT	110
6.2.3.12	MCL_DET_CACHE_ENABLE	111
6.2.3.13	MCL_DET_CACHE_DISABLE	111
6.2.3.14	MCL_DET_CACHE_INVALIDATE	111
6.2.3.15	MCL_DET_CACHE_CLEAN	111
6.2.3.16	MCL_DET_CACHE_INVALIDATE_BY_ADDRESS	111
6.2.3.17	MCL_DET_CACHE_CLEAN_BY_ADDRESS	112
6.2.3.18	MCL_DET_TRGMUX_INPUT	112
6.2.3.19	MCL_DET_TRGMUX_LOCK	112
6.2.3.20	MCL_DET_TRGMUX_SET_INPUT	112
6.2.3.21	MCL_DET_TRGMUX_SET_LOC	112
6.2.3.22	MCL_E_UNINIT	113
6.2.3.23	MCL_E_PARAM_POINTER	113
6.2.3.24	MCL_E_INVALID_INSTANCE	113
6.2.3.25	MCL_E_INVALID_CHANNEL	113
6.2.3.26	MCL_E_INVALID_COMMAND	113
6.2.3.27	MCL_E_INVALID_PARAMETER	113
6.2.3.28	MCL_E_INVALID_STATE	114
6.2.3.29	MCL_E_INCONSISTENCY	114
6.2.3.30	MCL_E_TIMEOUT	114
6.2.3.31	MCL_E_PARAM_CONFIG	114
6.2.3.32	MCL_E_PROTECTED	114
6.2.3.33	MCL_E_INIT_FAILED	114
6.2.4	Enum Reference	114
6.2.4.1	Mcl_DmaInstanceCmdType	115
6.2.4.2	Mcl_DmaChannelCmdType	115
6.2.4.3	Mcl_DmaChannelGlobalParamType	115
6.2.4.4	Mcl_DmaChannelTransferParamType	116
6.2.4.5	Mcl_DmaChannelStateType	117
6.2.4.6	Mcl_DmaChannelInfoParamType	118
6.2.4.7	Mcl_CacheType	118
6.2.4.8	Mcl_CacheBusType	118

6.2.4.9 Mcl_ReturnType	119
6.2.5 Function Reference	119
6.2.5.1 Mcl_Init()	119
6.2.5.2 Mcl_DeInit()	119
6.2.5.3 Mcl_SetDmaInstanceCommand()	120
6.2.5.4 Mcl_GetDmaInstanceStatus()	120
6.2.5.5 Mcl_SetDmaChannelCommand()	120
6.2.5.6 Mcl_GetDmaChannelStatus()	121
6.2.5.7 Mcl_SetDmaChannelGlobalList()	121
6.2.5.8 Mcl_SetDmaChannelTransferList()	122
6.2.5.9 Mcl_SetDmaChannelScatterGatherList()	122
6.2.5.10 Mcl_GetDmaChannelParam()	123
6.2.5.11 Mcl_SetDmaChannelScatterGatherConfig()	124
6.2.5.12 Mcl_CacheEnable()	124
6.2.5.13 Mcl_CacheDisable()	124
6.2.5.14 Mcl_CacheInvalidate()	125
6.2.5.15 Mcl_CacheClean()	125
6.2.5.16 Mcl_CacheInvalidateByAddr()	126
6.2.5.17 Mcl_CacheCleanByAddr()	126
6.2.5.18 Mcl_SetTrgMuxInput()	127
6.2.5.19 Mcl_SetTrgMuxLock()	127
6.2.5.20 Mcl_SelectCommonTimebase()	127
6.2.5.21 Mcl_GetVersionInfo()	128
6.3 DMA IP Driver	128
6.3.1 Detailed Description	128
6.3.2 Data Structure Documentation	130
6.3.2.1 struct Dma_Ip_LogicChannelGlobalListType	130
6.3.2.2 struct Dma_Ip_LogicChannelTransferListType	131
6.3.2.3 struct Dma_Ip_LogicInstanceStatusType	131
6.3.2.4 struct Dma_Ip_LogicChannelStatusType	132
6.3.2.5 struct Dma_Ip_LogicChannelIdType	133
6.3.2.6 struct Dma_Ip_GlobalConfigType	135
6.3.2.7 struct Dma_Ip_TransferConfigType	135
6.3.2.8 struct Dma_Ip_ScatterGatherConfigType	135
6.3.2.9 struct Dma_Ip_LogicChannelConfigType	135
6.3.2.10 struct Dma_Ip_LogicInstanceIdType	136
6.3.2.11 struct Dma_Ip_LogicInstanceConfigType	136
6.3.2.12 struct Dma_Ip_HwChannelStateType	137
6.3.2.13 struct Dma_Ip_VirtualSectionConfigType	137
6.3.2.14 struct Dma_Ip_VirtualMemoryConfigType	137
6.3.2.15 struct Dma_Ip_InitType	137
6.3.3 Types Reference	137
6.3.3.1 Dma_Ip_Callback	137
6.3.4 Enum Reference	138
6.3.4.1 Dma_Ip_LogicInstanceCmdType	138
6.3.4.2 Dma_Ip_LogicChannelCmdType	138
6.3.4.3 Dma_Ip_LogicChannelGlobalParamType	139
6.3.4.4 Dma_Ip_LogicChannelTransferParamType	139
6.3.4.5 Dma_Ip_LogicChannelInfoParamType	141
6.3.4.6 Dma_Ip_ReturnType	141
6.3.4.7 Dma_Ip_HwChannelStateValueType	141
6.3.5 Function Reference	141
6.3.5.1 Dma_Ip_Init()	141
6.3.5.2 Dma_Ip_Deinit()	142

6.3.5.3	Dma_Ip_SetLogicInstanceCommand()	142
6.3.5.4	Dma_Ip_GetLogicInstanceStatus()	142
6.3.5.5	Dma_Ip_LogicChannelInit()	143
6.3.5.6	Dma_Ip_LogicChannelDeinit()	143
6.3.5.7	Dma_Ip_SetLogicChannelCommand()	143
6.3.5.8	Dma_Ip_GetLogicChannelStatus()	144
6.3.5.9	Dma_Ip_SetLogicChannelGlobalList()	144
6.3.5.10	Dma_Ip_SetLogicChannelTransferList()	145
6.3.5.11	Dma_Ip_SetLogicChannelScatterGatherList()	146
6.3.5.12	Dma_Ip_GetLogicChannelParam()	147
6.3.5.13	Dma_Ip_SetLogicChannelScatterGatherConfig()	147
6.4	FLEXIO IP Driver	148
6.4.1	Detailed Description	148
6.4.2	Enum Reference	148
6.4.2.1	Flexio_Mcl_Ip_TimerPolarityType	148
6.4.2.2	Flexio_Mcl_Ip_PinPolarityType	148
6.4.2.3	Flexio_Mcl_Ip_PinConfigType	149
6.4.2.4	Flexio_Mcl_Ip_TriggerPolarityType	149
6.4.2.5	Flexio_Mcl_Ip_TriggerSourceType	149
6.4.2.6	Flexio_Mcl_Ip_TimerModeType	149
6.4.2.7	Flexio_Mcl_Ip_TimerOutputType	150
6.4.2.8	Flexio_Mcl_Ip_TimerDecrementType	150
6.4.2.9	Flexio_Mcl_Ip_TimerResetType	150
6.4.2.10	Flexio_Mcl_Ip_TimerDisableType	151
6.4.2.11	Flexio_Mcl_Ip_TimerEnableType	151
6.4.2.12	Flexio_Mcl_Ip_TimerStopType	152
6.4.2.13	Flexio_Mcl_Ip_TimerStartType	152
6.5	FTM IP Driver	152
6.5.1	Detailed Description	152
6.5.2	Function Reference	152
6.5.2.1	Ftm_Mcl_Ip_SelectCommonTimebase()	152
6.6	TRGMUX IP Driver	153
6.6.1	Detailed Description	153
6.6.2	Function Reference	153
6.6.2.1	Trgmux_Ip_Init()	153
6.6.2.2	Trgmux_Ip_SetInput()	153
6.6.2.3	Trgmux_Ip_SetLock()	154

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor's MCL driver for S32K1_S32M24X. MCL driver configuration parameters and deviations from the specification are described in Driver chapter of this document. MCL driver requirements and APIs are described in the MCL driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Driver Software Specification document (See Table [Reference List](#)). For CDD: MCL Driver is a Complex Device Driver (CDD), so there are no requirements regarding this module. It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The Mcl Driver controls the DMA(Direct Memory Access) and CACHE modules of the S32K1 device. It provides the following features:

- Configuration and initialization of the DMA.
- Configuration and initialization of the CACHE.
- Handling of the DMA interrupt requests.
- DMA Normal Transfer Mode and Scatter/Gather Mode.

3.3 Hardware Resources

The Mcl Driver consists of:

1. A DMA Peripheral which has 1 Hardware Instance with 4 Hardware Channels for S32K11X and 16 Hardware Channels for S32K14X.

2. A CACHE Memory is Local Memory Controller (LMEM).

3.4 Deviations from Requirements

The driver deviates from the Mcl Driver Software Specification in some places.

The table [Status Column Description](#) identifies the requirements that are not fully implemented, implemented differently, or out of scope for the Mcl Driver.

The table [Mcl Requirements Deviations](#) provides the "Status" column description.

Status Column Description

Term	Definition
N/S	Not In Scope
N/F	Not Fully Implemented
N/I	Not Implemented

Mcl Requirements Deviations

Requirement	Status	Description	Notes
None	None	None	None

Files **Mcl_<VariantName>_PBcfg.c** and **Mcl_<VariantName>_PBcfg.h** will contain the definitions for all parameters that are variant aware, independent of the configuration class that will be selected (PC, LT, PB).

Files **Mcl_Cfg.c** and **Mcl_Cfg.h** will contain the definitions for all parameters that are not variant aware.

3.5 Driver Limitations

The Mcl Driver has the following limitations:

- When DMA is used with CACHE enabled, the user shall Invalidate/Clean the CACHE in order to synchronize the transfered data.
- When using CACHE Invalidate and Clean functionalities, the user shall take into consideration that all variables that reside in the cache, are affected.
- The Cache Invalidate function need to be called before Cache Enable function is called (make sure that Cache wasn't enabled before).
- The Cache Clean function need to be called before Cache Disable function is called.
- The DMA Driver shall have the Source Address and Destination Address configured at runtime. These two parameters are not available in the configurator.
- When the DMA transfer has errors, user must call `Mcl_SetDmaChannelCommand/Dma_Ip_SetLogicChannelCommand` function to clear error state and error status.
- When using Virtual Address Mapping feature, the user shall handle the linker file to avoid the memory address is invalid.
- For DMA hardware version 2, the ActiveId is not available in CR register.

3.6 Driver usage and configuration tips

3.6.1 MCAL MCL DMA migration guide to RTD MCL

3.6.1.1 Introduction

The RTD MCL DMA Driver brings a Generic Interface, to help the User in application development across multiple SoCs. The Generic Interface consists of software functions that are fully configurable using User defined configurations. The Generic Interface is structured into four function groups:

1. Set Command Functions
2. Get Status Functions
3. Set List Functions
4. Get Information (Parameter) Functions

The Set Command Functions shall trigger actions specific to the invoked Logic Entity. For example, the Logic Dma Instance shall be commanded to: “Stop execution”, “Stop execution with error signaling”, “Pause execution” or “Resume execution”.

The Get Status Functions shall read the status specific to the invoked Logic Entity. For example, the Logic Dma Instance shall return: “Hardware Errors specific to the IP”, “The Active Channel Id” and “Active Status”.

The Set List Functions shall configure a user defined list of settings for the invoked Logic Entity. For example, the Logic Dma Channel shall be configurable with the Transfer List of Parameters like: “Source Address”, “Destination Address”, “Source Signed Offset”, “Destination Signed Offset”, “Major Loop Count”, etc... For the specific ScatterGather mode, the additional Logic Element parameter specifies the Software TCD that shall be loaded with the ScatterGather List of Parameters.

The Get Parameter Functions shall return specific parameter value. For example, the Logic Dma Channel shall return the “Destination Address” value, which contains the last accessed destination memory location.

3.6.1.2 TRESOS Configuration

Instance

Logic Instance Name

Hardware Instance

Debug ☐

Halt After Error ☐

Global Master ID Replication ☐

Round Robin Channel Arbitration ☐

Global Channel linking ☐

Dma Crc

Name

Enable Swap Bit ☐

Enable Swap Byte ☐

Enable Global ☐

Static Configuration

Figure 3.1 DMA Logic Instance

The DMA Logic Instance configuration presented in Figure 3.1 is static, thus it can't be changed dynamically during runtime. The "Logic Instance Name" represents the Handler(Tag) used with the DMA API. The Handler(Tag) is set by the User for the specific Application; for example the "DMA_LOGIC_INSTANCE_0" shall be changed to "DMA_LOGIC_INST_COMMUNICATION".

Logic Channel Configuration Global Transfer ScatterGather Crc

Logic Channel Name

Hardware Instance

Hardware Channel

Interrupt Callback

Error Interrupt Callback

Enable Global Config ☒

Enable Scatter/Gather ☒

Enable Transfer Config ☐

Enable CRC Config ☐

Static Configuration

Dynamic Configuration

Figure 3.2 DMA Logic Channel

The DMA Logic Channel configuration in Figure 3.2 is composed of static and dynamic settings.

1. The static configuration is needed in order for the DMA Logic Channel to be created and used. The static

configuration allocates resources for the DMA Logic Channel basic configuration. The static configuration can't be changed during runtime. The "Logic Channel Name" represents the Handler(Tag) used with the DMA API. The Handler(Tag) is set by the User based on the Application; For example the "DMA_LOGIC_CH_0" shall be changed to "DMA_CH_CAN0_RX" or "DMA_LOGIC_CH_SPI2_TX".

2. The dynamic configuration can be set in TRESOS and shall be automatically loaded in the DMA Logic Channel at initialization time. By enabling a checkbox, the respective configuration option is enabled and system memory is allocated during generation. The dynamic configuration can be set during runtime by using the specific API. By disabling a checkbox, the respective configuration option is disabled and system memory is not allocated during generation.

2.1. The "Enable Global Config" contains settings outside the Transfer Control Descriptor (TCD).

2.2. The "Enable Transfer Config" contains settings of the Transfer Control Descriptor (TCD).

2.3. The "Enable Scatter/Gather" contains settings that extend the "Transfer Config" by creating Software Transfer Control Descriptors (STCDs).

2.4. The "Enable CRC Config" contains settings for the DMA Logic Channel CRC computation functionality.

Note: The DMA Logic Channel can be configured in "Transfer Mode" or "Scatter/Gather Mode", thus only one of the 2 configurations can be set at any time. During runtime, the DMA Logic Channel can be configured between the 2 modes by calling the "SetTransfer" or "SetScatterGather" API.

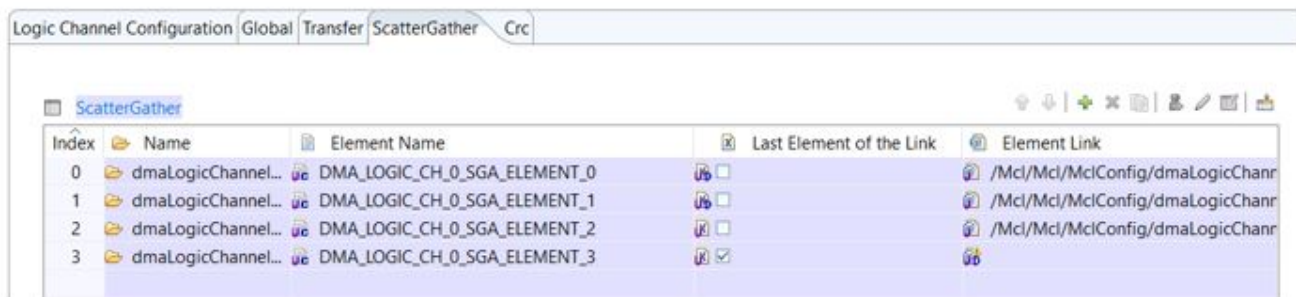


Figure 3.3 DMA Logic Channel – ScatterGather Element List

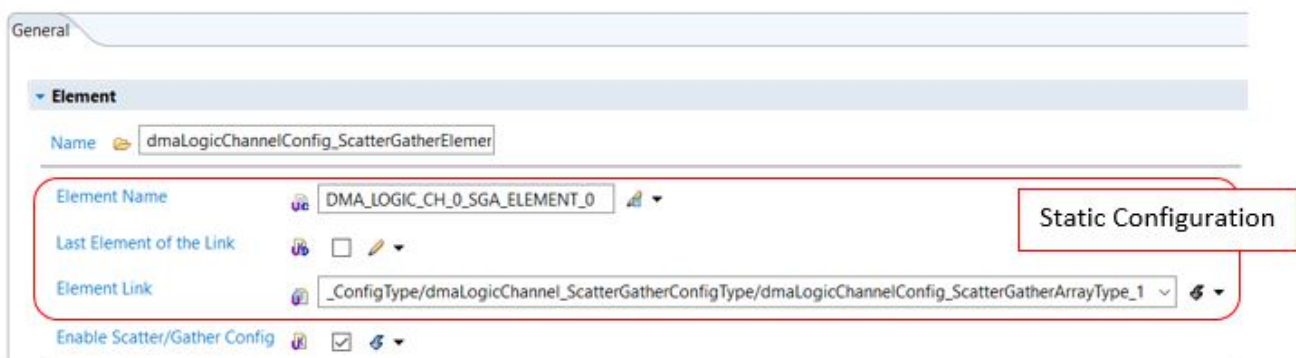


Figure 3.4 DMA Logic Channel – ScatterGather Element Configuration

The DMA Logic Channel ScatterGather Element list (Figure 3.3) shall be loaded with the required number of elements. Each element represents a STCD and is part of a linked list of elements.

The DMA Logic Channel ScatterGather Element Configuration (Figure 3.4) contains the static configuration of the element.

1. The “Element Name” represents the Handler (Tag) (static configuration).
2. The “Last Element of the Link” sets the element as the last link element (static configuration).
3. The “Element Link” points to the next element of the link (static configuration).
4. The “Enable Scatter/Gather Config” enables the element configuration.

Note1: Element can't be added during runtime. The elements are allocated resources during generation (System memory for STCDs and configuration if set). The element linkage can't be changed during runtime.

Note2: The DMA Logic Channel ScatterGather Element List can be configured to contain multiple independent chained lists. During runtime, the Logic Channel can be assigned a different Chained List.

Note3: Each Element is allocated 32 bytes of memory space aligned to 32 bytes, representing the STCD. Additionally, if the configuration is enabled from TRESOS, additional 60 bytes are allocated.

3.6.2 Channel State Machine

3.6.2.1 MCL Driver

The MCL DMA Driver runs based on the presented State Machine Diagram.

The MCL DMA State Machine applies to the MCL DMA Channels and it contains 5 States.

The State Machine supports the following 21 Transitions:

T0: Hardware Reset

T1: Mcl_Init() with DMA Channel no Transfer or Scatter/Gather generated configurations

T2: Mcl_DeInit()

T3: Mcl_SetDmaChannelTransferList()

T4: Mcl_Init() with DMA Channel Transfer generated configuration

T5: Mcl_SetDmaChannelScatterGatherList() + Mcl_SetDmaChannelScatterGatherConfig()

T6: Mcl_SetDmaChannelTransferList()

T7: Mcl_SetDmaChannelScatterGatherList() + Mcl_SetDmaChannelScatterGatherConfig()

T8: Mcl_Init() with DMA Channel Scatter/Gather generated configuration

T9: Detection of channel error

T10: Mcl_SetDmaInstanceCommand(CHANNEL, MCL_DMA_CH_ACK_ERROR)

T11: Detection of channel error

T12: Mcl_DeInit()

T13: Mcl_DeInit()

T14: Detection of channel error

T15: Detection of channel error

T16: Mcl_DeInit()

T17: Mcl_DeInit()

T18: Mcl_SetDmaChannelCommand(CHANNEL, MCL_DMA_CH_ACK_ERROR)

T19: Mcl_SetDmaChannelTransferList()

T20: Mcl_SetDmaChannelScatterGatherList() + Mcl_SetDmaChannelScatterGatherConfig()

T21: Detection of channel error

When the DMA Driver detects a software or hardware error for the Logic Channel, it shall enter the "MCL_DMA_CH_ERROR_STATE".

To exit from the "MCL_DMA_CH_ERROR_STATE", the application shall use the specified transitions.

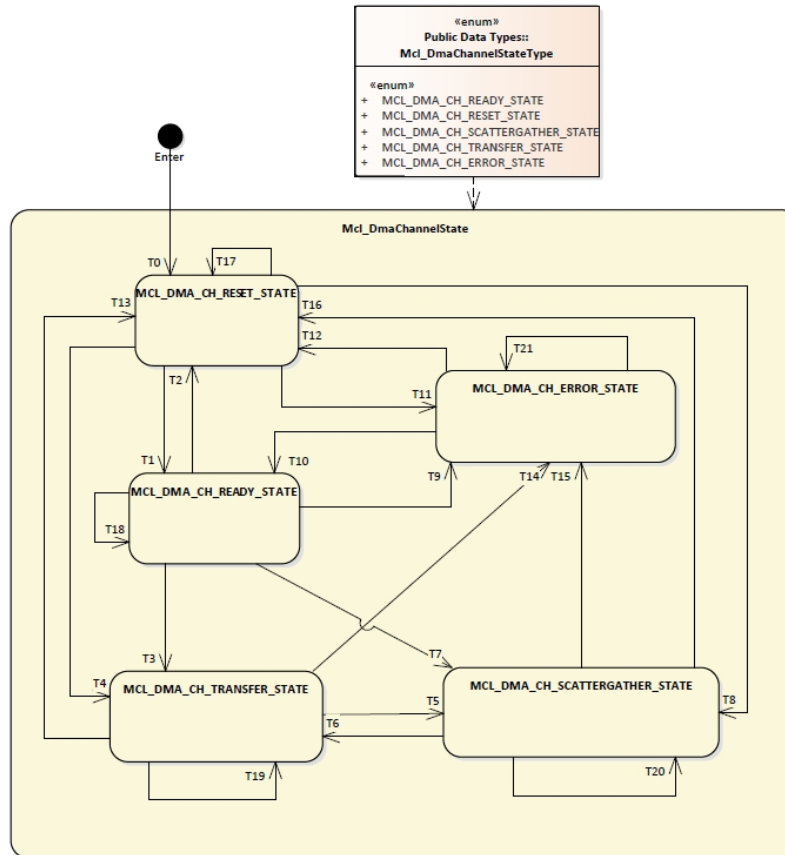


Figure 3.5 MCL DMA Channel State Machine

3.6.2.2 DMA Driver

The DMA IP Driver runs based on the presented State Machine Diagram.

The DMA State Machine applies to the DMA Logic Channels and it contains 5 States.

The State Machine supports the following 21 Transitions:

T0: Hardware Reset

T1: Dma_Ip_LogicChannelInit() with no Transfer or Scatter/Gather generated configurations

T2: Dma_Ip_LogicChannelDeinit()

T3: Dma_Ip_SetLogicChannelTransferList()

T4: Dma_Ip_LogicChannelInit() with Transfer generated configuration

```

T5: Dma_Ip_SetLogicChannelScatterGatherList () + Dma_Ip_SetLogicChannelScatterGatherConfig()
T6: Dma_Ip_SetLogicChannelTransferList ()
T7: Dma_Ip_SetLogicChannelScatterGatherList () + Dma_Ip_SetLogicChannelScatterGatherConfig()
T8: Dma_Ip_LogicChannelInit() with Scatter/Gather generated configuration
T9: Detection of channel error
T10: Dma_Ip_SetLogicChannelCommand(CHANNEL, DMA_IP_CH_CLEAR_ERROR)
T11: Detection of channel error
T12: Dma_Ip_LogicChannelDeinit()
T13: Dma_Ip_LogicChannelDeinit()
T14: Detection of channel error
T15: Detection of channel error
T16: Dma_Ip_LogicChannelDeinit()
T17: Dma_Ip_LogicChannelDeinit()
T18: Dma_Ip_SetLogicChannelCommand(CHANNEL, DMA_IP_CH_CLEAR_ERROR)
T19: Dma_Ip_SetLogicChannelTransferList ()
T20: Dma_Ip_SetLogicChannelScatterGatherList () + Dma_Ip_SetLogicChannelScatterGatherConfig()
T21: Detection of channel error

```

When the DMA Driver detects a software or hardware error for the Logic Channel, it shall enter the "DMA_IP_↔CH_ERROR_STATE".

To exit from the "DMA_IP_CH_ERROR_STATE", the application shall use the specified transitions.

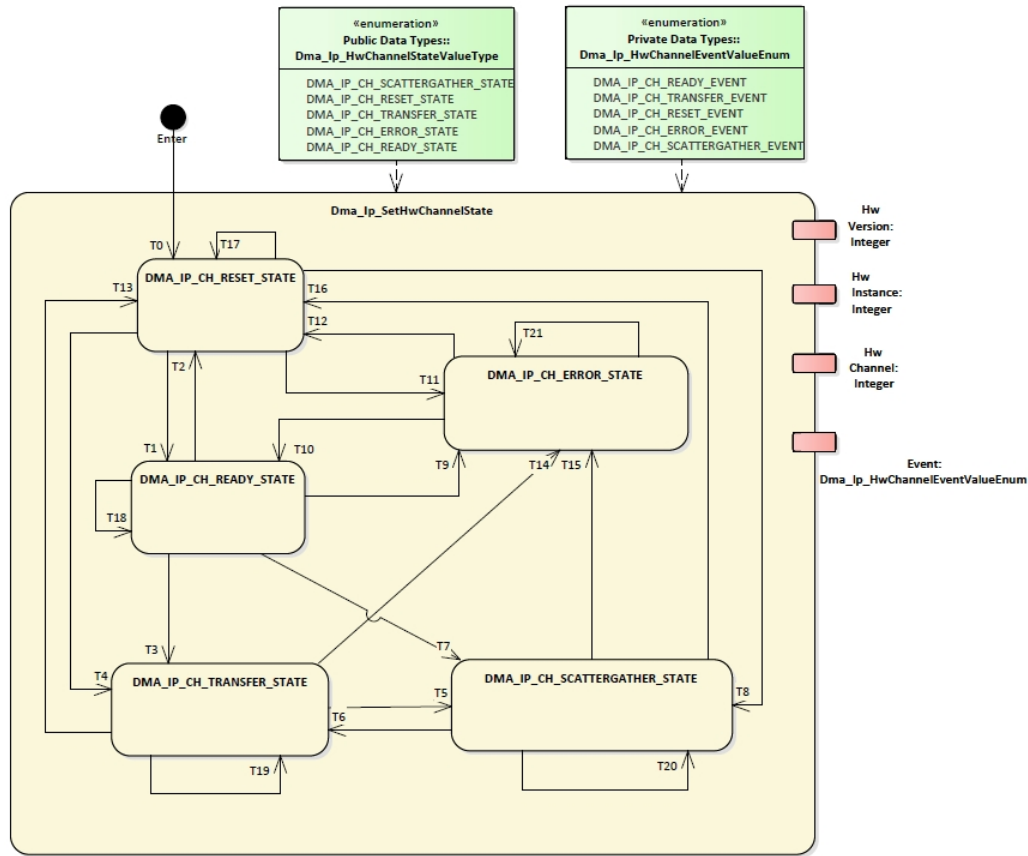


Figure 3.6 DMA IP Channel State Machine

3.7 Runtime errors

The driver generates the following DET errors at runtime.

Function	Error Code	Condition triggering the error
Mcl_Init()	MCL_E_UNINIT	API is called with a NULL pointer as parameter.
Mcl_DeInit()	MCL_E_PARAM_CONFIG	API is called with invalid configuration parameter.
Mcl_DeInit()	MCL_E_UNINIT	API is called with a NULL pointer as parameter.
Mcl_SetDmaInstanceCommand()	MCL_DET_DMA_INSTANCE← _COMMAND	API is called with invalid instance command.
Mcl_GetDmaInstanceStatus()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_SetDmaChannelCommand()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.
Mcl_SetDmaChannelCommand()	MCL_E_INVALID_COMMAND	API is called with invalid command.
Mcl_GetDmaChannelStatus()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.
Mcl_SetDmaChannelGlobalList()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.
Mcl_SetDmaChannelGlobalList()	MCL_E_INVALID_← PARAMETER	API is called with invalid parameter.
Mcl_SetDmaChannelTransferList()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.

Function	Error Code	Condition triggering the error
Mcl_SetDmaChannelTransferList()	MCL_E_INVALID_PARAMETER	API is called with invalid parameter.
Mcl_SetDmaChannelScatterGatherList()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.
Mcl_SetDmaChannelScatterGatherList()	MCL_E_INVALID_PARAMETER	API is called with invalid parameter.
Mcl_GetDmaChannelParam()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.
Mcl_GetDmaChannelParam()	MCL_E_INVALID_PARAMETER	API is called with invalid parameter.
Mcl_SetDmaChannelScatterGatherConfig()	MCL_E_INVALID_CHANNEL	API is called with invalid channel.
Mcl_CacheEnable()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_CacheDisable()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_CacheInvalidate()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_CacheClean()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_CacheInvalidateByAddr()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_CacheCleanByAddr()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.
Mcl_CacheCleanByAddr()	MCL_E_INVALID_INSTANCE	API is called with invalid instance.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

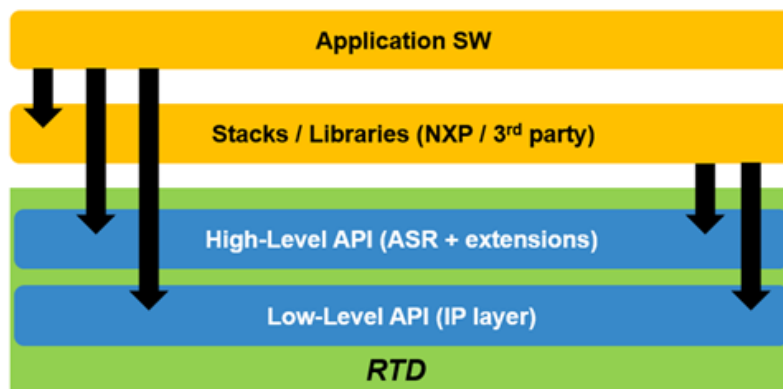


Figure 3.7 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

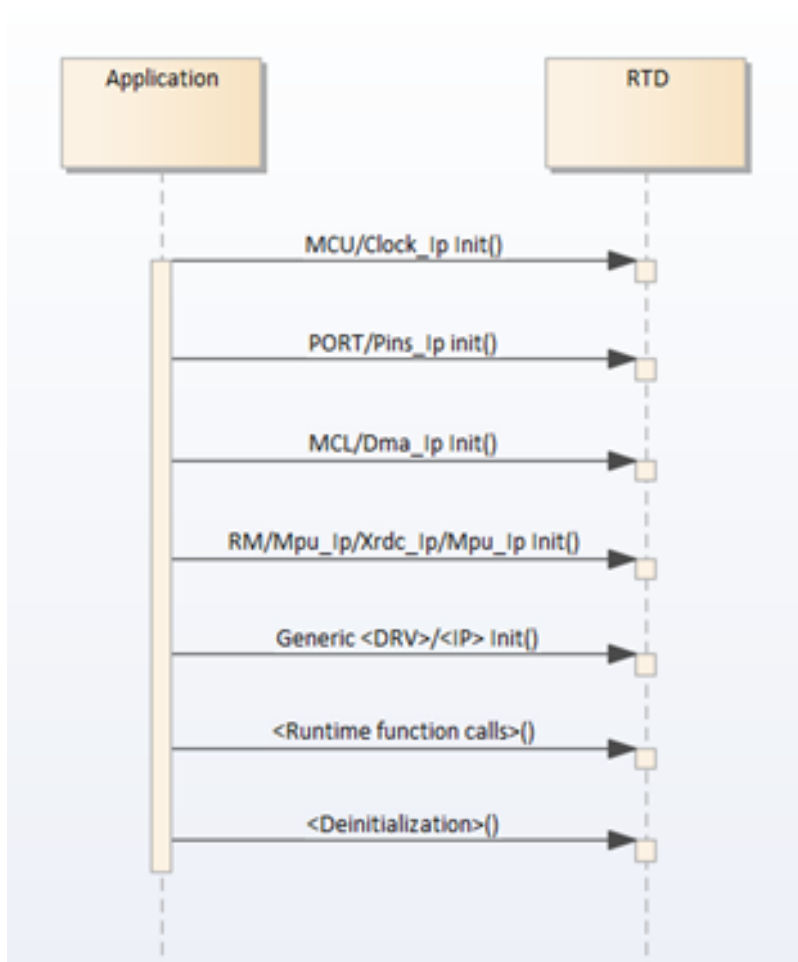


Figure 3.8 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

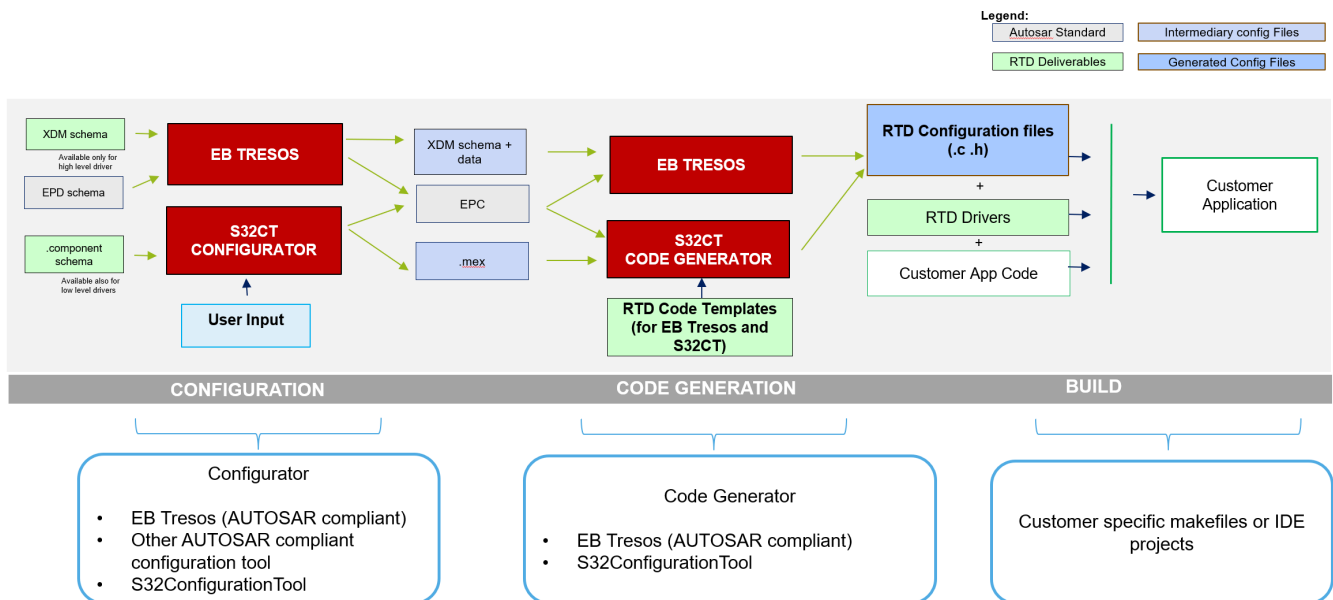


Figure 4.1 Configuration generation flow

- Mcl
- DMA_IP
- CACHE_IP
- TRGMUX_IP

- [FLEXIO_MCL_IP](#)
- [FTM_MCL_IP](#)

4.1 Mcl

- Module [Mcl](#)
 - Container [MclGeneral](#)
 - * Parameter [MclEnableDemReportErrorStatus](#)
 - * Parameter [MclEnableDevErrorDetect](#)
 - * Parameter [Mcl_VersionInfoApi](#)
 - * Parameter [MclEnableUserModeSupport](#)
 - * Parameter [MclEnableMultipartitionSupport](#)
 - * Parameter [MclEnableVirtualAddressMappingSupport](#)
 - * Container [MclDma](#)
 - Parameter [MclEnableDma](#)
 - * Container [MclCache](#)
 - Parameter [MclEnableCache](#)
 - Parameter [MclCacheTimeoutValue](#)
 - Parameter [MclCacheTimeoutMethod](#)
 - * Container [MclTrgMux](#)
 - Parameter [MclEnableTrgMux](#)
 - * Container [MclFlexioCommon](#)
 - Parameter [MclEnableFlexioCommon](#)
 - * Container [MclFtmCommon](#)
 - Parameter [Mcl_FtmCommonTimebase](#)
 - Parameter [Mcl_FtmSplitCommonTimebase](#)
 - Container [MclConfig](#)
 - * Container [MclVirtualMemorySection](#)
 - Parameter [MclVirtualAddressStart](#)
 - Parameter [MclVirtualAddressEnd](#)
 - Parameter [MclPhysicalAddressStart](#)
 - Parameter [MclPhysicalAddressEnd](#)
 - * Container [MclDemEventParameterRefs](#)
 - Reference [MCL_E_TIMEOUT_FAILURE](#)
 - * Container [dmaLogicInstance_ConfigType](#)
 - Parameter [dmaLogicInstance_IdName](#)
 - Parameter [dmaLogicInstance_hwId](#)

- Parameter [dmaLogicInstance_enDebug](#)
- Parameter [dmaLogicInstance_enRoundRobin](#)
- Parameter [dmaLogicInstance_enHaltAfterError](#)
- Parameter [dmaLogicInstance_enChLinking](#)
- Reference [dmaLogicInstance_EcucPartitionRef](#)
- * Container [dmaLogicChannel_Type](#)
 - Parameter [dmaLogicChannel_LogicName](#)
 - Parameter [dmaLogicChannel_HwInstId](#)
 - Parameter [dmaLogicChannel_HwChId](#)
 - Parameter [dmaLogicChannel_InterruptCallback](#)
 - Parameter [dmaLogicChannel_ErrorInterruptCallback](#)
 - Parameter [dmaLogicChannel_EnableGlobalConfig](#)
 - Parameter [dmaLogicChannel_EnableTransferConfig](#)
 - Parameter [dmaLogicChannel_EnableScatterGather](#)
 - Reference [dmaLogicChannel_EcucPartitionRef](#)
 - Container [dmaLogicChannel_ConfigType](#)
 - Container [dmaLogicChannel_GlobalConfigType](#)
 - Container [dmaLogicChannelConfig_GlobalRequestType](#)
 - Parameter [dmaGlobalRequest_enDmaRequest](#)
 - Container [dmaLogicChannelConfig_GlobalInterruptType](#)
 - Parameter [dmaGlobalInterrupt_enDmaErrorInterrupt](#)
 - Container [dmaLogicChannelConfig_GlobalPriorityType](#)
 - Parameter [dmaGlobalPriority_LevelPriority](#)
 - Parameter [dmaGlobalPriority_enPreemption](#)
 - Parameter [dmaGlobalPriority_disPreempt](#)
 - Container [dmaLogicChannel_TransferConfigType](#)
 - Container [dmaLogicChannelConfig_TransferControlType](#)
 - Parameter [dmaLogicChannelConfig_enDmaMajorInterrupt](#)
 - Parameter [dmaLogicChannelConfig_enDmaHalfMajorInterrupt](#)
 - Parameter [dmaLogicChannelConfig_disDmaAutoHwReq](#)
 - Parameter [dmaLogicChannelConfig_bandwidthControl](#)
 - Parameter [dmaLogicChannelConfig_ScatterGatherAddressType](#)
 - Parameter [dmaLogicChannelConfig_DestinationStoreAddressType](#)
 - Container [dmaLogicChannelConfig_TransferSourceType](#)
 - Parameter [dmaLogicChannelConfig_SourceAddressType](#)
 - Parameter [dmaLogicChannelConfig_SourceSignedOffsetType](#)

- Parameter [dmaLogicChannelConfig_SourceLastAddressAdjustmentType](#)
- Parameter [dmaTransferConfig_TransferSizeType](#)
- Parameter [dmaLogicChannelConfig_SourceModuloType](#)
- Container [dmaLogicChannelConfig_TransferDestinationType](#)
 - Parameter [dmaLogicChannelConfig_DestinationAddressType](#)
 - Parameter [dmaLogicChannelConfig_DestinationSignedOffsetType](#)
 - Parameter [dmaLogicChannelConfig_DestinationLastAddressAdjustmentType](#)
 - Parameter [dmaTransferConfig_TransferSizeType](#)
 - Parameter [dmaLogicChannelConfig_DestinationModuloType](#)
- Container [dmaLogicChannelConfig_TransferMinorLoopType](#)
 - Parameter [dmaLogicChannelConfig_enSourceOffset](#)
 - Parameter [dmaLogicChannelConfig_enDestinationOffset](#)
 - Parameter [dmaLogicChannelConfig_OffsetValueType](#)
 - Parameter [dmaLogicChannelConfig_enMinorLoopLinkCh](#)
 - Parameter [dmaLogicChannelConfig_MinorLoopSizeType](#)
 - Reference [dynamic_dmaLogicChannelConfig_MinorLoopLinkChValueType](#)
- Container [dmaLogicChannelConfig_TransferMajorLoopType](#)
 - Parameter [dmaLogicChannelConfig_enMajorLoopLinkCh](#)
 - Parameter [dmaLogicChannelConfig_MajorLoopCountType](#)
 - Reference [dynamic_dmaLogicChannelConfig_MajorLoopLinkChValueType](#)
- Container [dmaLogicChannel_ScatterGatherConfigType](#)
- Container [dmaLogicChannelConfig_ScatterGatherArrayType](#)
 - Container [dmaLogicChannelConfig_ScatterGatherElementConfigType](#)
 - Parameter [dmaLogicChannelConfig_ScatterGatherElementNameType](#)
 - Parameter [dmaLogicChannelConfig_LastElementLink_ScatterGatherType](#)
 - Parameter [dmaLogicChannelConfig_enScatterGatherConfig](#)
 - Reference [dynamic_dmaLogicChannelConfig_BasicElementLink_ScatterGatherType](#)
 - Container [dmaLogicChannelConfig_TransferControlType](#)
 - Parameter [dmaLogicChannelConfig_enStart](#)
 - Parameter [dmaLogicChannelConfig_enDmaMajorInterrupt](#)
 - Parameter [dmaLogicChannelConfig_enDmaHalfMajorInterrupt](#)
 - Parameter [dmaLogicChannelConfig_disDmaAutoHwReq](#)
 - Parameter [dmaLogicChannelConfig_bandwidthControl](#)
 - Parameter [dmaLogicChannelConfig_ScatterGatherAddressType](#)
 - Parameter [dmaLogicChannelConfig_DestinationStoreAddressType](#)
 - Container [dmaLogicChannelConfig_TransferSourceType](#)

- Parameter [dmaLogicChannelConfig_SourceAddressType](#)
- Parameter [dmaLogicChannelConfig_SourceSignedOffsetType](#)
- Parameter [dmaLogicChannelConfig_SourceLastAddressAdjustmentType](#)
- Parameter [dmaTransferConfig_TransferSizeType](#)
- Parameter [dmaLogicChannelConfig_SourceModuloType](#)
- Container [dmaLogicChannelConfig_TransferDestinationType](#)
 - Parameter [dmaLogicChannelConfig_DestinationAddressType](#)
 - Parameter [dmaLogicChannelConfig_DestinationSignedOffsetType](#)
 - Parameter [dmaLogicChannelConfig_DestinationLastAddressAdjustmentType](#)
 - Parameter [dmaTransferConfig_TransferSizeType](#)
 - Parameter [dmaLogicChannelConfig_DestinationModuloType](#)
- Container [dmaLogicChannelConfig_TransferMinorLoopType](#)
 - Parameter [dmaLogicChannelConfig_enSourceOffset](#)
 - Parameter [dmaLogicChannelConfig_enDestinationOffset](#)
 - Parameter [dmaLogicChannelConfig_OffsetValueType](#)
 - Parameter [dmaLogicChannelConfig_enMinorLoopLinkCh](#)
 - Parameter [dmaLogicChannelConfig_MinorLoopSizeType](#)
 - Reference [dynamic_dmaLogicChannelConfig_MinorLoopLinkChValueType](#)
- Container [dmaLogicChannelConfig_TransferMajorLoopType](#)
 - Parameter [dmaLogicChannelConfig_enMajorLoopLinkCh](#)
 - Parameter [dmaLogicChannelConfig_MajorLoopCountType](#)
 - Reference [dynamic_dmaLogicChannelConfig_MajorLoopLinkChValueType](#)
- * Container [trgmuxInstaceList](#)
 - Parameter [trgmuxHardwareInstance](#)
- * Container [trgmuxLogicGroup](#)
 - Parameter [trgmuxLogicGroupHardwareInstance](#)
 - Parameter [trgmuxLogicGroup_Name](#)
 - Parameter [trgmuxLogicGroup_Lock](#)
 - Container [trgmuxLogicTrigger](#)
 - Parameter [trgmuxLogicTrigger_Name](#)
 - Parameter [trgmuxLogicTrigger_Output](#)
 - Parameter [trgmuxLogicTrigger_Input](#)
 - Reference [trgmuxLogicTrigger_EcucPartitionRef](#)
- * Container [FlexioCommon](#)
 - Parameter [FlexioMcInstances](#)
 - Parameter [FlexioDebugEnabled](#)

- Container [FlexioMclLogicChannels](#)
 - Parameter [FlexioMclChannelId](#)
 - Parameter [FlexioMclPinId](#)
 - Parameter [FlexioMclAddPinEnable](#)
 - Parameter [FlexioMclAddPinId](#)
 - Parameter [FlexioMclAddChannelEnable](#)
 - Parameter [FlexioMclAddChannelId](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorId](#)

4.1.1 Module Mcl

Vendor specific: Configuration of the Mcl (MicroController Library) module.

Included containers:

- [MclGeneral](#)
- [MclConfig](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container MclGeneral

Vendor specific: Configuration of general Mcl parameters.

Included subcontainers:

- [MclDma](#)
- [MclCache](#)
- [MclTrgMux](#)
- [MclFlexioCommon](#)

Configuration Parameters

- [MclFtmCommon](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter MclEnableDemReportErrorStatus

Enable/Disable the Production Error Reporting (DEM).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.4 Parameter MclEnableDevErrorDetect

Vendor specific:

Enable/Disable the Development Error Detection (DET).

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.5 Parameter Mcl_VersionInfoApi

Vendor specific: Enables/Disables the get version info API function

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.6 Parameter MclEnableUserModeSupport

When this parameter is enabled, the MCL module will adapt to run from User Mode, with the following measures:

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.7 Parameter MclEnableMultipartitionSupport

Enable/Disable MultiPartition support

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.8 Parameter McIEnableVirtualAddressMappingSupport

Enable/Disable Virtual Address Mapping support

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.9 Container McIDma

Vendor specific:

Container for the Dma related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.10 Parameter McIEnableDma

Vendor specific: Enable/Disable DMA support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.11 Container MclCache

Vendor specific:

Container for the CACHE related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.12 Parameter MclEnableCache

Vendor specific:

Enable/Disable all CACHE support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.13 Parameter MclCacheTimeoutValue

Set Cache timeout value

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	4294967295
min	0

4.1.14 Parameter MclCacheTimeoutMethod

MclCacheTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.15 Container MclTrgMux

Vendor specific:

Container for the TRGMUX related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.16 Parameter MclEnableTrgMux

Vendor specific: Enable/Disable TRGMUX support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.17 Container MclFlexioCommon

Vendor specific:

Container for the Flexio Common related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.18 Parameter MclEnableFlexioCommon

Vendor specific: Enable/Disable Flexio common support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.19 Container MclFtmCommon

Vendor specific:

Container for the FTM Common related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.20 Parameter Mcl_FtmCommonTimebase

Enables/Disables the option to set the a common timebase for multiple FTM modules.

Note:

Note This is an Implementation Specific Parameter. Enabling this feature will allow the use of the Mcl_SelectCommonTimebase API

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.21 Parameter Mcl_FtmSplitCommonTimebase

Enables/Disables the option to use the split common timebase feature.

Note:

Note This is an Implementation Specific Parameter. Enabling this feature will allow the use of the Mcl_SplitCommonTimebase API

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.22 Container MclConfig

Vendor specific: This container is the base for a multiple configuration set

Included subcontainers:

- [MclVirtualMemorySection](#)
- [MclDemEventParameterRefs](#)
- [dmaLogicInstance_ConfigType](#)
- [dmaLogicChannel_Type](#)
- [trgmuxInstaceList](#)
- [trgmuxLogicGroup](#)
- [FlexioCommon](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.23 Container MciVirtualMemorySection

Vendor specific:

Data to configure Virtual address and Physical address.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.24 Parameter MciVirtualAddressStart

This parameter represents the Virtual Address Start.

The address must be aligned to 4 bytes.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	4294967295
min	0

4.1.25 Parameter MciVirtualAddressEnd

This parameter represents the Virtual Address End.

The address must be aligned to 4 bytes.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	255
max	4294967295
min	0

4.1.26 Parameter MclPhysicalAddressStart

This parameter represents the Physical Address Start.

The address must be aligned to 4 bytes.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	4294967295
min	0

4.1.27 Parameter MclPhysicalAddressEnd

This parameter represents the Physical Address End.

The address must be aligned to 4 bytes.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	255
max	4294967295
min	0

4.1.28 Container McIdemEventParameterRefs

Container for the references to *DemEventParameter* elements which shall be invoked using the API *Dem_ReportErrorStatus* API in case the corresponding error occurs.

The *EventId* is taken from the referenced *DemEventParameter*'s *DemEventId* value.

The standardized errors are provided in the container and can be extended by vendor specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE

4.1.29 Reference MCL_E_TIMEOUT_FAILURE

Reference to configured DEM event to report Timeout failure.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.1.30 Container dmaLogicInstance__ConfigType

Vendor specific: Configuration of a DMA Instance.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.31 Parameter dmaLogicInstance__IdName

Vendor specific:

Logic Instance Name.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DMA_LOGIC_INST_0

4.1.32 Parameter dmaLogicInstance__hwId

Vendor specific:

Select the Hardware DMA Instance.

NOTE: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	DMA_IP_HW_INST_0
literals	['DMA_IP_HW_INST_0']

4.1.33 Parameter dmaLogicInstance_enDebug

Vendor specific:

DMA_CR[EDBG].

Enable Debug.

0 - The assertion of the system debug control input is ignored.

1 - The assertion of the system debug control input causes the eDMA to stall the start of a new channel.

Executing channels are allowed to complete.

Channel execution will resume when either the system debug control input is negated or the EDBG bit is cleared.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.34 Parameter dmaLogicInstance_enRoundRobin

Vendor specific:

DMA_CR[ERCA].

Enable Round Robin Channel Arbitration.

0 - Fixed-priority arbitration is used for channel selection within each group.

1 - Round-Robin arbitration is used for channel selection within each group.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.35 Parameter dmaLogicInstance_enHaltAfterError

Vendor specific:

DMA_CR[HOE] for eDMA2 instances or DMA_CSR[HAE] for eDMA3 instances.

Halt On/After Error.

0 - Normal operation.

1 - Any error will cause the HALT bit to be set.

Subsequently, all service requests will be ignored until the HALT bit is cleared.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.36 Parameter dmaLogicInstance_enChLinking

Vendor specific:

Configuration Parameters

DMA_CR[GCLC].

Global Channel Linking Control.

0 - Channel linking is disabled for all channels.

1 - Channel linking is available and controlled by each channel's link settings.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.37 Reference dmaLogicInstance_EcucPartitionRef

Maps a DMA instance to zero or multiple ECUC partitions to limit the access to this mappable element.

The ECUC partitions referenced are a subset of the ECUC partitions where the MCL driver is mapped to

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.38 Container dmaLogicChannel_Type

Vendor specific:

Logic Channel Configuration.

Included subcontainers:

- [dmaLogicChannel_ConfigType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	16
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.39 Parameter dmaLogicChannel_LogicName

Vendor specific:

Logic Channel Name.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	DMA_LOGIC_CH_0

4.1.40 Parameter dmaLogicChannel_HwInstId

Vendor specific:

Select the Hardware DMA Instance.

NOTE: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_HW_INST_0
literals	['DMA_IP_HW_INST_0']

4.1.41 Parameter dmaLogicChannel_HwChId

Vendor specific:

Select the physical eDMA Channel.

NOTE: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_HW_CH_0
literals	['DMA_IP_HW_CH_0', 'DMA_IP_HW_CH_1', 'DMA_IP_HW_CH_↵ 2', 'DMA_IP_HW_CH_3', 'DMA_IP_HW_CH_4', 'DMA_IP_HW_CH_↵ 5', 'DMA_IP_HW_CH_6', 'DMA_IP_HW_CH_7', 'DMA_IP_HW_CH_↵ 8', 'DMA_IP_HW_CH_9', 'DMA_IP_HW_CH_10', 'DMA_IP_HW_CH↵ _11', 'DMA_IP_HW_CH_12', 'DMA_IP_HW_CH_13', 'DMA_IP_HW_↵ CH_14', 'DMA_IP_HW_CH_15']

4.1.42 Parameter dmaLogicChannel_InterruptCallback

Vendor specific:

User callback function to report that the transfer is half or complete depending on configuration.

NOTE: Use NULL_PTR w/o quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.43 Parameter dmaLogicChannel_ErrorInterruptCallback

Vendor specific:

User callback function

NOTE: Use NULL_PTR w/o quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.44 Parameter dmaLogicChannel_EnableGlobalConfig

Vendor specific: Enable and allocate memory for Global Configuration. Global Configuration can be configured during generation time or during runtime.

Note: If the check box is enabled, then memory space is allocated for the configuration.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.45 Parameter dmaLogicChannel_EnableTransferConfig

Vendor specific: Enable and allocate memory for Transfer Configuration. Transfer Configuration can be configured during generation time or during runtime.

Note: If the check box is enabled, then memory space is allocated for the configuration.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.46 Parameter dmaLogicChannel_EnableScatterGather

Vendor specific: Enable and allocate memory for ScatterGather Transfer Mode.

The ScatterGather Transfer Mode shall allocate memory for each Element, comprised of: Element Linkage and Element Software TCD.

The Element allocation can be done only in the configurator.

The Element Configuration can be further enabled for each individual element.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.47 Reference dmaLogicChannel_EcucPartitionRef

Maps a DMA channel to zero or multiple ECUC partitions to limit the access to this mappable element.

The ECUC partitions referenced are a subset of the ECUC partitions where the MCL driver is mapped to

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.48 Container dmaLogicChannel_ConfigType

Vendor specific: Logic Channel Configuration.

Included subcontainers:

- [dmaLogicChannel_GlobalConfigType](#)
- [dmaLogicChannel_TransferConfigType](#)
- [dmaLogicChannel_ScatterGatherConfigType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.49 Container dmaLogicChannel_GlobalConfigType

Vendor specific:

Logic Channel Global Configuration.

Included subcontainers:

- [dmaLogicChannelConfig_GlobalRequestType](#)
- [dmaLogicChannelConfig_GlobalInterruptType](#)
- [dmaLogicChannelConfig_GlobalPriorityType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.50 Container dmaLogicChannelConfig_GlobalRequestType

Vendor specific:

TCD Request Control.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.51 Parameter dmaGlobalRequest_enDmaRequest

Vendor specific: Enable the Dma Channel Hardware Request.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.52 Container dmaLogicChannelConfig_GlobalInterruptType

Vendor specific:

TCD Global Interrupt.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.53 Parameter dmaGlobalInterrupt_enDmaErrorInterrupt

Vendor specific: Enable the Dma Channel Error Interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.54 Container dmaLogicChannelConfig_GlobalPriorityType

Vendor specific:

TCD Global Interrupt.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.55 Parameter dmaGlobalPriority_LevelPriority

Vendor specific: Set the Dma Channel Level Priority.

Note:

- If you want to change this value, you must Enable Global Config on "Logic Channel Configuration" tag.

- When have larger than one configured channel, the selected priority shall be from the pool of the configured channels, maintaining priority uniqueness. Ex: the user configures 3 channels: 0, 5, 15, by default the channel priority: 0-0, 5-5, 15-15. But the user wants to changes their channel priorities, the value only is selected one of 3 values(0, 5, 15) and they must be unique.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Configuration Parameters

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_LEVEL_PRI00
literals	['DMA_IP_LEVEL_PRI00', 'DMA_IP_LEVEL_PRI01', 'DMA_IP_↵ LEVEL_PRI02', 'DMA_IP_LEVEL_PRI03', 'DMA_IP_LEVEL_PRI04', 'DMA_IP_LEVEL_PRI05', 'DMA_IP_LEVEL_PRI06', 'DMA_IP_↵ LEVEL_PRI07', 'DMA_IP_LEVEL_PRI08', 'DMA_IP_LEVEL_PRI09', 'DMA_IP_LEVEL_PRI010', 'DMA_IP_LEVEL_PRI011', 'DMA_IP_↵ LEVEL_PRI012', 'DMA_IP_LEVEL_PRI013', 'DMA_IP_LEVEL_↵ PRI014', 'DMA_IP_LEVEL_PRI015']

4.1.56 Parameter dmaGlobalPriority_enPreemption

Vendor specific: Enable the Dma Channel Preemption.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.57 Parameter dmaGlobalPriority_disPreempt

Vendor specific: Disable the Dma Channel Preempt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.58 Container dmaLogicChannel__TransferConfigType

Vendor specific:

Logic Channel Transfer Configuration.

Included subcontainers:

- [dmaLogicChannelConfig__TransferControlType](#)
- [dmaLogicChannelConfig__TransferSourceType](#)
- [dmaLogicChannelConfig__TransferDestinationType](#)
- [dmaLogicChannelConfig__TransferMinorLoopType](#)
- [dmaLogicChannelConfig__TransferMajorLoopType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.59 Container dmaLogicChannelConfig__TransferControlType

Vendor specific:

TCD Global Interrupt.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.60 Parameter dmaLogicChannelConfig_enDmaMajorInterrupt

Vendor specific: Enable the Dma Channel major interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.61 Parameter dmaLogicChannelConfig_enDmaHalfMajorInterrupt

Vendor specific: Enable the Dma Channel half major interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.62 Parameter dmaLogicChannelConfig_disDmaAutoHwReq

Vendor specific: Disable the Dma Channel automatic request.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.63 Parameter dmaLogicChannelConfig_bandwidthControl

Vendor specific: Set the Dma Channel bandwidth control.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_BWC_ENGINE_NO_STALL
literals	['DMA_IP_BWC_ENGINE_NO_STALL', 'DMA_IP_BWC_ENGINE_4←CYCLE_STALL', 'DMA_IP_BWC_ENGINE_8CYCLE_STALL']

4.1.64 Parameter dmaLogicChannelConfig_ScatterGatherAddressType

Vendor specific: ScatterGather Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0U

4.1.65 Parameter dmaLogicChannelConfig_DestinationStoreAddressType

Vendor specific: Destination Store Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0U

4.1.66 Container dmaLogicChannelConfig_TransferSourceType

Vendor specific: Transfer Source

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.67 Parameter dmaLogicChannelConfig_SourceAddressType

Vendor specific: Source Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0

4.1.68 Parameter dmaLogicChannelConfig_SourceSignedOffsetType

Vendor specific: Set the Dma Channel source signed offset value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	32767
min	-32767

4.1.69 Parameter dmaLogicChannelConfig_SourceLastAddressAdjustmentType

Vendor specific: Set the Dma Channel source signed last address adjustment.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	2147483647
min	-2147483647

4.1.70 Parameter dmaTransferConfig_TransferSizeType

Vendor specific: Set the Dma Channel source transfer size.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Configuration Parameters

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_TRANSFER_SIZE_1_BYTE
literals	['DMA_IP_TRANSFER_SIZE_1_BYTE', 'DMA_IP_TRANSFER_SIZE_2_BYTE', 'DMA_IP_TRANSFER_SIZE_4_BYTE', 'DMA_IP_TRANSFER_SIZE_16_BYTE', 'DMA_IP_TRANSFER_SIZE_32_BYTE']

4.1.71 Parameter dmaLogicChannelConfig_SourceModuloType

Vendor specific: Set the Dma Channel source modulo.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.1.72 Container dmaLogicChannelConfig_TransferDestinationType

Vendor specific: Transfer Destination

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.73 Parameter dmaLogicChannelConfig_DestinationAddressType

Vendor specific: Destination Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0

4.1.74 Parameter dmaLogicChannelConfig_DestinationSignedOffsetType

Vendor specific: Set the Dma Channel destination signed offset value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	32767
min	-32767

4.1.75 Parameter dmaLogicChannelConfig_DestinationLastAddressAdjustmentType

Vendor specific: Set the Dma Channel destination signed last address adjustment.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	2147483647
min	-2147483647

4.1.76 Parameter dmaTransferConfig_TransferSizeType

Vendor specific: Set the Dma Channel destination transfer size.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_TRANSFER_SIZE_1_BYTE
literals	['DMA_IP_TRANSFER_SIZE_1_BYTE', 'DMA_IP_TRANSFER_SIZE_2_BYTE', 'DMA_IP_TRANSFER_SIZE_4_BYTE', 'DMA_IP_TRANSFER_SIZE_16_BYTE', 'DMA_IP_TRANSFER_SIZE_32_BYTE']

4.1.77 Parameter dmaLogicChannelConfig_DestinationModuloType

Vendor specific: Set the Dma Channel destination modulo.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.1.78 Container dmaLogicChannelConfig_TransferMinorLoopType

Vendor specific: Transfer Minor Loop

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.79 Parameter dmaLogicChannelConfig_enSourceOffset

Vendor specific: Enable the Dma Channel minor loop source offset.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.80 Parameter dmaLogicChannelConfig_enDestinationOffset

Vendor specific: Enable the Dma Channel minor loop destination offset.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.81 Parameter dmaLogicChannelConfig_OffsetValueType

Vendor specific: Set the Dma Channel minor loop signed offset.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	1048575
min	-1048575

4.1.82 Parameter dmaLogicChannelConfig_enMinorLoopLinkCh

Vendor specific: Enable the Dma Channel minor loop logic channel linking.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.83 Parameter dmaLogicChannelConfig_MinorLoopSizeType

Vendor specific: Set the Dma Channel minor loop transfer size.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	1073741823
min	0

4.1.84 Reference dynamic_dmaLogicChannelConfig_MinorLoopLinkChValueType

Vendor specific: Set the Dma Channel minor loop logic channel link.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.1.85 Container dmaLogicChannelConfig_TransferMajorLoopType

Vendor specific: Transfer Major Loop

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.86 Parameter dmaLogicChannelConfig_enMajorLoopLinkCh

Vendor specific: Enable the Dma Channel major loop logic channel linking.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.87 Parameter dmaLogicChannelConfig_MajorLoopCountType

Vendor specific: Sets the Dma Channel major loop count.

If minor loop channel linking is enabled, the major loop count limit is between 0 and 511.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	1
max	32767
min	0

4.1.88 Reference dynamic_dmaLogicChannelConfig_MajorLoopLinkChValueType

Vendor specific: Set the Dma Channel major loop logic channel link.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.1.89 Container dmaLogicChannel_ScatterGatherConfigType

Vendor specific:

Logic Channel ScatterGather Configuration.

Included subcontainers:

- [dmaLogicChannelConfig_ScatterGatherArrayType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.90 Container dmaLogicChannelConfig_ScatterGatherArrayType

Vendor specific: Logic Channel ScatterGather Configuration.

Included subcontainers:

- [dmaLogicChannelConfig_ScatterGatherElementConfigType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	256
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.91 Container dmaLogicChannelConfig_ScatterGatherElementConfigType

Vendor specific: Element

Included subcontainers:

- [dmaLogicChannelConfig_TransferControlType](#)
- [dmaLogicChannelConfig_TransferSourceType](#)
- [dmaLogicChannelConfig_TransferDestinationType](#)

Configuration Parameters

- [dmaLogicChannelConfig_TransferMinorLoopType](#)
- [dmaLogicChannelConfig_TransferMajorLoopType](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.92 Parameter dmaLogicChannelConfig_ScatterGatherElementNameType

Vendor specific: Element Name

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0_SGA_ELEMENT_0

4.1.93 Parameter dmaLogicChannelConfig_LastElementLink_ScatterGatherType

Vendor specific: For non-circular lists, the last element shall have this checkbox set.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.94 Parameter dmaLogicChannelConfig_enScatterGatherConfig

Vendor specific: Enable Scatter/Gather Configuration

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.95 Reference

dynamic_dmaLogicChannelConfig_BasicElementLink_ScatterGatherType

Vendor specific: Element Link. Elements shall be part of the same Logic Channel.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type/dmaLogicChannel_ConfigType/dmaLogicChannel_ScatterGatherConfigType/dmaLogicChannelConfig_ScatterGatherArrayType

4.1.96 Container dmaLogicChannelConfig_TransferControlType

Vendor specific:

TCD Global Interrupt.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.97 Parameter dmaLogicChannelConfig_enStart

Vendor specific: Enable the Dma Channel start service request (software request).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.98 Parameter dmaLogicChannelConfig_enDmaMajorInterrupt

Vendor specific: Enable the Dma Channel major interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.99 Parameter dmaLogicChannelConfig_enDmaHalfMajorInterrupt

Vendor specific: Enable the Dma Channel half major interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.100 Parameter dmaLogicChannelConfig_disDmaAutoHwReq

Vendor specific: Disable the Dma Channel automatic request.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.101 Parameter dmaLogicChannelConfig_bandwidthControl

Vendor specific: Set the Dma Channel bandwidth control.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Configuration Parameters

Property	Value
defaultValue	DMA_IP_BWC_ENGINE_NO_STALL
literals	['DMA_IP_BWC_ENGINE_NO_STALL', 'DMA_IP_BWC_ENGINE_4←CYCLE_STALL', 'DMA_IP_BWC_ENGINE_8CYCLE_STALL']

4.1.102 Parameter dmaLogicChannelConfig_ScatterGatherAddressType

Vendor specific: ScatterGather Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0U

4.1.103 Parameter dmaLogicChannelConfig_DestinationStoreAddressType

Vendor specific: Destination Store Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0U

4.1.104 Container dmaLogicChannelConfig_TransferSourceType

Vendor specific: Transfer Source

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.105 Parameter dmaLogicChannelConfig_SourceAddressType

Vendor specific: Source Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0

4.1.106 Parameter dmaLogicChannelConfig_SourceSignedOffsetType

Vendor specific: Set the Dma Channel source signed offset value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	32767
min	-32767

4.1.107 Parameter dmaLogicChannelConfig_SourceLastAddressAdjustmentType

Vendor specific: Set the Dma Channel source signed last address adjustment.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	2147483647
min	-2147483647

4.1.108 Parameter dmaTransferConfig_TransferSizeType

Vendor specific: Set the Dma Channel source transfer size.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_TRANSFER_SIZE_1_BYTE
literals	['DMA_IP_TRANSFER_SIZE_1_BYTE', 'DMA_IP_TRANSFER_↵ SIZE_2_BYTE', 'DMA_IP_TRANSFER_SIZE_4_BYTE', 'DMA_IP_↵ TRANSFER_SIZE_16_BYTE', 'DMA_IP_TRANSFER_SIZE_32_BYTE']

4.1.109 Parameter dmaLogicChannelConfig_SourceModuloType

Vendor specific: Set the Dma Channel source modulo.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.1.110 Container dmaLogicChannelConfig_TransferDestinationType

Vendor specific: Transfer Destination

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.111 Parameter dmaLogicChannelConfig_DestinationAddressType

Vendor specific: Destination Address

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0

4.1.112 Parameter dmaLogicChannelConfig_DestinationSignedOffsetType

Vendor specific: Set the Dma Channel destination signed offset value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	32767
min	-32767

4.1.113 Parameter dmaLogicChannelConfig_DestinationLastAddressAdjustmentType

Vendor specific: Set the Dma Channel destination signed last address adjustment.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	2147483647
min	-2147483647

4.1.114 Parameter dmaTransferConfig_TransferSizeType

Vendor specific: Set the Dma Channel destination transfer size.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DMA_IP_TRANSFER_SIZE_1_BYTE
literals	['DMA_IP_TRANSFER_SIZE_1_BYTE', 'DMA_IP_TRANSFER_SIZE_2_BYTE', 'DMA_IP_TRANSFER_SIZE_4_BYTE', 'DMA_IP_TRANSFER_SIZE_16_BYTE', 'DMA_IP_TRANSFER_SIZE_32_BYTE']

4.1.115 Parameter dmaLogicChannelConfig_DestinationModuloType

Vendor specific: Set the Dma Channel destination modulo.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.1.116 Container dmaLogicChannelConfig_TransferMinorLoopType

Vendor specific: Transfer Minor Loop

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.117 Parameter dmaLogicChannelConfig_enSourceOffset

Vendor specific: Enable the Dma Channel minor loop source offset.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.118 Parameter dmaLogicChannelConfig_enDestinationOffset

Vendor specific: Enable the Dma Channel minor loop destination offset.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.119 Parameter dmaLogicChannelConfig_OffsetValueType

Vendor specific: Set the Dma Channel minor loop signed offset.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	1048575
min	-1048575

4.1.120 Parameter dmaLogicChannelConfig_enMinorLoopLinkCh

Vendor specific: Enable the Dma Channel minor loop logic channel linking.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.121 Parameter dmaLogicChannelConfig_MinorLoopSizeType

Vendor specific: Set the Dma Channel minor loop transfer size.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	1073741823
min	0

4.1.122 Reference `dynamic_dmaLogicChannelConfig_MinorLoopLinkChValueType`

Vendor specific: Set the Dma Channel minor loop logic channel link.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.1.123 Container `dmaLogicChannelConfig_TransferMajorLoopType`

Vendor specific: Transfer Major Loop

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.124 Parameter `dmaLogicChannelConfig_enMajorLoopLinkCh`

Vendor specific: Enable the Dma Channel major loop logic channel linking.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
default Value	false

4.1.125 Parameter dmaLogicChannelConfig_MajorLoopCountType

Vendor specific: Set the Dma Channel major loop count.

If minor loop channel linking is enabled, the major loop count limit is between 0 and 511.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	1
max	32767
min	0

4.1.126 Reference dynamic_dmaLogicChannelConfig_MajorLoopLinkChValueType

Vendor specific: Set the Dma Channel major loop logic channel link.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.1.127 Container trgmuxInstaceList

List of Instance.

Included subcontainers:

Configuration Parameters

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.128 Parameter `trgmuxHardwareInstance`

Trigger Mux Hardware Instance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TRGMUX_IP_HW_INST_0
literals	['TRGMUX_IP_HW_INST_0']

4.1.129 Container `trgmuxLogicGroup`

List of Logic Trigger Groups.

Included subcontainers:

- [trgmuxLogicTrigger](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	20
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.130 Parameter `trgmuxLogicGroupHardwareInstance`

Trigger Mux Hardware Instance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TRGMUX_IP_HW_INST_0
literals	['TRGMUX_IP_HW_INST_0']

4.1.131 Parameter trgmuxLogicGroup_Name

Logic Trigger Group.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TRGMUX_IP_DMA
literals	['TRGMUX_IP_DMA', 'TRGMUX_IP_EXTOUT0', 'TRGMUX_IP_↵ EXTOUT1', 'TRGMUX_IP_ADC0', 'TRGMUX_IP_ADC1', 'TRGMUX_↵ IP_CMP0', 'TRGMUX_IP_FTM0', 'TRGMUX_IP_FTM1', 'TRGMUX_↵ IP_FTM2', 'TRGMUX_IP_FTM3', 'TRGMUX_IP_PDB0', 'TRGMUX_↵ IP_PDB1', 'TRGMUX_IP_FLEXIO', 'TRGMUX_IP_LPIT0', 'TRGMUX_↵ _IP_LPUART0', 'TRGMUX_IP_LPUART1', 'TRGMUX_IP_LPI2C0', 'TRGMUX_IP_LPSPi0', 'TRGMUX_IP_LPSPi1', 'TRGMUX_IP_↵ LPTMR0']

4.1.132 Parameter trgmuxLogicGroup_Lock

Logic Trigger Lock.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.133 Container `trgmuxLogicTrigger`

List of Logic Triggers.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	53
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.134 Parameter `trgmuxLogicTrigger_Name`

Logic Trigger Name.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TRGMUX_LOGIC_GROUP_0_TRIGGER↔ _0

4.1.135 Parameter `trgmuxLogicTrigger_Output`

Logic Trigger Output.

Property	Value
<code>type</code>	ECUC-ENUMERATION-PARAM-DEF
<code>origin</code>	NXP
<code>symbolicNameValue</code>	false
<code>lowerMultiplicity</code>	1
<code>upperMultiplicity</code>	1
<code>postBuildVariantMultiplicity</code>	N/A
<code>multiplicityConfigClasses</code>	N/A
<code>postBuildVariantValue</code>	true
<code>valueConfigClasses</code>	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
<code>defaultValue</code>	TRGMUX_IP_OUTPUT_DMA_CH0

Property	Value
literals	['TRGMUX_IP_OUTPUT_DMA_CH0', 'TRGMUX_IP_OUTPUT_DMA_CH1', 'TRGMUX_IP_OUTPUT_DMA_CH2', 'TRGMUX_IP_OUTPUT_DMA_CH3', 'TRGMUX_IP_OUTPUT_EXTOUT0_TRGMUX_OUT0', 'TRGMUX_IP_OUTPUT_EXTOUT0_TRGMUX_OUT1', 'TRGMUX_IP_OUTPUT_EXTOUT0_TRGMUX_OUT2', 'TRGMUX_IP_OUTPUT_EXTOUT0_TRGMUX_OUT3', 'TRGMUX_IP_OUTPUT_EXTOUT1_TRGMUX_OUT4', 'TRGMUX_IP_OUTPUT_EXTOUT1_TRGMUX_OUT5', 'TRGMUX_IP_OUTPUT_EXTOUT1_TRGMUX_OUT6', 'TRGMUX_IP_OUTPUT_EXTOUT1_TRGMUX_OUT7', 'TRGMUX_IP_OUTPUT_ADC0_ADHWT_TRIG_0', 'TRGMUX_IP_OUTPUT_ADC0_ADHWT_TRIG_1', 'TRGMUX_IP_OUTPUT_ADC0_ADHWT_TRIG_2', 'TRGMUX_IP_OUTPUT_ADC0_ADHWT_TRIG_3', 'TRGMUX_IP_OUTPUT_ADC1_ADHWT_TRIG_0', 'TRGMUX_IP_OUTPUT_ADC1_ADHWT_TRIG_1', 'TRGMUX_IP_OUTPUT_ADC1_ADHWT_TRIG_2', 'TRGMUX_IP_OUTPUT_ADC1_ADHWT_TRIG_3', 'TRGMUX_IP_OUTPUT_CMP0_SAMPLE_INPUT', 'TRGMUX_IP_OUTPUT_FTM0_HWTRIG0', 'TRGMUX_IP_OUTPUT_FTM0_FAULT0', 'TRGMUX_IP_OUTPUT_FTM0_FAULT1', 'TRGMUX_IP_OUTPUT_FTM0_FAULT2', 'TRGMUX_IP_OUTPUT_FTM1_HWTRIG0', 'TRGMUX_IP_OUTPUT_FTM1_FAULT0', 'TRGMUX_IP_OUTPUT_FTM1_FAULT1', 'TRGMUX_IP_OUTPUT_FTM1_FAULT2', 'TRGMUX_IP_OUTPUT_FTM2_HWTRIG0', 'TRGMUX_IP_OUTPUT_FTM2_FAULT0', 'TRGMUX_IP_OUTPUT_FTM2_FAULT1', 'TRGMUX_IP_OUTPUT_FTM2_FAULT2', 'TRGMUX_IP_OUTPUT_FTM3_HWTRIG0', 'TRGMUX_IP_OUTPUT_FTM3_FAULT0', 'TRGMUX_IP_OUTPUT_FTM3_FAULT1', 'TRGMUX_IP_OUTPUT_FTM3_FAULT2', 'TRGMUX_IP_OUTPUT_PDB0_TRIGGER_IN0', 'TRGMUX_IP_OUTPUT_PDB1_TRIGGER_IN0', 'TRGMUX_IP_OUTPUT_FLEXIO_TRG_TIM0', 'TRGMUX_IP_OUTPUT_FLEXIO_TRG_TIM1', 'TRGMUX_IP_OUTPUT_FLEXIO_TRG_TIM2', 'TRGMUX_IP_OUTPUT_FLEXIO_TRG_TIM3', 'TRGMUX_IP_OUTPUT_LPIT0_TRG_CH0', 'TRGMUX_IP_OUTPUT_LPIT0_TRG_CH1', 'TRGMUX_IP_OUTPUT_LPIT0_TRG_CH2', 'TRGMUX_IP_OUTPUT_LPIT0_TRG_CH3', 'TRGMUX_IP_OUTPUT_LPUART0_TRG', 'TRGMUX_IP_OUTPUT_LPUART1_TRG', 'TRGMUX_IP_OUTPUT_LPI2C0_TRG', 'TRGMUX_IP_OUTPUT_LPSPI0_TRG', 'TRGMUX_IP_OUTPUT_LPSPI1_TRG', 'TRGMUX_IP_OUTPUT_LPTMR0_ALT0']

4.1.136 Parameter trgmuxLogicTrigger_Input

Logic Trigger Input.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TRGMUX_IP_INPUT_LOGIC0_VSS
literals	['TRGMUX_IP_INPUT_LOGIC0_VSS', 'TRGMUX_IP_INPUT_LOGIC1↵ _VDD', 'TRGMUX_IP_INPUT_TRGMUX_IN0', 'TRGMUX_IP_INPUT↵ _TRGMUX_IN1', 'TRGMUX_IP_INPUT_TRGMUX_IN2', 'TRGMUX↵ _IP_INPUT_TRGMUX_IN3', 'TRGMUX_IP_INPUT_TRGMUX_IN4', 'TRGMUX_IP_INPUT_TRGMUX_IN5', 'TRGMUX_IP_INPUT↵ TRGMUX_IN6', 'TRGMUX_IP_INPUT_TRGMUX_IN7', 'TRGMUX↵ _IP_INPUT_TRGMUX_IN8', 'TRGMUX_IP_INPUT_TRGMUX_IN9', 'TRGMUX_IP_INPUT_TRGMUX_IN10', 'TRGMUX_IP_INPUT↵ TRGMUX_IN11', 'TRGMUX_IP_INPUT_CMP0_OUT', 'TRGMUX_IP↵ _INPUT_LPIT_CH0', 'TRGMUX_IP_INPUT_LPIT_CH1', 'TRGMUX↵ IP_INPUT_LPIT_CH2', 'TRGMUX_IP_INPUT_LPIT_CH3', 'TRGMUX↵ _IP_INPUT_LPTMR0', 'TRGMUX_IP_INPUT_FTM0_INIT_TRIG', 'TRGMUX_IP_INPUT_FTM0_EXT_TRIG', 'TRGMUX_IP_INPUT↵ _FTM1_INIT_TRIG', 'TRGMUX_IP_INPUT_FTM1_EXT_TRIG', 'TRGMUX_IP_INPUT_FTM2_INIT_TRIG', 'TRGMUX_IP_INPUT↵ _FTM2_EXT_TRIG', 'TRGMUX_IP_INPUT_FTM3_INIT_TRIG', 'TRGMUX_IP_INPUT_FTM3_EXT_TRIG', 'TRGMUX_IP_INPUT↵ ADC0_COCO_0', 'TRGMUX_IP_INPUT_ADC0_COCO_1', 'TRGMUX↵ _IP_INPUT_ADC1_COCO_0', 'TRGMUX_IP_INPUT_ADC1_COCO_1', 'TRGMUX_IP_INPUT_PDB0_ADCH0_TRIG', 'TRGMUX_IP_INPUT↵ _PDB0_PULSE_OUT', 'TRGMUX_IP_INPUT_PDB1_ADCH0_TRIG', 'TRGMUX_IP_INPUT_PDB1_PULSE_OUT', 'TRGMUX_IP_INPUT↵ _RTC_ALARM', 'TRGMUX_IP_INPUT_RTC_SECOND', 'TRGMUX↵ _IP_INPUT_FLEXIO_TRIG0', 'TRGMUX_IP_INPUT_FLEXIO↵ TRIG1', 'TRGMUX_IP_INPUT_FLEXIO_TRIG2', 'TRGMUX_IP↵ INPUT_FLEXIO_TRIG3', 'TRGMUX_IP_INPUT_LPUART0_RX_DATA', 'TRGMUX_IP_INPUT_LPUART0_TX_DATA', 'TRGMUX_IP_INPUT↵ _LPUART0_RX_IDLE', 'TRGMUX_IP_INPUT_LPUART1_RX_DATA', 'TRGMUX_IP_INPUT_LPUART1_TX_DATA', 'TRGMUX_IP_INPUT↵ _LPUART1_RX_IDLE', 'TRGMUX_IP_INPUT_LPI2C0_MASTER↵ TRIG', 'TRGMUX_IP_INPUT_LPI2C0_SLAVE_TRIG', 'TRGMUX_IP↵ _INPUT_LPSPI0_FRAME', 'TRGMUX_IP_INPUT_LPSPI0_RX_DATA', 'TRGMUX_IP_INPUT_LPSPI1_FRAME', 'TRGMUX_IP_INPUT↵ LPSPI1_RX_DATA', 'TRGMUX_IP_INPUT_SIM_SW_TRIG']

4.1.137 Reference trgmuxLogicTrigger_EcucPartitionRef

Maps a TRGMUX logic channel to zero or multiple ECUC partitions to limit the access to this mappable element.

The ECUC partitions referenced are a subset of the ECUC partitions where the MCL driver is mapped to

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Configuration Parameters

Property	Value
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.138 Container FlexioCommon

List of Flexio instances available on the platform.

Included subcontainers:

- [FlexioMclLogicChannels](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	8
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.139 Parameter FlexioMclInstances

Select one of the Flexio instance available on the platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FLEXIO_0
literals	['FLEXIO_0']

4.1.140 Parameter FlexioDebugEnabled

Enable Debug Mode

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.141 Container FlexioMclLogicChannels

Flexio Logic Channel Configuration

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	4
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.142 Parameter FlexioMclChannelId

Select one of the Flexio channels available on the platform for selected instance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	CHANNEL_0
literals	['CHANNEL_0', 'CHANNEL_1', 'CHANNEL_2', 'CHANNEL_3']

4.1.143 Parameter FlexioMclPinId

Select one of the Flexio pins available on the platform for selected instance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	PIN_0
literals	['PIN_0', 'PIN_1', 'PIN_2', 'PIN_3', 'PIN_4', 'PIN_5', 'PIN_6', 'PIN_7']

4.1.144 Parameter FlexioMclAddPinEnable

Enable feature to select one more Flexio pin.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.145 Parameter FlexioMclAddPinId

Select one of the Flexio pins available on the platform for selected instance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	PIN_1
literals	['PIN_0', 'PIN_1', 'PIN_2', 'PIN_3', 'PIN_4', 'PIN_5', 'PIN_6', 'PIN_7']

4.1.146 Parameter FlexioMclAddChannelEnable

Enable feature to select one more Flexio channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.147 Parameter FlexioMclAddChannelId

Select one of the Flexio channels available on the platform for selected instance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CHANNEL_0
literals	['CHANNEL_0', 'CHANNEL_1', 'CHANNEL_2', 'CHANNEL_3']

4.1.148 Container CommonPublishedInformation

Vendor specific:

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.149 Parameter ArReleaseMajorVersion

Vendor specific:

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
default Value	4
max	4
min	4

4.1.150 Parameter ArReleaseMinorVersion

Vendor specific:

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.151 Parameter ArReleaseRevisionVersion

Vendor specific:

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.152 Parameter ModuleId

Vendor specific:

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Property	Value
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.1.153 Parameter SwMajorVersion

Vendor specific:

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.154 Parameter SwMinorVersion

Vendor specific:

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.155 Parameter SwPatchVersion

Vendor specific:

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.156 Parameter VendorId

Vendor specific:

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 DMA_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Dma_Ip](#)
 - Container [MclGeneral](#)

Configuration Parameters

- * Parameter [MclEnableUserModeSupport](#)
- * Container [MclDma](#)
 - Parameter [MclEnableDma](#)
 - Parameter [MclEnableDmaDevErrorDetect](#)
- Container [MclConfig](#)
 - * Container [dmaLogicInstance__ConfigType](#)
 - Parameter [dmaLogicInstance__IdName](#)
 - Parameter [dmaLogicInstance__hwId](#)
 - Parameter [dmaLogicInstance__enDebug](#)
 - Parameter [dmaLogicInstance__enRoundRobin](#)
 - Parameter [dmaLogicInstance__enHaltAfterError](#)
 - Parameter [dmaLogicInstance__enChLinking](#)
 - * Container [dmaLogicChannel__Type](#)
 - Parameter [dmaLogicChannel__LogicName](#)
 - Parameter [dmaLogicChannel__HwInstId](#)
 - Parameter [dmaLogicChannel__HwChId](#)
 - Parameter [dmaLogicChannel__InterruptCallback](#)
 - Parameter [dmaLogicChannel__ErrorInterruptCallback](#)
 - Parameter [dmaLogicChannel__EnableGlobalConfig](#)
 - Parameter [dmaLogicChannel__EnableTransferConfig](#)
 - Parameter [dmaLogicChannel__EnableScatterGather](#)
 - Container [dmaLogicChannel__ConfigType](#)

4.2.1 Module Dma_Ip

Direct Memory Access

Included containers:

- [MclGeneral](#)
- [MclConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.2.2 Parameter MclEnableDmaDevErrorDetect

Enable/Disable Dev Error Detect.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.3 CACHE_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Cache_Ip](#)
 - Container [MclGeneral](#)
 - * Parameter [MclEnableUserModeSupport](#)
 - * Container [MclCache](#)
 - Parameter [MclEnableCache](#)
 - Parameter [MclCacheTimeoutValue](#)
 - Parameter [MclCacheTimeoutMethod](#)
 - Parameter [MclEnableCacheDevErrorDetect](#)

4.3.1 Module Cache_Ip

Definition of Cache configurations

Included containers:

- [MclGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3.2 Parameter MclEnableCacheDevErrorDetect

Enable/Disable Dev Error Detect.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.4 TRGMUX_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Trgmux_Ip](#)
 - Container [MclGeneral](#)
 - * Parameter [MclEnableUserModeSupport](#)
 - * Container [MclTrgMux](#)
 - Parameter [MclEnableTrgMux](#)
 - Parameter [MclEnableTrgMuxDevErrorDetect](#)
 - Container [MclConfig](#)
 - * Container [trgmuxInstaceList](#)
 - Parameter [trgmuxHardwareInstance](#)
 - * Container [trgmuxLogicGroup](#)
 - Parameter [trgmuxLogicGroupHardwareInstance](#)
 - Parameter [trgmuxLogicGroup_Name](#)
 - Parameter [trgmuxLogicGroup_Lock](#)
 - Container [trgmuxLogicTrigger](#)

4.4.1 Module Trgmux_Ip

Definition of TRGMUX configurations

Included containers:

- [MclGeneral](#)

- [MclConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.4.2 Parameter MclEnableTrgMuxDevErrorDetect

Enable/Disable Dev Error Detect.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.5 FLEXIO_MCL_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Flexio_Mcl_Ip](#)
 - Container [MclGeneral](#)
 - * Parameter [MclEnableUserModeSupport](#)
 - * Container [MclFlexioCommon](#)
 - Parameter [MclEnableFlexioCommon](#)
 - Parameter [MclEnableFlexioDevErrorDetect](#)
 - Container [MclConfig](#)
 - * Container [FlexioCommon](#)
 - Parameter [FlexioMclInstances](#)
 - Parameter [FlexioDebugEnable](#)

- Container [FlexioMclLogicChannels](#)

4.5.1 Module Flexio_Mcl_Ip

Definition of FLEXIO Configuration

Included containers:

- [MclGeneral](#)
- [MclConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.5.2 Parameter MclEnableFlexioDevErrorDetect

Enable/Disable Dev Error Detect.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.6 FTM_MCL_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Ftm_Mcl_Ip](#)
 - Container [MclGeneral](#)
 - * Container [MclFtmCommon](#)
 - Parameter [Mcl_FtmCommonTimebase](#)

- Parameter [Mcl_FtmSplitCommonTimebase](#)
- Parameter [MclFtmEnableUserModeSupport](#)
- Parameter [MclFtmEnableDevErrorDetect](#)

4.6.1 Module Ftm_Mcl_Ip

Definition of FTM Configuration

Included containers:

- [MclGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.6.2 Parameter MclFtmEnableUserModeSupport

Vendor specific: Enable/Disable FTM user mode support.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.6.3 Parameter MclFtmEnableDevErrorDetect

Vendor specific: Enable/Disable FTM Dev error detect.

Property	Value
type	BOOLEAN

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

CACHE IP Driver	96
MCL Driver	100
DMA IP Driver	128
FLEXIO IP Driver	148
FTM IP Driver	152
TRGMUX IP Driver	153

Chapter 6

Module Documentation

6.1 CACHE IP Driver

6.1.1 Detailed Description

Enum Reference

- enum [Cache_Ip_Type](#)
This type contains the Cache Ip types.
- enum [Cache_Ip_BusType](#)
This type contains the Cache Ip Bus types.

Function Reference

- Std_ReturnType [Cache_Ip_Enable](#) (const [Cache_Ip_Type](#) CacheType, const [Cache_Ip_BusType](#) BusType)
This function enables the Cache Ip Driver.
- Std_ReturnType [Cache_Ip_Disable](#) (const [Cache_Ip_Type](#) CacheType, const [Cache_Ip_BusType](#) BusType)
This function disables the Cache Ip Driver.
- Std_ReturnType [Cache_Ip_Invalidate](#) (const [Cache_Ip_Type](#) CacheType, const [Cache_Ip_BusType](#) BusType)
This function Invalidates the Cache Ip Driver.
- Std_ReturnType [Cache_Ip_Clean](#) (const [Cache_Ip_Type](#) CacheType, const [Cache_Ip_BusType](#) BusType, const boolean EnInvalidate)
This function Clean the Cache Ip Driver.
- Std_ReturnType [Cache_Ip_InvalidateByAddr](#) (const [Cache_Ip_Type](#) CacheType, const [Cache_Ip_BusType](#) BusType, const uint32 Addr, const uint32 Length)
This function Invalidates By Address the Cache Ip Driver.
- Std_ReturnType [Cache_Ip_CleanByAddr](#) (const [Cache_Ip_Type](#) CacheType, const [Cache_Ip_BusType](#) BusType, const boolean EnInvalidate, const uint32 Addr, const uint32 Length)
This function Clean By Address the Cache Ip Driver.

6.1.2 Enum Reference

6.1.2.1 Cache_Ip_Type

enum [Cache_Ip_Type](#)

This type contains the Cache Ip types.

The cache types specifies what type of cache shall be used when calling the interface. The `CACHE_IP_LMEM` select LMEM caches. The `CACHE_IP_CORE` select CORE caches.

Definition at line 109 of file [Cache_Ip.h](#).

6.1.2.2 Cache_Ip_BusType

enum [Cache_Ip_BusType](#)

This type contains the Cache Ip Bus types.

The cache bus types specifies what type of cache shall be used when calling the interface. The `CACHE_IP_PC_BUS` selects Processor Code (PC) bus (used with Cache Lmem). The `CACHE_IP_PS_BUS` selects Processor System (PS) bus (used with Cache Lmem). The `CACHE_IP_ALL_BUS` selects PC and PS bus (used with Cache Lmem). The `CACHE_IP_INSTRUCTION` selects instruction cache (used with Cache Core). The `CACHE_IP_DATA` selects data cache (used with Cache Core).

Definition at line 125 of file [Cache_Ip.h](#).

6.1.3 Function Reference

6.1.3.1 Cache_Ip_Enable()

```
Std_ReturnType Cache_Ip_Enable (
    const Cache\_Ip\_Type CacheType,
    const Cache\_Ip\_BusType BusType )
```

This function enables the Cache Ip Driver.

This service is a reentrant function that shall enable the Cache Ip driver.

Parameters

in	<i>CacheType</i>	Specifies the cache type.
in	<i>BusType</i>	Specifies the bus type.

Returns

`E_OK` is returned if the enable action finished correctly. `E_NOT_OK` is returned if the enable action finished incorrectly.

6.1.3.2 Cache_Ip_Disable()

```
Std_ReturnType Cache_Ip_Disable (
    const Cache\_Ip\_Type CacheType,
    const Cache\_Ip\_BusType BusType )
```

This function disables the Cache Ip Driver.

This service is a reentrant function that shall disables the Cache Ip driver.

Parameters

in	<i>CacheType</i>	Specifies the cache type.
in	<i>BusType</i>	Specifies the bus type.

Returns

E_OK is returned if the disable action finished correctly. E_NOT_OK is returned if the disable action finished incorrectly.

6.1.3.3 Cache_Ip_Invalidate()

```
Std_ReturnType Cache_Ip_Invalidate (
    const Cache_Ip_Type CacheType,
    const Cache_Ip_BusType BusType )
```

This function Invalidates the Cache Ip Driver.

This service is a reentrant function that shall Invalidates the Cache Ip driver. The Invalidate operation applies for the entire selected Cache Type.

A cache invalidate operation ensures that updates made visible by observers that access memory at the point to which the invalidate is defined are made visible to an observer that controls the cache. This might result in the loss of updates to the locations affected by the invalidate operation that have been written by observers that access the cache. If the address of an entry on which the invalidate operates does not have a Normal Cacheable attribute, or if the cache is disabled, then an invalidate operation also ensures that this address is not present in the cache.

Parameters

in	<i>CacheType</i>	Specifies the cache type.
in	<i>BusType</i>	Specifies the bus type.

Returns

E_OK is returned if the invalidation finished correctly. E_NOT_OK is returned if the invalidation finished incorrectly.

6.1.3.4 Cache_Ip_Clean()

```
Std_ReturnType Cache_Ip_Clean (
    const Cache_Ip_Type CacheType,
    const Cache_Ip_BusType BusType,
    const boolean EnInvalidate )
```

This function Clean the Cache Ip Driver.

This service is a reentrant function that shall Clean the Cache Ip driver. The Clean operation applies for the entire selected Cache Type.

A cache clean operation ensures that updates made by an observer that controls the cache are made visible to other observers that can access memory at the point to which the operation is performed. Once the Clean has completed, the new memory values are guaranteed to be visible to the point to which the operation is performed, for example

to the point of unification. The cleaning of a cache entry from a cache can overwrite memory that has been written by another observer only if the entry contains a location that has been written to by an observer in the shareability domain of that memory location.

A cache clean and invalidate operation behaves as the execution of a clean operation followed immediately by an invalidate operation. Both operations are performed to the same location.

Parameters

in	<i>CacheType</i>	Specifies the cache type.
in	<i>BusType</i>	Specifies the bus type.
in	<i>enInvalidate</i>	Specifies to execute operation Clean&Invalidate.

Returns

E_OK is returned if the cleaning finished correctly. E_NOT_OK is returned if the cleaning finished incorrectly.

6.1.3.5 Cache_Ip_InvalidateByAddr()

```
Std_ReturnType Cache_Ip_InvalidateByAddr (
    const Cache_Ip_Type CacheType,
    const Cache_Ip_BusType BusType,
    const uint32 Addr,
    const uint32 Length )
```

This function Invalidates By Address the Cache Ip Driver.

This service is a reentrant function that shall Invalidates By Address the Cache Ip driver. The Invalidate By Address operation applies for the memory segment specified by the start Address and Length. The operation Invalidates only multiple of Cache Line Size, thus the specified memory segment shall be aligned and multiple of the Cache Line Size.

A cache invalidate operation ensures that updates made visible by observers that access memory at the point to which the invalidate is defined are made visible to an observer that controls the cache. This might result in the loss of updates to the locations affected by the invalidate operation that have been written by observers that access the cache. If the address of an entry on which the invalidate operates does not have a Normal Cacheable attribute, or if the cache is disabled, then an invalidate operation also ensures that this address is not present in the cache.

Parameters

in	<i>CacheType</i>	Specifies the cache type.
in	<i>BusType</i>	Specifies the bus type.
in	<i>Addr</i>	Specifies the memory segment start address.
in	<i>Length</i>	Specifies the memory segment length.

Returns

E_OK is returned if the invalidation finished correctly. E_NOT_OK is returned if the invalidation finished incorrectly.

6.1.3.6 Cache_Ip_CleanByAddr()

```
Std_ReturnType Cache_Ip_CleanByAddr (
    const Cache_Ip_Type CacheType,
```

```
const Cache_Ip_BusType BusType,
const boolean EnInvalidate,
const uint32 Addr,
const uint32 Length )
```

This function Clean By Address the Cache Ip Driver.

This service is a reentrant function that shall Clean By Address the Cache Ip driver. The Clean By Address operation applies for the memory segment specified by the start Address and Length. The operation Cleans only multiple of Cache Line Size, thus the specified memory segment shall be aligned and multiple of the Cache Line Size.

A cache clean operation ensures that updates made by an observer that controls the cache are made visible to other observers that can access memory at the point to which the operation is performed. Once the Clean has completed, the new memory values are guaranteed to be visible to the point to which the operation is performed, for example to the point of unification. The cleaning of a cache entry from a cache can overwrite memory that has been written by another observer only if the entry contains a location that has been written to by an observer in the shareability domain of that memory location.

A cache clean and invalidate operation behaves as the execution of a clean operation followed immediately by an invalidate operation. Both operations are performed to the same location.

Parameters

in	<i>CacheType</i>	Specifies the cache type.
in	<i>BusType</i>	Specifies the bus type.
in	<i>enInvalidate</i>	Specifies to execute operation Clean&Invalidate.
in	<i>Addr</i>	Specifies the memory segment start address.
in	<i>Length</i>	Specifies the memory segment length.

Returns

E_OK is returned if the cleaning finished correctly. E_NOT_OK is returned if the cleaning finished incorrectly.

6.2 MCL Driver

6.2.1 Detailed Description

Data Structures

- struct [Mcl_DmaChannelGlobalListType](#)
This type contains the Mcl Dma Channel Global List. [More...](#)
- struct [Mcl_DmaChannelTransferListType](#)
This type contains the Mcl Dma Channel Transfer List. [More...](#)
- struct [Mcl_DmaChannelScatterGatherListType](#)
This type contains the Mcl Dma Channel Scatter/Gather List. [More...](#)
- struct [Mcl_DmaInstanceStatusType](#)
This type contains the Mcl Dma Instance Status. [More...](#)
- struct [Mcl_DmaChannelStatusType](#)
This type contains the Mcl Dma Channel Status. [More...](#)
- struct [Mcl_ConfigType](#)
This type contains the Mcl Configuration. [More...](#)

Macros

- #define **MCL_DET_INIT**
API service ID for *Mcl_Init* function.
- #define **MCL_DET_DMA_INSTANCE_COMMAND**
API service ID for *Mcl_SetDmaInstanceCommand* function.
- #define **MCL_DET_DMA_INSTANCE_STATUS**
API service ID for *Mcl_GetDmaInstanceStatus* function.
- #define **MCL_DET_DMA_CHANNEL_COMMAND**
API service ID for *Mcl_SetDmaChannelCommand* function.
- #define **MCL_DET_DMA_CHANNEL_STATUS**
API service ID for *Mcl_GetDmaChannelStatus* function.
- #define **MCL_DET_DMA_GLOBAL**
API service ID for *Mcl_SetDmaChannelGlobalList* function.
- #define **MCL_DET_DMA_TRANSFER**
API service ID for *Mcl_SetDmaChannelTransferList* function.
- #define **MCL_DET_DMA_SCATTER_GATHER_LIST**
API service ID for *Mcl_SetDmaChannelScatterGatherList* function.
- #define **MCL_DET_DMA_INFORMATION**
API service ID for *Mcl_GetDmaChannelParam* function.
- #define **MCL_DET_DMA_SCATTER_GATHER_CONFIG**
API service ID for *Mcl_SetDmaChannelScatterGatherConfig* function.
- #define **MCL_DET_DEINIT**
API service ID for *Mcl_DeInit* function.
- #define **MCL_DET_CACHE_ENABLE**
API service ID for *Mcl_CacheEnable* function.
- #define **MCL_DET_CACHE_DISABLE**
API service ID for *Mcl_CacheDisable* function.
- #define **MCL_DET_CACHE_INVALIDATE**
API service ID for *Mcl_CacheInvalidate* function.
- #define **MCL_DET_CACHE_CLEAN**
API service ID for *Mcl_CacheClean* function.
- #define **MCL_DET_CACHE_INVALIDATE_BY_ADDRESS**
API service ID for *Mcl_CacheInvalidateByAddr* function.
- #define **MCL_DET_CACHE_CLEAN_BY_ADDRESS**
API service ID for *Mcl_CacheCleanByAddr* function.
- #define **MCL_DET_TRGMUX_INPUT**
API service ID for *Mcl_SetTrgMuxInput* function.
- #define **MCL_DET_TRGMUX_LOCK**
API service ID for *Mcl_SetTrgMuxLock* function.
- #define **MCL_DET_TRGMUX_SET_INPUT**

- API service ID for Mcl_SetTrgMuxInput function.*

 - #define [MCL_DET_TRGMUX_SET_LOC](#)

API service ID for Mcl_SetTrgMuxLock function.

 - #define [MCL_E_UNINIT](#)

All API's having pointers as parameters shall return this error if called with with a NULL value.

 - #define [MCL_E_PARAM_POINTER](#)

All API's having pointers as parameters shall return this error if called with with a NULL value.

 - #define [MCL_E_INVALID_INSTANCE](#)

All API's called with wrong instance shall return this error.

 - #define [MCL_E_INVALID_CHANNEL](#)

All API's called with wrong channel shall return this error.

 - #define [MCL_E_INVALID_COMMAND](#)

All API's called with wrong instance shall return this error.

 - #define [MCL_E_INVALID_PARAMETER](#)

All API's called with wrong read parameter shall return this error.

 - #define [MCL_E_INVALID_STATE](#)

All API's called in wrong sequence shall return this error.

 - #define [MCL_E_INCONSISTENCY](#)

All API's called while hardware has error status shall return this error.

 - #define [MCL_E_TIMEOUT](#)

All API's called with a timeout value shall return this error if execution is not finished in the allocated timeframe.

 - #define [MCL_E_PARAM_CONFIG](#)

If DET error reporting is enabled, the MCL will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return MCL_E_PARAM_CONFIG.

 - #define [MCL_E_PROTECTED](#)

If DET error reporting is enabled, the MCL will check if registers are protected.

 - #define [MCL_E_INIT_FAILED](#)

If VariantPreCompile is used, the configuration pointer shall have a NULL_PTR value. If VariantPostBuild is used, the configuration pointer shall be different from NULL_PTR. And in case of violate will return MCL_E_INIT_↵ FAILED.

Enum Reference

- enum [Mcl_DmaInstanceCmdType](#)

This type contains the Mcl Dma Instance Commands.
- enum [Mcl_DmaChannelCmdType](#)

This type contains the Mcl Dma Channel Commands.
- enum [Mcl_DmaChannelGlobalParamType](#)

This type contains the Mcl Dma Channel Global Parameters.
- enum [Mcl_DmaChannelTransferParamType](#)

This type contains the Mcl Dma Channel Transfer Parameters.
- enum [Mcl_DmaChannelStateType](#)

This type contains the Mcl Dma Channel State values.

- enum [Mcl_DmaChannelInfoParamType](#)
This type contains the Mcl Dma Channel Information Parameters.
- enum [Mcl_CacheType](#)
This type contains the Mcl Cache Type selection.
- enum [Mcl_CacheBusType](#)
This type contains the Mcl Cache Bus Type selection.
- enum [Mcl_ReturnType](#)
This type contains the Mcl Return Type.

Function Reference

- void [Mcl_Init](#) (const [Mcl_ConfigType](#) *const ConfigPtr)
This function initializes the Mcl Driver.
- void [Mcl_DeInit](#) (void)
This function deinitializes the Mcl Driver.
- void [Mcl_SetDmaInstanceCommand](#) (const uint32 Instance, const [Mcl_DmaInstanceCmdType](#) Command)
This function sets Dma Instance Command.
- void [Mcl_GetDmaInstanceStatus](#) (const uint32 Instance, [Mcl_DmaInstanceStatusType](#) *const Status)
This function gets Dma Instance Status.
- void [Mcl_SetDmaChannelCommand](#) (const uint32 Channel, const [Mcl_DmaChannelCmdType](#) Command)
This function sets Dma Channel Command.
- void [Mcl_GetDmaChannelStatus](#) (const uint32 Channel, [Mcl_DmaChannelStatusType](#) *const Status)
This function gets Dma Channel Status.
- void [Mcl_SetDmaChannelGlobalList](#) (const uint32 Channel, const [Mcl_DmaChannelGlobalListType](#) List[], const uint32 ListDimension)
This function sets Dma Channel Global List settings.
- void [Mcl_SetDmaChannelTransferList](#) (const uint32 Channel, const [Mcl_DmaChannelTransferListType](#) List[], const uint32 ListDimension)
This function sets Dma Channel Transfer List settings.
- void [Mcl_SetDmaChannelScatterGatherList](#) (const uint32 Channel, const uint32 Element, const [Mcl_DmaChannelScatterGatherListType](#) List[], const uint32 ListDimension)
This function sets Dma Channel Scatter/Gather List settings.
- void [Mcl_GetDmaChannelParam](#) (const uint32 Channel, const [Mcl_DmaChannelInfoParamType](#) Param, uint32 *const Value)
This function gets the Dma Channel Parameter value.
- void [Mcl_SetDmaChannelScatterGatherConfig](#) (const uint32 Channel, const uint32 Element)
This function configures the Dma Channel Scatter/Gather.
- void [Mcl_CacheEnable](#) ([Mcl_CacheType](#) CacheType, [Mcl_CacheBusType](#) BusType)
This function enables the Cache.
- void [Mcl_CacheDisable](#) ([Mcl_CacheType](#) CacheType, [Mcl_CacheBusType](#) BusType)
This function disables the Cache.
- void [Mcl_CacheInvalidate](#) ([Mcl_CacheType](#) CacheType, [Mcl_CacheBusType](#) BusType)
This function Invalidates the Cache.

- void [Mcl_CacheClean](#) ([Mcl_CacheType](#) CacheType, [Mcl_CacheBusType](#) BusType, boolean EnInvalidate)
This function Cleans the Cache.
- void [Mcl_CacheInvalidateByAddr](#) ([Mcl_CacheType](#) CacheType, [Mcl_CacheBusType](#) BusType, uint32 Addr, uint32 Length)
This function Invalidates the Cache by address.
- void [Mcl_CacheCleanByAddr](#) ([Mcl_CacheType](#) CacheType, [Mcl_CacheBusType](#) BusType, boolean EnInvalidate, uint32 Addr, uint32 Length)
This function Cleans the Cache by address.
- void [Mcl_SetTrgMuxInput](#) (const uint32 Trigger, const uint32 Input)
This function sets the Trgmux Trigger Input selection.
- void [Mcl_SetTrgMuxLock](#) (const uint32 Trigger)
This function sets the Trgmux Trigger Lock.
- void [Mcl_SelectCommonTimebase](#) (uint8 instance, uint16 elementSyncList)
Implementation specific function to updates the Global Timebase bits of configured modules.
- void [Mcl_GetVersionInfo](#) (Std_VersionInfoType *const VersionInfo)
Returns the version information of this module.

6.2.1.1 MISRA-C:2012 violations

6.2.2 Data Structure Documentation

6.2.2.1 struct [Mcl_DmaChannelGlobalListType](#)

This type contains the Mcl Dma Channel Global List.

The Mcl Dma Channel Global List contains a pair composed from Dma Channel Global Parameter Type and the Value of the parameter.

Definition at line 535 of file [CDD_Mcl.h](#).

Data Fields

- [Mcl_DmaChannelGlobalParamType](#) Param
The Mcl Dma Channel Global Parameter Type selects a parameter form the Global Parameter enum type.
- uint32 [Value](#)
The Value stores the parameter's value.

Field Documentation

Param

[Mcl_DmaChannelGlobalParamType](#) Param

The Mcl Dma Channel Global Parameter Type selects a parameter form the Global Parameter enum type.

Definition at line 537 of file [CDD_Mcl.h](#).

Value

uint32 Value

The Value stores the parameter's value.

Definition at line 538 of file [CDD_Mcl.h](#).

6.2.2.2 struct Mcl_DmaChannelTransferListType

This type contains the Mcl Dma Channel Transfer List.

The Mcl Dma Channel Transfer List contains a pair composed from Dma Channel Transfer Parameter Type and the Value of the parameter.

Definition at line 548 of file [CDD_Mcl.h](#).

Data Fields

- [Mcl_DmaChannelTransferParamType Param](#)

The Mcl Dma Channel Transfer Parameter Type selects a parameter form the Transfer Parameter enum type.

- uint32 [Value](#)

The Value stores the parameter's value.

Field Documentation**Param**

[Mcl_DmaChannelTransferParamType](#) Param

The Mcl Dma Channel Transfer Parameter Type selects a parameter form the Transfer Parameter enum type.

Definition at line 550 of file [CDD_Mcl.h](#).

Value

uint32 Value

The Value stores the parameter's value.

Definition at line 551 of file [CDD_Mcl.h](#).

6.2.2.3 struct Mcl_DmaChannelScatterGatherListType

This type contains the Mcl Dma Channel Scatter/Gather List.

The Mcl Dma Channel Scatter/Gather List contains a pair composed from Dma Channel Scatter/Gather Parameter Type and the Value of the parameter.

Definition at line 561 of file [CDD_Mcl.h](#).

Data Fields

- [Mcl_DmaChannelTransferParamType Param](#)

The Mcl Dma Channel Transfer Parameter Type selects a parameter form the Transfer Parameter enum type.

- uint32 [Value](#)

The Value stores the parameter's value.

Field Documentation

Param

`Mcl_DmaChannelTransferParamType` Param

The Mcl Dma Channel Transfer Parameter Type selects a parameter form the Transfer Parameter enum type.

Definition at line 563 of file `CDD_Mcl.h`.

Value

`uint32` Value

The Value stores the parameter's value.

Definition at line 564 of file `CDD_Mcl.h`.

6.2.2.4 struct Mcl_DmaInstanceStatusType

This type contains the Mcl Dma Instance Status.

The Mcl Dma Instance Status contains the Hardware Errors, Active Id and Active indication for the running Dma Channel.

Definition at line 575 of file `CDD_Mcl.h`.

Data Fields

- `uint32` [Errors](#)
[VALUE] The Errors value is read from the DMA Instance Error Register (ES) as it is.
- `uint8` [ActiveId](#)
[VALUE] The ActiveId value is read from the DMA Instance Control Register (CR) field ACTIVE_ID.
- `boolean` [Active](#)
[BOOLEAN] The Active value is read from the DMA Instance Control Register (CR) field ACTIVE.
- `uint8` [Version](#)
[VALUE] The Version value is read from the DMA Instance Control Register (CR) field VER.

Field Documentation

Errors

`uint32` Errors

[VALUE] The Errors value is read from the DMA Instance Error Register (ES) as it is.

Definition at line 577 of file `CDD_Mcl.h`.

ActiveId

uint8 ActiveId

[VALUE] The ActiveId value is read from the DMA Instance Control Register (CR) field ACTIVE_ID.

Definition at line 578 of file [CDD_Mcl.h](#).

Active

boolean Active

[BOOLEAN] The Active value is read from the DMA Instance Control Register (CR) field ACTIVE.

Definition at line 579 of file [CDD_Mcl.h](#).

Version

uint8 Version

[VALUE] The Version value is read from the DMA Instance Control Register (CR) field VER.

Definition at line 580 of file [CDD_Mcl.h](#).

6.2.2.5 struct Mcl_DmaChannelStatusType

This type contains the Mcl Dma Channel Status.

The Mcl Dma Channel Status contains the Hardware Errors, Active status and Done indication for the running Dma Channel.

Definition at line 590 of file [CDD_Mcl.h](#).

Data Fields

- [Mcl_DmaChannelStateType ChannelState](#)

[VALUE] The ChStateValue value is read from the internal DMA Driver Channel State Machine. Check UM for additional information.

- uint32 [Errors](#)

[VALUE] The Errors value is read from the DMA Channel Error Register (CHx_ES) as it is.

- boolean [Active](#)

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field ACTIVE.

- boolean [Done](#)

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field DONE.

Field Documentation

ChannelState

`Mcl_DmaChannelStateType` ChannelState

[VALUE] The ChStateValue value is read from the internal DMA Driver Channel State Machine. Check UM for additional information.

Definition at line 592 of file [CDD_Mcl.h](#).

Errors

`uint32` Errors

[VALUE] The Errors value is read from the DMA Channel Error Register (CHx_ES) as it is.

Definition at line 593 of file [CDD_Mcl.h](#).

Active

`boolean` Active

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field ACTIVE.

Definition at line 594 of file [CDD_Mcl.h](#).

Done

`boolean` Done

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field DONE.

Definition at line 595 of file [CDD_Mcl.h](#).

6.2.2.6 struct Mcl_ConfigType

This type contains the Mcl Configuration.

The Mcl Configuration structure contains pointers to the Ip's configuration structure. Based on the available support, specific configurations shall be stored.

Definition at line 330 of file [Mcl_Types.h](#).

6.2.3 Macro Definition Documentation

6.2.3.1 MCL_DET_INIT

```
#define MCL_DET_INIT
```

API service ID for Mcl_Init function.

Parameters used when raising an error/exception

Definition at line 126 of file [CDD_Mcl.h](#).

6.2.3.2 MCL_DET_DMA_INSTANCE_COMMAND

```
#define MCL_DET_DMA_INSTANCE_COMMAND
```

API service ID for Mcl_SetDmaInstanceCommand function.

Parameters used when raising an error/exception

Definition at line 134 of file [CDD_Mcl.h](#).

6.2.3.3 MCL_DET_DMA_INSTANCE_STATUS

```
#define MCL_DET_DMA_INSTANCE_STATUS
```

API service ID for Mcl_GetDmaInstanceStatus function.

Parameters used when raising an error/exception

Definition at line 140 of file [CDD_Mcl.h](#).

6.2.3.4 MCL_DET_DMA_CHANNEL_COMMAND

```
#define MCL_DET_DMA_CHANNEL_COMMAND
```

API service ID for Mcl_SetDmaChannelCommand function.

Parameters used when raising an error/exception

Definition at line 146 of file [CDD_Mcl.h](#).

6.2.3.5 MCL_DET_DMA_CHANNEL_STATUS

```
#define MCL_DET_DMA_CHANNEL_STATUS
```

API service ID for Mcl_GetDmaChannelStatus function.

Parameters used when raising an error/exception

Definition at line 152 of file [CDD_Mcl.h](#).

6.2.3.6 MCL_DET_DMA_GLOBAL

```
#define MCL_DET_DMA_GLOBAL
```

API service ID for Mcl_SetDmaChannelGlobalList function.

Parameters used when raising an error/exception

Definition at line 158 of file [CDD_Mcl.h](#).

6.2.3.7 MCL_DET_DMA_TRANSFER

```
#define MCL_DET_DMA_TRANSFER
```

API service ID for Mcl_SetDmaChannelTransferList function.

Parameters used when raising an error/exception

Definition at line 164 of file [CDD_Mcl.h](#).

6.2.3.8 MCL_DET_DMA_SCATTER_GATHER_LIST

```
#define MCL_DET_DMA_SCATTER_GATHER_LIST
```

API service ID for Mcl_SetDmaChannelScatterGatherList function.

Parameters used when raising an error/exception

Definition at line 170 of file [CDD_Mcl.h](#).

6.2.3.9 MCL_DET_DMA_INFORMATION

```
#define MCL_DET_DMA_INFORMATION
```

API service ID for Mcl_GetDmaChannelParam function.

Parameters used when raising an error/exception

Definition at line 176 of file [CDD_Mcl.h](#).

6.2.3.10 MCL_DET_DMA_SCATTER_GATHER_CONFIG

```
#define MCL_DET_DMA_SCATTER_GATHER_CONFIG
```

API service ID for Mcl_SetDmaChannelScatterGatherConfig function.

Parameters used when raising an error/exception

Definition at line 182 of file [CDD_Mcl.h](#).

6.2.3.11 MCL_DET_DEINIT

```
#define MCL_DET_DEINIT
```

API service ID for Mcl_DeInit function.

Parameters used when raising an error/exception

Definition at line 192 of file [CDD_Mcl.h](#).

6.2.3.12 MCL_DET_CACHE_ENABLE

```
#define MCL_DET_CACHE_ENABLE
```

API service ID for Mcl_CacheEnable function.

Parameters used when raising an error/exception

Definition at line 199 of file [CDD_Mcl.h](#).

6.2.3.13 MCL_DET_CACHE_DISABLE

```
#define MCL_DET_CACHE_DISABLE
```

API service ID for Mcl_CacheDisable function.

Parameters used when raising an error/exception

Definition at line 205 of file [CDD_Mcl.h](#).

6.2.3.14 MCL_DET_CACHE_INVALIDATE

```
#define MCL_DET_CACHE_INVALIDATE
```

API service ID for Mcl_CacheInvalidate function.

Parameters used when raising an error/exception

Definition at line 211 of file [CDD_Mcl.h](#).

6.2.3.15 MCL_DET_CACHE_CLEAN

```
#define MCL_DET_CACHE_CLEAN
```

API service ID for Mcl_CacheClean function.

Parameters used when raising an error/exception

Definition at line 217 of file [CDD_Mcl.h](#).

6.2.3.16 MCL_DET_CACHE_INVALIDATE_BY_ADDRESS

```
#define MCL_DET_CACHE_INVALIDATE_BY_ADDRESS
```

API service ID for Mcl_CacheInvalidateByAddr function.

Parameters used when raising an error/exception

Definition at line 223 of file [CDD_Mcl.h](#).

6.2.3.17 MCL_DET_CACHE_CLEAN_BY_ADDRESS

```
#define MCL_DET_CACHE_CLEAN_BY_ADDRESS
```

API service ID for Mcl_CacheCleanByAddr function.

Parameters used when raising an error/exception

Definition at line 229 of file [CDD_Mcl.h](#).

6.2.3.18 MCL_DET_TRGMUX_INPUT

```
#define MCL_DET_TRGMUX_INPUT
```

API service ID for Mcl_SetTrgMuxInput function.

Parameters used when raising an error/exception

Definition at line 239 of file [CDD_Mcl.h](#).

6.2.3.19 MCL_DET_TRGMUX_LOCK

```
#define MCL_DET_TRGMUX_LOCK
```

API service ID for Mcl_SetTrgMuxLock function.

Parameters used when raising an error/exception

Definition at line 245 of file [CDD_Mcl.h](#).

6.2.3.20 MCL_DET_TRGMUX_SET_INPUT

```
#define MCL_DET_TRGMUX_SET_INPUT
```

API service ID for Mcl_SetTrgMuxInput function.

Parameters used when raising an error/exception

Definition at line 258 of file [CDD_Mcl.h](#).

6.2.3.21 MCL_DET_TRGMUX_SET_LOC

```
#define MCL_DET_TRGMUX_SET_LOC
```

API service ID for Mcl_SetTrgMuxLock function.

Parameters used when raising an error/exception

Definition at line 264 of file [CDD_Mcl.h](#).

6.2.3.22 MCL_E_UNINIT

```
#define MCL_E_UNINIT
```

All API's having pointers as parameters shall return this error if called with with a NULL value.

Definition at line 278 of file [CDD_Mcl.h](#).

6.2.3.23 MCL_E_PARAM_POINTER

```
#define MCL_E_PARAM_POINTER
```

All API's having pointers as parameters shall return this error if called with with a NULL value.

Definition at line 285 of file [CDD_Mcl.h](#).

6.2.3.24 MCL_E_INVALID_INSTANCE

```
#define MCL_E_INVALID_INSTANCE
```

All API's called with wrong instance shall return this error.

Definition at line 291 of file [CDD_Mcl.h](#).

6.2.3.25 MCL_E_INVALID_CHANNEL

```
#define MCL_E_INVALID_CHANNEL
```

All API's called with wrong channel shall return this error.

Definition at line 297 of file [CDD_Mcl.h](#).

6.2.3.26 MCL_E_INVALID_COMMAND

```
#define MCL_E_INVALID_COMMAND
```

All API's called with wrong instance shall return this error.

Definition at line 303 of file [CDD_Mcl.h](#).

6.2.3.27 MCL_E_INVALID_PARAMETER

```
#define MCL_E_INVALID_PARAMETER
```

All API's called with wrong read parameter shall return this error.

Definition at line 309 of file [CDD_Mcl.h](#).

6.2.3.28 MCL_E_INVALID_STATE

```
#define MCL_E_INVALID_STATE
```

All API's called in wrong sequence shall return this error.

Definition at line 315 of file [CDD_Mcl.h](#).

6.2.3.29 MCL_E_INCONSISTENCY

```
#define MCL_E_INCONSISTENCY
```

All API's called while hardware has error status shall return this error.

Definition at line 321 of file [CDD_Mcl.h](#).

6.2.3.30 MCL_E_TIMEOUT

```
#define MCL_E_TIMEOUT
```

All API's called with a timeout value shall return this error if execution is not finished in the allocated timeframe.

Definition at line 328 of file [CDD_Mcl.h](#).

6.2.3.31 MCL_E_PARAM_CONFIG

```
#define MCL_E_PARAM_CONFIG
```

If DET error reporting is enabled, the MCL will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return MCL_E_PARAM_CONFIG.

Definition at line 336 of file [CDD_Mcl.h](#).

6.2.3.32 MCL_E_PROTECTED

```
#define MCL_E_PROTECTED
```

If DET error reporting is enabled, the MCL will check if registers are protected.

Definition at line 341 of file [CDD_Mcl.h](#).

6.2.3.33 MCL_E_INIT_FAILED

```
#define MCL_E_INIT_FAILED
```

If VariantPreCompile is used, the configuration pointer shall have a NULL_PTR value. If VariantPostBuild is used, the configuration pointer shall be different from NULL_PTR. And in case of violate will return MCL_E_INIT_FAILED.

Definition at line 348 of file [CDD_Mcl.h](#).

6.2.4 Enum Reference

6.2.4.1 Mcl_DmaInstanceCmdType

enum [Mcl_DmaInstanceCmdType](#)

This type contains the Mcl Dma Instance Commands.

The Commands trigger specific actions in the Dma Instance.

Enumerator

MCL_DMA_INST_STOP	The Stop Command stops the executing channel and forces the Minor Loop to finish.
MCL_DMA_INST_STOP_ERROR	The StopError Command stops the executing channel, forces the Minor Loop to finish and generates an error interrupt.
MCL_DMA_INST_PAUSE	The Pause Command allows the ongoing transfer to finish and pauses any new transfer.
MCL_DMA_INST_RESUME	The Resume Command allows the transfer to continue.

Definition at line 360 of file [CDD_Mcl.h](#).

6.2.4.2 Mcl_DmaChannelCmdType

enum [Mcl_DmaChannelCmdType](#)

This type contains the Mcl Dma Channel Commands.

The Commands trigger specific actions in the Dma Channel.

Enumerator

MCL_DMA_CH_START_REQUEST	The Start Request Command enables the Dma Channel to be triggered by hardware requests.
MCL_DMA_CH_STOP_REQUEST	The Stop Request Command disables the Dma Channel to be triggered by hardware requests.
MCL_DMA_CH_START_SERVICE	The Start Service Command sends a start request to the Dma Channel.
MCL_DMA_CH_ACK_DONE	The Ack Done Command resets the Dma Channel Done status.
MCL_DMA_CH_ACK_ERROR	The Ack Error Command resets the Dma Channel Error status.
MCL_DMA_CH_ACK_INTERRUPT_STATUS	The Ack Done Command resets the Dma Channel Done status.

Definition at line 374 of file [CDD_Mcl.h](#).

6.2.4.3 Mcl_DmaChannelGlobalParamType

enum [Mcl_DmaChannelGlobalParamType](#)

This type contains the Mcl Dma Channel Global Parameters.

The Parameters set specific functionalities.

Enumerator

MCL_DMA_CH_SET_EN_HARDWARE_REQ	[BOOLEAN] The EnRequest Parameter enables the Dma Channel Request.
MCL_DMA_CH_SET_EN_ERROR_INTERRUPT	[BOOLEAN] The EnError Parameter enables the Dma Channel Error Interrupt.
MCL_DMA_CH_SET_GROUP_PRIORITY	[VALUE] The Group Parameter sets the Dma Channel Group Priority.
MCL_DMA_CH_SET_LEVEL_PRIORITY	[VALUE] The Level Parameter sets the Dma Channel Level Priority.
MCL_DMA_CH_SET_EN_PREEMPTION_↔ PRIORITY	[BOOLEAN] The EnPreemption Parameter enables the Dma Channel Preemption.
MCL_DMA_CH_SET_DIS_PREEMPT_↔ PRIORITY	[BOOLEAN] The DisPreempt Parameter disables the Dma Channel Preempt.

Definition at line 390 of file [CDD_Mcl.h](#).

6.2.4.4 Mcl_DmaChannelTransferParamType

enum [Mcl_DmaChannelTransferParamType](#)

This type contains the Mcl Dma Channel Transfer Parameters.

The Parameters set specific functionalities.

Enumerator

MCL_DMA_CH_SET_SOURCE_ADDRESS	[VALUE] The Source Address Parameter sets the Dma Channel source address value.
MCL_DMA_CH_SET_SOURCE_SIGNED_↔ OFFSET	[VALUE] The Source Signed Offset Parameter sets the Dma Channel source signed offset value.
MCL_DMA_CH_SET_SOURCE_SIGNED_↔ LAST_ADDR_ADJ	[VALUE] The Source Signed Last Address Adjustment Parameter sets the Dma Channel source signed last address adjustment.
MCL_DMA_CH_SET_SOURCE_TRANSFER_↔ SIZE	[VALUE] The Source Transfer Size Parameter sets the Dma Channel source transfer size.
MCL_DMA_CH_SET_SOURCE_MODULO	[VALUE] The Source Modulo Parameter sets the Dma Channel source modulo.
MCL_DMA_CH_SET_DESTINATION_ADDRESS	[VALUE] The Destination Address Parameter sets the Dma Channel destination address value.
MCL_DMA_CH_SET_DESTINATION_↔ SIGNED_OFFSET	[VALUE] The Destination Signed Offset Parameter sets the Dma Channel destination signed offset value.
MCL_DMA_CH_SET_DESTINATION_↔ SIGNED_LAST_ADDR_ADJ	[VALUE] The Destination Signed Last Address Adjustment Parameter sets the Dma Channel destination signed last address adjustment.
MCL_DMA_CH_SET_DESTINATION_↔ TRANSFER_SIZE	[VALUE] The Destination Transfer Size Parameter sets the Dma Channel destination transfer size.
MCL_DMA_CH_SET_DESTINATION_MODULO	[VALUE] The Destination Modulo Parameter sets the Dma Channel destination modulo.

Enumerator

MCL_DMA_CH_SET_MINORLOOP_EN_SRC↔ _OFFSET	[BOOLEAN] The Minor Loop Enable Source Offset Parameter enables the Dma Channel minor loop source offset.
MCL_DMA_CH_SET_MINORLOOP_EN_DST↔ _OFFSET	[BOOLEAN] The Minor Loop Enable Destination Offset Parameter enables the Dma Channel minor loop destination offset.
MCL_DMA_CH_SET_MINORLOOP_SIGNED↔ _OFFSET	[VALUE] The Minor Loop Signed Offset Parameter sets the Dma Channel minor loop signed offset.
MCL_DMA_CH_SET_MINORLOOP_EN_LINK	[BOOLEAN] The Minor Loop Enable Link Parameter enables the Dma Channel minor loop logic channel linking.
MCL_DMA_CH_SET_MINORLOOP_LOGIC↔ LINK_CH	[VALUE] The Minor Loop Logic Channel Link Parameter sets the Dma Channel minor loop logic channel link.
MCL_DMA_CH_SET_MINORLOOP_SIZE	[VALUE] The Minor Loop Size Parameter sets the Dma Channel minor loop transfer size.
MCL_DMA_CH_SET_MAJORLOOP_EN_LINK	[BOOLEAN] The Major Loop Enable Link Parameter enables the Dma Channel major loop logic channel linking.
MCL_DMA_CH_SET_MAJORLOOP_LOGIC↔ LINK_CH	[VALUE] The Major Loop Logic Channel Link Parameter sets the Dma Channel major loop logic channel link.
MCL_DMA_CH_SET_MAJORLOOP_COUNT	[VALUE] The Major Loop Count Parameter sets the Dma Channel major loop count.
MCL_DMA_CH_SET_CONTROL_SOFTWARE↔ _REQUEST	[BOOLEAN] The Enable Start Parameter enables the Dma Channel start service request.
MCL_DMA_CH_SET_CONTROL_EN↔ MAJOR_INTERRUPT	[BOOLEAN] The Enable Major Interrupt Parameter enables the Dma Channel major interrupt.
MCL_DMA_CH_SET_CONTROL_EN_HALF↔ MAJOR_INTERRUPT	[BOOLEAN] The Enable Half Interrupt Parameter enables the Dma Channel half major interrupt.
MCL_DMA_CH_SET_CONTROL_DIS_AUTO↔ _REQUEST	[BOOLEAN] The Disable Automatic Request Parameter disables the Dma Channel automatic request.
MCL_DMA_CH_SET_CONTROL_BANDWIDTH	[VALUE] The Bandwidth Control Parameter sets the Dma Channel bandwidth control.

Definition at line 416 of file [CDD_Mcl.h](#).

6.2.4.5 Mcl_DmaChannelStateType

```
enum Mcl_DmaChannelStateType
```

This type contains the Mcl Dma Channel State values.

The states represent the Channel status during runtime.

Definition at line 458 of file [CDD_Mcl.h](#).

6.2.4.6 Mcl_DmaChannelInfoParamType

enum [Mcl_DmaChannelInfoParamType](#)

This type contains the Mcl Dma Channel Information Parameters.

The Parameters get specific information.

Enumerator

MCL_DMA_CH_GET_SOURCE_ADDRESS	[VALUE] The Source Address Parameter gets the Dma Channel source address.
MCL_DMA_CH_GET_DESTINATION_ADDRESS	[VALUE] The Destination Address Parameter gets the Dma Channel destination address.
MCL_DMA_CH_GET_BEGIN_ITER_COUNT	[VALUE] The Begin Iteration Count Parameter gets the Dma Channel begin iteration count.
MCL_DMA_CH_GET_CURRENT_ITER_↔ COUNT	[VALUE] The Current Iteration Count Parameter gets the Dma Channel current iteration count.
MCL_DMA_CH_GET_MAJOR_INTERRUPT	[BOOLEAN] The Major Interrupt Parameter gets the Dma Channel major interrupt.
MCL_DMA_CH_GET_HALF_MAJOR_↔ INTERRUPT	[BOOLEAN] The Half Major Interrupt Parameter gets the Dma Channel half major interrupt.

Definition at line 474 of file [CDD_Mcl.h](#).

6.2.4.7 Mcl_CacheType

enum [Mcl_CacheType](#)

This type contains the Mcl Cache Type selection.

The Cache Types select specific cache memory types.

Enumerator

MCL_CACHE_LMEM	The Cache Lmem Parameter selects LMEM cache types.
MCL_CACHE_CORE	The Cache Core Parameter selects CORE cache types.

Definition at line 498 of file [CDD_Mcl.h](#).

6.2.4.8 Mcl_CacheBusType

enum [Mcl_CacheBusType](#)

This type contains the Mcl Cache Bus Type selection.

The Cache Bus Types select Code and System caches and bus.

Enumerator

MCL_CACHE_PC_BUS	The Cache PC Bus selects Processor Code (PC) bus (used with Cache Lmem).
------------------	--

Enumerator

MCL_CACHE_PS_BUS	The Cache PS Bus selects Processor System (PS) bus (used with Cache Lmem).
MCL_CACHE_ALL_BUS	The Cache All Bus selects PC and PS bus (used with Cache Lmem).
MCL_CACHE_INSTRUCTION	The Cache Instruction Parameter selects instruction cache (used with Cache Core).
MCL_CACHE_DATA	The Cache Data Parameter selects data cache (used with Cache Core).

Definition at line 510 of file [CDD_Mcl.h](#).

6.2.4.9 Mcl_ReturnType

enum [Mcl_ReturnType](#)

This type contains the Mcl Return Type.

The Return Type give information for the execution of interfaces.

Definition at line 314 of file [Mcl_Types.h](#).

6.2.5 Function Reference

6.2.5.1 Mcl_Init()

```
void Mcl_Init (
    const Mcl\_ConfigType *const ConfigPtr )
```

This function initializes the Mcl Driver.

This service is a non reentrant function that shall initialize the Mcl driver. The initialization is applied for the enabled IPs, configured statically.

Parameters

in	<i>ConfigPtr</i>	Pointer to the configuration structure.
----	------------------	---

Returns

void

6.2.5.2 Mcl_DeInit()

```
void Mcl_DeInit (
    void )
```

This function deinitializes the Mcl Driver.

This service is a non reentrant function that shall deinitialize the Mcl driver. The deinitialization is applied for the enabled IPs, configured statically.

Returns

void

6.2.5.3 Mcl_SetDmaInstanceCommand()

```
void Mcl_SetDmaInstanceCommand (
    const uint32 Instance,
    const Mcl_DmaInstanceCmdType Command )
```

This function sets Dma Instance Command.

This service is a reentrant function that shall command the Dma Instance. The command shall trigger specific functionalities of the Dma Instance.

Parameters

in	<i>Instance</i>	Selection value of the Logic Instance.
in	<i>Command</i>	The command for the Logic Instance.

Returns

void

6.2.5.4 Mcl_GetDmaInstanceStatus()

```
void Mcl_GetDmaInstanceStatus (
    const uint32 Instance,
    Mcl_DmaInstanceStatusType *const Status )
```

This function gets Dma Instance Status.

This service is a reentrant function that shall get the Dma Instance status. The command shall read specific functionalities of the Dma Instance.

Parameters

in	<i>Instance</i>	Selection value of the Logic Instance.
out	<i>Status</i>	Pointer to the Dma Instance status.

Returns

void

6.2.5.5 Mcl_SetDmaChannelCommand()

```
void Mcl_SetDmaChannelCommand (
    const uint32 Channel,
    const Mcl_DmaChannelCmdType Command )
```

This function sets Dma Channel Command.

This service is a reentrant function that shall command the Dma Channel. The command shall trigger specific functionalities of the Dma Channel.

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
in	<i>Command</i>	The command for the Logic Channel.

Returns

void

6.2.5.6 Mcl_GetDmaChannelStatus()

```
void Mcl_GetDmaChannelStatus (
    const uint32 Channel,
    Mcl_DmaChannelStatusType *const Status )
```

This function gets Dma Channel Status.

This service is a reentrant function that shall get the Dma Channel status. The command shall read specific functionalities of the Dma Channel.

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
out	<i>Status</i>	Pointer to the Dma Channel status.

Returns

void

6.2.5.7 Mcl_SetDmaChannelGlobalList()

```
void Mcl_SetDmaChannelGlobalList (
    const uint32 Channel,
    const Mcl_DmaChannelGlobalListType List[],
    const uint32 ListDimension )
```

This function sets Dma Channel Global List settings.

This service is a reentrant function that shall set the Dma Channel global parameters list. The list is composed of an array of Dma Channel global parameters settings. The settings list(array) is defined by the user needs: it contains the desired parameters to be configured, in any desired order.

How to use this interface:

1. Use the "Mcl_DmaChannelGlobalListType" to create a list(array) with the desired paramaters to configure (see parameters: "Mcl_DmaChannelGlobalParamType") The list can declared globally or locally: Global example: `Mcl_DmaChannelGlobalListType global_Mcl_DmaChannelGlobalList0[NUMBER_OF_PARAMETERS] = {...};` Local example: `Mcl_DmaChannelGlobalListType Mcl_DmaChannelGlobalList[NUMBER_OF_PARAMETERS]; Mcl_DmaChannelGlobalList[PARAMETER0].Param = MCL_DMA_CH_SET_EN_BUFFERED_WRITES; Mcl_DmaChannelGlobalList[PARAMETER0].Value = TRUE; Mcl_DmaChannelGlobalList[PARAMETER1].Param = ...; Mcl_DmaChannelGlobalList[PARAMETER1].Value = ...;`
2. Call the "Mcl_SetDmaChannelGlobalList()" interface: `Mcl_SetDmaChannelGlobalList(LOGIC_CHANNELx, Mcl_DmaChannelGlobalList, NUMBER_OF_PARAMETERS);`

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
in	<i>List</i>	Pointer to the Global List Array.
in	<i>ListDimension</i>	Number of entries in the List.

Returns

void

6.2.5.8 Mcl_SetDmaChannelTransferList()

```
void Mcl_SetDmaChannelTransferList (
    const uint32 Channel,
    const Mcl_DmaChannelTransferListType List[],
    const uint32 ListDimension )
```

This function sets Dma Channel Transfer List settings.

This service is a reentrant function that shall set the Dma Channel transfer parameters list. The list is composed of an array of Dma Channel transfer parameters settings. The settings array is defined by the user needs: it contains entries for each desired parameter to be configured, in any suitable order.

How to use this interface:

1. Use the "Mcl_DmaChannelTransferListType" to create a list(array) with the desired paramaters to configure (see parameters: "Mcl_DmaChannelTransferParamType") The list can declared globally or locally: Global example: `Mcl_DmaChannelTransferListType global_Mcl_DmaChannelTransferList0[NUMBER_OF_PARAMETERS] = {...};` Local example: `Mcl_DmaChannelTransferListType Mcl_DmaChannelTransferList[NUMBER_OF_PARAMETERS]; Mcl_DmaChannelTransferList[PARAMETER0].Param = MCL_DMA_CH_SET_SOURCE_ADDRESS; Mcl_DmaChannelTransferList[PARAMETER0].Value = &SourceBuffer; Mcl_DmaChannelTransferList[PARAMETER1].Param = MCL_DMA_CH_SET_DESTINATION_ADDRESS; Mcl_DmaChannelTransferList[PARAMETER1].Value = &DestinationBuffer;`
2. Call the "Mcl_SetDmaChannelTransferList()" interface: `Mcl_SetDmaChannelTransferList(LOGIC_CHANNELx, Mcl_DmaChannelTransferList, NUMBER_OF_PARAMETERS);`

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
in	<i>List</i>	Pointer to the Transfer List Array.
in	<i>ListDimension</i>	Number of entries in the List.

Returns

void

6.2.5.9 Mcl_SetDmaChannelScatterGatherList()

```
void Mcl_SetDmaChannelScatterGatherList (
    const uint32 Channel,
    const uint32 Element,
    const Mcl_DmaChannelScatterGatherListType List[],
```

```
const uint32 ListDimension )
```

This function sets Dma Channel Scatter/Gather List settings.

This service is a reentrant function that shall set the Dma Channel scatter/gather parameters list. The Scatter/Gather List configures Logic Elements belonging to the same Dma Logic Channel. The settings array is defined by the user needs: it contains entries for each desired parameter to be configured, in any suitable order.

How to use this interface:

1. Use the "Mcl_DmaChannelScatterGatherListType" to create a list(array) with the desired parameters to configure (see parameters: "Mcl_DmaChannelTransferParamType") The list can be declared globally or locally: Global example: [Mcl_DmaChannelScatterGatherListType](#) global_Mcl_DmaChannelScatterGatherList0[NUMBER_OF_PARAMETERS] = {...}; Local example: [Mcl_DmaChannelScatterGatherListType](#) Mcl_DmaChannelScatterGatherList[NUMBER_OF_PARAMETERS]; Mcl_DmaChannelScatterGatherList[PARAMETER0].Param = MCL_DMA_CH_SET_SOURCE_ADDRESS; Mcl_DmaChannelScatterGatherList[PARAMETER0].Value = &SourceBuffer; Mcl_DmaChannelScatterGatherList[PARAMETER1].Param = MCL_DMA_CH_SET_DESTINATION_ADDRESS; Mcl_DmaChannelScatterGatherList[PARAMETER1].Value = &DestinationBuffer;
2. Call the "Mcl_SetDmaChannelScatterGatherList()" interface: Mcl_SetDmaChannelScatterGatherList(← LOGIC_CHANNELx, LOGIC_ELEMENTy, Mcl_DmaChannelScatterGatherList, NUMBER_OF_← PARAMETERS);

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
in	<i>Element</i>	Specifies the Logic Element Id.
in	<i>List</i>	Pointer to the Scatter/Gather List Array.
in	<i>ListDimension</i>	Number of entries in the List.

Returns

void

6.2.5.10 Mcl_GetDmaChannelParam()

```
void Mcl_GetDmaChannelParam (
    const uint32 Channel,
    const Mcl_DmaChannelInfoParamType Param,
    uint32 *const Value )
```

This function gets the Dma Channel Parameter value.

This service is a reentrant function that shall get the Dma Channel parameters value.

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
in	<i>Param</i>	Selection parameter.
out	<i>Value</i>	Pointer to the parameter value.

Returns

void

6.2.5.11 Mcl_SetDmaChannelScatterGatherConfig()

```
void Mcl_SetDmaChannelScatterGatherConfig (
    const uint32 Channel,
    const uint32 Element )
```

This function configures the Dma Channel Scatter/Gather.

This service is a reentrant function that shall configure the Dma Channel scatter/gather functionality. The Scatter/Gather settings, for the specified Dma Logic Channel, are loaded into the Software TCDs. Each software TCD corresponds to a Logic Element. The specified Logic Element shall be loaded into the Dma Logic Channel's Hardware TCD. The Logic Elements (describing the Software TCDs) form a simple chained list, the "Element" function parameter representing the lists's head.

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Tag defined by the user.
in	<i>Element</i>	Specifies the Logic Element Id representing the list's head.

Returns

void

6.2.5.12 Mcl_CacheEnable()

```
void Mcl_CacheEnable (
    Mcl_CacheType CacheType,
    Mcl_CacheBusType BusType )
```

This function enables the Cache.

This service is a reentrant function that shall enable the Cache functionality.

Parameters

in	<i>CacheType</i>	Selection value of the Cache Type.
in	<i>BusType</i>	Selection value of the Bus Type.

Returns

void

6.2.5.13 Mcl_CacheDisable()

```
void Mcl_CacheDisable (
    Mcl_CacheType CacheType,
    Mcl_CacheBusType BusType )
```

This function disables the Cache.

This service is a reentrant function that shall disable the Cache functionality.

Parameters

in	<i>CacheType</i>	Selection value of the Cache Type.
in	<i>BusType</i>	Selection value of the Bus Type.

Returns

void

6.2.5.14 Mcl_CacheInvalidate()

```
void Mcl_CacheInvalidate (
    Mcl_CacheType CacheType,
    Mcl_CacheBusType BusType )
```

This function Invalidates the Cache.

This service is a reentrant function that shall Invalidate the Cache functionality. The Invalidation applies to the entire Cache.

Parameters

in	<i>CacheType</i>	Selection value of the Cache Type.
in	<i>BusType</i>	Selection value of the Bus Type.

Returns

void

6.2.5.15 Mcl_CacheClean()

```
void Mcl_CacheClean (
    Mcl_CacheType CacheType,
    Mcl_CacheBusType BusType,
    boolean EnInvalidate )
```

This function Cleans the Cache.

This service is a reentrant function that shall Clean the Cache functionality. The Clean applies to the entire Cache. By enabling the Invalidation, the function shall execute specific Cache Clean&Invalidate function.

Parameters

in	<i>CacheType</i>	Selection value of the Cache Type.
in	<i>BusType</i>	Selection value of the Bus Type.
in	<i>EnInvalidate</i>	Enable the Invalidation specific functionality.

Returns

void

6.2.5.16 Mcl_CacheInvalidateByAddr()

```
void Mcl_CacheInvalidateByAddr (
    Mcl_CacheType CacheType,
    Mcl_CacheBusType BusType,
    uint32 Addr,
    uint32 Length )
```

This function Invalidates the Cache by address.

This service is a reentrant function that shall Invalidate the Cache by address. The Invalidation applies to the area in Cache specified by the address and length. The buffer shall be aligned to the Cache Line size.

Parameters

in	<i>CacheType</i>	Selection value of the Cache Type.
in	<i>BusType</i>	Selection value of the Bus Type.
in	<i>Addr</i>	Address value of the buffer.
in	<i>Length</i>	Length value of the buffer.

Returns

void

6.2.5.17 Mcl_CacheCleanByAddr()

```
void Mcl_CacheCleanByAddr (
    Mcl_CacheType CacheType,
    Mcl_CacheBusType BusType,
    boolean EnInvalidate,
    uint32 Addr,
    uint32 Length )
```

This function Cleans the Cache by address.

This service is a reentrant function that shall Clean the Cache by address. The Clean applies to the area in Cache specified by the address and length. By enabling the Invalidation, the function shall execute specific Cache Clean&Invalidate function.

Parameters

in	<i>CacheType</i>	Selection value of the Cache Type.
in	<i>BusType</i>	Selection value of the Bus Type.
in	<i>EnInvalidate</i>	Enable the Invalidation specific functionality.
in	<i>Addr</i>	Address value of the buffer.
in	<i>Length</i>	Length value of the buffer.

Returns

void

6.2.5.18 Mcl_SetTrgMuxInput()

```
void Mcl_SetTrgMuxInput (
    const uint32 Trigger,
    const uint32 Input )
```

This function sets the Trgmux Trigger Input selection.

This service is a reentrant function that shall configure the Trgmux Trigger functionality.

Parameters

in	<i>Trigger</i>	Selection value of the Logic Trigger.
in	<i>Input</i>	Selection value for the Logic Trigger's Input.

Returns

void

6.2.5.19 Mcl_SetTrgMuxLock()

```
void Mcl_SetTrgMuxLock (
    const uint32 Trigger )
```

This function sets the Trgmux Trigger Lock.

This service is a reentrant function that shall configure the Trgmux Trigger Lock functionality.

Parameters

in	<i>Trigger</i>	Selection value of the Logic Trigger.
----	----------------	---------------------------------------

Returns

void

6.2.5.20 Mcl_SelectCommonTimebase()

```
void Mcl_SelectCommonTimebase (
    uint8 instance,
    uint16 elementSyncList )
```

Implementation specific function to updates the Global Timebase bits of configured modules.

This function is used to set the global timebase bits for modules that support the global timebase feature. The function selects the module that gives the common timebase and the modules that are use this timebase (as bits in elementSyncList). Then it synchronizes the modules. example: elementSyncList is 0x0003 - modules 0 and 1 use the timebase given by instance elementSyncList is 0x0005 - modules 0 and 2 use the timebase given by instance

Parameters

in	<i>instance</i>	FTM module id
in	<i>elementSyncList</i>	FTM module mask value

Returns

void

6.2.5.21 Mcl_GetVersionInfo()

```
void Mcl_GetVersionInfo (
    Std_VersionInfoType *const VersionInfo )
```

Returns the version information of this module.

Returns the version information of MCL module.

Parameters

out	<i>VersionInfo</i>	A pointer to a structure used to get version information.
-----	--------------------	---

Returns

void

6.3 DMA IP Driver

6.3.1 Detailed Description

Data Structures

- struct [Dma_Ip_LogicChannelGlobalListType](#)
This type contains the Dma Ip Logic Channel Global List. [More...](#)
- struct [Dma_Ip_LogicChannelTransferListType](#)
This type contains the Dma Ip Channel Transfer List. [More...](#)
- struct [Dma_Ip_LogicInstanceStatusType](#)
This type contains the Dma Ip Instance Status. [More...](#)
- struct [Dma_Ip_LogicChannelStatusType](#)
This type contains the Dma Ip Channel Status. [More...](#)
- struct [Dma_Ip_LogicChannelIdType](#)
This type contains the Dma Ip Logic Channel Identification. [More...](#)
- struct [Dma_Ip_GlobalConfigType](#)
This type contains the Dma Ip Global Configuration. [More...](#)
- struct [Dma_Ip_TransferConfigType](#)
This type contains the Dma Ip Transfer Configuration. [More...](#)
- struct [Dma_Ip_ScatterGatherConfigType](#)
This type contains the Dma Ip Scatter/Gather Configuration. [More...](#)

- struct [Dma_Ip_LogicChannelConfigType](#)
This type contains the Dma Ip Logic Channel Configuration. [More...](#)
- struct [Dma_Ip_LogicInstanceIdType](#)
This type contains the Dma Ip Logic Instance Identification. [More...](#)
- struct [Dma_Ip_LogicInstanceConfigType](#)
This type contains the Dma Ip Logic Instance Configuration. [More...](#)
- struct [Dma_Ip_HwChannelStateType](#)
This type contains the Dma Ip Hardware Channel State. [More...](#)
- struct [Dma_Ip_VirtualSectionConfigType](#)
This type contains the Dma Ip Virtual memory section configuration Type. [More...](#)
- struct [Dma_Ip_VirtualMemoryConfigType](#)
This type contains the Dma Ip Virtual memory configuration Type. [More...](#)
- struct [Dma_Ip_InitType](#)
This type contains the Dma Ip Initialization. [More...](#)

Types Reference

- typedef void(* [Dma_Ip_Callback](#)) (void)
This type contains the Dma Ip Callback interface.

Enum Reference

- enum [Dma_Ip_LogicInstanceCmdType](#)
This type contains the Dma Ip Logic Instance Commands.
- enum [Dma_Ip_LogicChannelCmdType](#)
This type contains the Dma Ip Logic Channel Commands.
- enum [Dma_Ip_LogicChannelGlobalParamType](#)
This type contains the Dma Ip Logic Channel Global Parameters.
- enum [Dma_Ip_LogicChannelTransferParamType](#)
This type contains the Dma Ip Logic Channel Transfer Parameters.
- enum [Dma_Ip_LogicChannelInfoParamType](#)
This type contains the Dma Ip Logic Channel Information Parameters.
- enum [Dma_Ip_ReturnType](#)
This type contains the Dma Ip Return Type.
- enum [Dma_Ip_HwChannelStateValueType](#)
This type contains the Dma Ip Channel State Value Type.

Function Reference

- [Dma_Ip_ReturnType Dma_Ip_Init](#) (const [Dma_Ip_InitType](#) *const DmaInit)
This function initializes the Dma Ip Driver.
- [Dma_Ip_ReturnType Dma_Ip_Deinit](#) (void)
This function deinitializes the Dma Ip Driver.

- [Dma_Ip_ReturnType Dma_Ip_SetLogicInstanceCommand](#) (const uint32 LogicInst, const [Dma_Ip_LogicInstanceCmdType](#) Command)

This function sets Dma Ip Instance Command.

- [Dma_Ip_ReturnType Dma_Ip_GetLogicInstanceStatus](#) (const uint32 LogicInst, [Dma_Ip_LogicInstanceStatusType](#) *const Status)

This function gets Dma Ip Instance Status.

- [Dma_Ip_ReturnType Dma_Ip_LogicChannelInit](#) (const uint32 LogicCh)

This function initializes the Dma Ip Logic Channel.

- [Dma_Ip_ReturnType Dma_Ip_LogicChannelDeinit](#) (const uint32 LogicCh)

This function deinitializes the Dma Ip Logic Channel.

- [Dma_Ip_ReturnType Dma_Ip_SetLogicChannelCommand](#) (const uint32 LogicCh, const [Dma_Ip_LogicChannelCmdType](#) Command)

This function sets Dma Ip Logic Channel Command.

- [Dma_Ip_ReturnType Dma_Ip_GetLogicChannelStatus](#) (const uint32 LogicCh, [Dma_Ip_LogicChannelStatusType](#) *const ChStatus)

This function gets Dma Ip Logic Channel Status.

- [Dma_Ip_ReturnType Dma_Ip_SetLogicChannelGlobalList](#) (const uint32 LogicCh, const [Dma_Ip_LogicChannelGlobalListType](#) List[], const uint32 ListDimension)

This function sets Dma Ip Logic Channel Global List settings.

- [Dma_Ip_ReturnType Dma_Ip_SetLogicChannelTransferList](#) (const uint32 LogicCh, const [Dma_Ip_LogicChannelTransferListType](#) List[], const uint32 ListDimension)

This function sets Dma Ip Logic Channel Transfer List settings.

- [Dma_Ip_ReturnType Dma_Ip_SetLogicChannelScatterGatherList](#) (const uint32 LogicCh, const uint32 Element, const [Dma_Ip_LogicChannelScatterGatherListType](#) List[], const uint32 ListDimension)

This function sets Dma Ip Logic Channel Scatter/Gather List settings.

- [Dma_Ip_ReturnType Dma_Ip_GetLogicChannelParam](#) (const uint32 LogicCh, const [Dma_Ip_LogicChannelInfoParamType](#) Param, uint32 *const Value)

This function gets the Dma Ip Logic Channel Parameter value.

- [Dma_Ip_ReturnType Dma_Ip_SetLogicChannelScatterGatherConfig](#) (const uint32 LogicCh, const uint32 Element)

This function configures the Dma Ip Logic Channel Scatter/Gather.

6.3.2 Data Structure Documentation

6.3.2.1 struct [Dma_Ip_LogicChannelGlobalListType](#)

This type contains the Dma Ip Logic Channel Global List.

The Dma Ip Channel Global List contains a pair composed from Dma Channel Global Parameter Type and the Value of the parameter. The Dma Ip Channel Global Parameter Type selects a parameter form the Global Parameter enum type. The Value stores the parameter's value.

Definition at line 240 of file [Dma_Ip.h](#).

6.3.2.2 struct Dma_Ip_LogicChannelTransferListType

This type contains the Dma Ip Channel Transfer List.

The Dma Ip Channel Transfer List contains a pair composed from Dma Channel Transfer Parameter Type and the Value of the parameter. The Dma Ip Channel Transfer Parameter Type selects a parameter form the Transfer Parameter enum type. The Value stores the parameter's value.

This type contains the Dma Ip Channel ScatterGather List.

The Dma Ip Channel Transfer List contains a pair composed from Dma Channel ScatterGather Parameter Type and the Value of the parameter. The Dma Ip Channel ScatterGather Parameter Type selects a parameter form the ScatterGather Parameter enum type. The Value stores the parameter's value.

Definition at line 264 of file [Dma_Ip.h](#).

6.3.2.3 struct Dma_Ip_LogicInstanceStatusType

This type contains the Dma Ip Instance Status.

The Dma Ip Instance Status contains the Hardware Errors, Active Id and Active indication for the running Dma Channel. The Errors shall contain the Hardware Errors. The ActiveId shall contain the running Dma Channel Id. The Active shall contain the running Dma Channel Active status.

Definition at line 281 of file [Dma_Ip.h](#).

Data Fields

- uint32 [Errors](#)
[VALUE] The Errors value is read from the DMA Instance Error Register (ES) as it is.
- uint8 [ActiveId](#)
[VALUE] The ActiveId value is read from the DMA Instance Control Register (CR) field ACTIVE_ID.
- boolean [Active](#)
[BOOLEAN] The Active value is read from the DMA Instance Control Register (CR) field ACTIVE.
- uint8 [Version](#)
[VALUE] The Version value is read from the DMA Instance Control Register (CR) field VER.

Field Documentation

Errors

uint32 Errors

[VALUE] The Errors value is read from the DMA Instance Error Register (ES) as it is.

Definition at line 283 of file [Dma_Ip.h](#).

ActiveId

uint8 ActiveId

[VALUE] The ActiveId value is read from the DMA Instance Control Register (CR) field ACTIVE_ID.

Definition at line 284 of file [Dma_Ip.h](#).

Active

boolean Active

[BOOLEAN] The Active value is read from the DMA Instance Control Register (CR) field ACTIVE.

Definition at line 285 of file [Dma_Ip.h](#).

Version

uint8 Version

[VALUE] The Version value is read from the DMA Instance Control Register (CR) field VER.

Definition at line 286 of file [Dma_Ip.h](#).

6.3.2.4 struct Dma_Ip_LogicChannelStatusType

This type contains the Dma Ip Channel Status.

The Dma Ip Channel Status contains the Hardware Errors, Active status and Done indication for the running Dma Channel. The Channel State Value shall contain the internal driver state of the Dma Channel. The Errors shall contain the Hardware Dma Channel Errors. The Active shall contain the running Dma Channel Id. The Done shall contain the running Dma Channel Active status.

Definition at line 300 of file [Dma_Ip.h](#).

Data Fields

- [Dma_Ip_HwChannelStateValueType ChStateValue](#)

[VALUE] The ChStateValue value is read from the internal DMA Driver Channel State Machine. Check UM for additional information.

- uint32 [Errors](#)

[VALUE] The Errors value is read from the DMA Channel Error Register (CHx_ES) as it is.

- boolean [Active](#)

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field ACTIVE.

- boolean [Done](#)

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field DONE.

Field Documentation

ChStateValue

[Dma_Ip_HwChannelStateValueType](#) ChStateValue

[VALUE] The ChStateValue value is read from the internal DMA Driver Channel State Machine. Check UM for additional information.

Definition at line 302 of file [Dma_Ip.h](#).

Errors

uint32 Errors

[VALUE] The Errors value is read from the DMA Channel Error Register (CHx_ES) as it is.

Definition at line 304 of file [Dma_Ip.h](#).

Active

boolean Active

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field ACTIVE.

Definition at line 305 of file [Dma_Ip.h](#).

Done

boolean Done

[BOOLEAN] The Active value is read from the DMA Channel Control and Status Register (CHx_CSR) field DONE.

Definition at line 306 of file [Dma_Ip.h](#).

6.3.2.5 struct Dma_Ip_LogicChannelIdType

This type contains the Dma Ip Logic Channel Identification.

The Logic Channel is identified by the following structure: The Logic Channel Id contains the ID value. The Hardware Version Id contains the DMA Hardware Ip Block Version. The Hardware Instance Id contains the DMA Hardware Instance identification. The Hardware Channel Id contains the DMA Hardware Channel identification. The Hardware Crc Instance Id contains the DMA Hardware Crc Instance identification if CRC_IP is available. The Hardware Crc Channel Id contains the DMA Hardware Crc Channel identification if CRC_IP is available. The Interrupt Callback stores pointer to the user defined interrupt callback. The Error Interrupt Callback stores pointer to the user defined error interrupt callback.

Definition at line 181 of file [Dma_Ip_Types.h](#).

Data Fields

- uint32 [LogicChId](#)
DMA logic channel number.
- uint8 [HwVersId](#)
DMA hardware version.
- uint8 [HwInstId](#)
DMA hardware instance number.
- uint8 [HwChId](#)
DMA hardware channel number.
- [Dma_Ip_Callback](#) [IntCallback](#)
The channel callback is installed in the interrupt and is called automatically from the interrupt every time it is triggered.
- [Dma_Ip_Callback](#) [ErrIntCallback](#)

Module Documentation

The channel error callback is installed in the error interrupt and is called automatically from the interrupt every time it is triggered.

Field Documentation

LogicChId

uint32 LogicChId

DMA logic channel number.

Definition at line 183 of file [Dma_Ip_Types.h](#).

HwVersId

uint8 HwVersId

DMA hardware version.

Definition at line 184 of file [Dma_Ip_Types.h](#).

HwInstId

uint8 HwInstId

DMA hardware instance number.

Definition at line 185 of file [Dma_Ip_Types.h](#).

HwChId

uint8 HwChId

DMA hardware channel number.

Definition at line 186 of file [Dma_Ip_Types.h](#).

IntCallback

[Dma_Ip_Callback](#) IntCallback

The channel callback is installed in the interrupt and is called automatically from the interrupt every time it is triggered.

Definition at line 187 of file [Dma_Ip_Types.h](#).

ErrIntCallback

[Dma_Ip_Callback](#) ErrIntCallback

The channel error callback is installed in the error interrupt and is called automatically from the interrupt every time it is triggered.

Definition at line 189 of file [Dma_Ip_Types.h](#).

6.3.2.6 struct Dma_Ip_GlobalConfigType

This type contains the Dma Ip Global Configuration.

The Global Configuration of an Logic Channel, contains all the information describing a channel, but are not present in the DMA TCD. It contains configuration split in four main areas: Control defines functionality covering the channel's bus control. Request defines functionality covering the channel's request interface Interrupt defines functionality covering special channel interrupts. Priority defines functionality covering the channel's priority mechanism.

Definition at line 205 of file [Dma_Ip_Types.h](#).

6.3.2.7 struct Dma_Ip_TransferConfigType

This type contains the Dma Ip Transfer Configuration.

The Transfer Configuration of an Logic Channel, contains all the information describing a channel transfer functionalities and are present in the DMA TCD. It contains configuration split in five main areas: Control defines functionality covering the channel control functions. It includes additionally the Scatter/Gather Address and Destination Store Address.

Definition at line 244 of file [Dma_Ip_Types.h](#).

6.3.2.8 struct Dma_Ip_ScatterGatherConfigType

This type contains the Dma Ip Scatter/Gather Configuration.

The Scatter/Gather Configuration of a Logic Channel, contains all the information describing the channel's needed resources for a Scatter/Gather element. It contains a pointer to the transfer configuration, a pointer to a Software TCD, a pointer to the next Scatter/Gather configuration and a status flag for loading the transfer configuration into the Software TCD.

Definition at line 309 of file [Dma_Ip_Types.h](#).

Data Fields

Dma_Ip_TransferConfigType *	TransferConfig	Configuration that shall be loaded into the Software TCD.
Dma_Ip_SwTcdRegType *	Stcd	Software TCD shall be loaded with own TCD configuration.
boolean	Loaded	
struct Dma_Ip_ScatterGatherConfigType *	NextConfig	Next Logic Channel configuration. If the address is not NULL, then the ESG flag is enabled (Scatter/Gather address) and the destination adjustment is disabled. The next configuration address is stored.

6.3.2.9 struct Dma_Ip_LogicChannelConfigType

This type contains the Dma Ip Logic Channel Configuration.

The Logic Channel Configuration consists of the Logic Channel Identifier, pointer to the Global Configuration, pointer to the Transfer Configuration and pointer to the Scatter/Gather configuration. The Logic Channel Configuration contains all data needed to initialize a Logic Channel.

Definition at line 329 of file [Dma_Ip_Types.h](#).

Data Fields

Dma_Ip_LogicChannelIdType	LogicChId	The Logic Channel ID contains configuration information and identification.
const Dma_Ip_GlobalConfigType *	pxGlobalConfig	The Global Configuration pointer shall contain the global data.
const Dma_Ip_TransferConfigType *	pxTransferConfig	The Transfer Configuration pointer shall contain the transfer data.
Dma_Ip_ScatterGatherConfigType **	ppxScatterGatherConfigArray	The Scatter/Gather Configuration pointer shall contain a pointer to an array containing all Scatter/Gather Logic Elements.

6.3.2.10 struct Dma_Ip_LogicInstanceIdType

This type contains the Dma Ip Logic Instance Identification.

The Logic Instance is identified by the following structure: The Logic Instance Id contains the ID value. The Hardware Version Id contains the DMA Hardware Ip Block Version. The Hardware Instance Id contains the DMA Hardware Instance identification.

Definition at line 346 of file [Dma_Ip_Types.h](#).

Data Fields

uint32	LogicInstId	DMA logic instance number.
uint8	HwVersId	DMA hardware version number.
uint8	HwInstId	DMA hardware instance number.

6.3.2.11 struct Dma_Ip_LogicInstanceConfigType

This type contains the Dma Ip Logic Instance Configuration.

The Logic Instance Configuration contains all the information describing an instance functionality.

Definition at line 359 of file [Dma_Ip_Types.h](#).

Data Fields

Dma_Ip_LogicInstanceIdType	LogicInstId	DMA logic instance number.
boolean	EnDebug	When in debug mode, the DMA stalls the start of a new channel. Executing channels are allowed to complete. DMA resumes channel execution when the system exits debug mode or clears the EDBG field to 0.
boolean	EnRoundRobin	Enable Round Robin Channel Arbitration.

Data Fields

boolean	EnHaltAfterError	When this field is set to 1, any error causes the HALT field to be set to 1. Then all service requests are ignored until the HALT field is cleared to 0.
boolean	EnChLinking	Global Channel Linking Control.
boolean	EnGIMasterIdReplication	If master ID replication is disabled, the privileged protection level (Supervisor mode) for DMA transfers is used.

6.3.2.12 struct Dma_Ip_HwChannelStateType

This type contains the Dma Ip Hardware Channel State.

The Hardware Channel State contains the channel's state based on runtime actions. The structure links the hardware channel state with the Logic Channel.

Definition at line 378 of file [Dma_Ip_Types.h](#).

6.3.2.13 struct Dma_Ip_VirtualSectionConfigType

This type contains the Dma Ip Virtual memory section configuration Type.

Definition at line 402 of file [Dma_Ip_Types.h](#).

6.3.2.14 struct Dma_Ip_VirtualMemoryConfigType

This type contains the Dma Ip Virtual memory configuration Type.

Definition at line 414 of file [Dma_Ip_Types.h](#).

6.3.2.15 struct Dma_Ip_InitType

This type contains the Dma Ip Initialization.

The Dma Ip Initialization contains all the information required to initialize the Dma Peripheral. Each pointer shall be loaded with a specific configuration used by the Dma.

Definition at line 428 of file [Dma_Ip_Types.h](#).

6.3.3 Types Reference**6.3.3.1 Dma_Ip_Callback**

```
typedef void(* Dma_Ip_Callback) (void)
```

This type contains the Dma Ip Callback interface.

The Callback is defined by the user and installed by the driver in the corresponding IRQ.

Returns

void

Definition at line 165 of file [Dma_Ip_Types.h](#).

6.3.4 Enum Reference

6.3.4.1 Dma_Ip_LogicInstanceCmdType

enum [Dma_Ip_LogicInstanceCmdType](#)

This type contains the Dma Ip Logic Instance Commands.

The Commands trigger specific actions in the Dma Logic Instance.

Enumerator

DMA_IP_INST_CANCEL_TRANSFER	The Cancel Transfer cancels the executing channel and forces the Minor Loop to finish.
DMA_IP_INST_CANCEL_TRANSFER_WITH_ERROR	The Cancel Transfer With Error Command cancels the executing channel, forces the Minor Loop to finish and generates an error interrupt.
DMA_IP_INST_HALT	The Halt Command allows the ongoing transfer to finish and halts any new transfer.
DMA_IP_INST_RESUME	The Resume Command allows the transfer to continue.

Definition at line 105 of file [Dma_Ip.h](#).

6.3.4.2 Dma_Ip_LogicChannelCmdType

enum [Dma_Ip_LogicChannelCmdType](#)

This type contains the Dma Ip Logic Channel Commands.

The Commands trigger specific actions in the Dma Ip Logic Channel.

Enumerator

DMA_IP_CH_SET_HARDWARE_REQUEST	The Set Hardware Request Command enables the Dma Channel to be triggered by hardware requests.
DMA_IP_CH_CLEAR_HARDWARE_REQUEST	The Clear Hardware Request Command disables the Dma Channel to be triggered by hardware requests.
DMA_IP_CH_SET_SOFTWARE_REQUEST	The Set Software Request Command sends a soft start request to the Dma Channel.
DMA_IP_CH_CLEAR_DONE	The Clear Done Command resets the Dma Channel Done status.
DMA_IP_CH_CLEAR_ERROR	The Clear Error Command resets the Dma Channel Error status.
DMA_IP_CH_CLEAR_INTERRUPT_STATUS	The Clear Interrupt Status Command clears the Dma Channel Interrupt status.

Definition at line 118 of file [Dma_Ip.h](#).

6.3.4.3 Dma_Ip_LogicChannelGlobalParamType

enum [Dma_Ip_LogicChannelGlobalParamType](#)

This type contains the Dma Ip Logic Channel Global Parameters.

The Parameters set specific functionalities for the Dma Ip Logic Channel.

Enumerator

DMA_IP_CH_SET_EN_HARDWARE_REQ	[BOOLEAN] The EnRequest Parameter enables the Dma Channel Request.
DMA_IP_CH_SET_EN_ERROR_INTERRUPT	[BOOLEAN] The EnError Parameter enables the Dma Channel Error Interrupt.
DMA_IP_CH_SET_GROUP_PRIORITY	[VALUE] The Group Parameter sets the Dma Channel Group Priority.
DMA_IP_CH_SET_LEVEL_PRIORITY	[VALUE] The Level Parameter sets the Dma Channel Level Priority.
DMA_IP_CH_SET_EN_PREEMPTION_ PRIORITY	[BOOLEAN] The EnPreemption Parameter enables the Dma Channel Preemption.
DMA_IP_CH_SET_DIS_PREEMPT_PRIORITY	[BOOLEAN] The DisPreempt Parameter disables the Dma Channel Preempt.

Definition at line 133 of file [Dma_Ip.h](#).

6.3.4.4 Dma_Ip_LogicChannelTransferParamType

enum [Dma_Ip_LogicChannelTransferParamType](#)

This type contains the Dma Ip Logic Channel Transfer Parameters.

The Parameters set specific functionalities.

Enumerator

DMA_IP_CH_SET_SOURCE_ADDRESS	[VALUE] The Source Address Parameter sets the Dma Channel source address value.
DMA_IP_CH_SET_SOURCE_SIGNED_OFFSET	[VALUE] The Source Signed Offset Parameter sets the Dma Channel source signed offset value.
DMA_IP_CH_SET_SOURCE_SIGNED_LAST_ _ADDR_ADJ	[VALUE] The Source Signed Last Address Adjustment Parameter sets the Dma Channel source signed last address adjustment.
DMA_IP_CH_SET_SOURCE_TRANSFER_SIZE	[VALUE] The Source Transfer Size Parameter sets the Dma Channel source transfer size.
DMA_IP_CH_SET_SOURCE_MODULO	[VALUE] The Source Modulo Parameter sets the Dma Channel source modulo.
DMA_IP_CH_SET_DESTINATION_ADDRESS	[VALUE] The Destination Address Parameter sets the Dma Channel destination address value.
DMA_IP_CH_SET_DESTINATION_SIGNED_ OFFSET	[VALUE] The Destination Signed Offset Parameter sets the Dma Channel destination signed offset value.

Enumerator

DMA_IP_CH_SET_DESTINATION_SIGNED_↔ LAST_ADDR_ADJ	[VALUE] The Destination Signed Last Address Adjustment Parameter sets the Dma Channel destination signed last address adjustment.
DMA_IP_CH_SET_DESTINATION_↔ TRANSFER_SIZE	[VALUE] The Destination Transfer Size Parameter sets the Dma Channel destination transfer size.
DMA_IP_CH_SET_DESTINATION_MODULO	[VALUE] The Destination Modulo Parameter sets the Dma Channel destination modulo.
DMA_IP_CH_SET_MINORLOOP_EN_SRC_↔ OFFSET	[BOOLEAN] The Minor Loop Enable Source Offset Parameter enables the Dma Channel minor loop source offset.
DMA_IP_CH_SET_MINORLOOP_EN_DST_↔ OFFSET	[BOOLEAN] The Minor Loop Enable Destination Offset Parameter enables the Dma Channel minor loop destination offset.
DMA_IP_CH_SET_MINORLOOP_SIGNED_↔ OFFSET	[VALUE] The Minor Loop Signed Offset Parameter sets the Dma Channel minor loop signed offset.
DMA_IP_CH_SET_MINORLOOP_EN_LINK	[BOOLEAN] The Minor Loop Enable Link Parameter enables the Dma Channel minor loop logic channel linking.
DMA_IP_CH_SET_MINORLOOP_LOGIC_↔ LINK_CH	[VALUE] The Minor Loop Logic Channel Link Parameter sets the Dma Channel minor loop logic channel link.
DMA_IP_CH_SET_MINORLOOP_SIZE	[VALUE] The Minor Loop Size Parameter sets the Dma Channel minor loop transfer size.
DMA_IP_CH_SET_MAJORLOOP_EN_LINK	[BOOLEAN] The Major Loop Enable Link Parameter enables the Dma Channel major loop logic channel linking.
DMA_IP_CH_SET_MAJORLOOP_LOGIC_↔ LINK_CH	[VALUE] The Major Loop Logic Channel Link Parameter sets the Dma Channel major loop logic channel link.
DMA_IP_CH_SET_MAJORLOOP_COUNT	[VALUE] The Major Loop Count Parameter sets the Dma Channel major loop count.
DMA_IP_CH_SET_CONTROL_SOFTWARE_↔ REQUEST	[BOOLEAN] The Enable Start Parameter enables the Dma Channel start service request. The main usage is for ScatterGather Element configuration.
DMA_IP_CH_SET_CONTROL_EN_MAJOR_↔ INTERRUPT	[BOOLEAN] The Enable Major Interrupt Parameter enables the Dma Channel major interrupt.
DMA_IP_CH_SET_CONTROL_EN_HALF_↔ MAJOR_INTERRUPT	[BOOLEAN] The Enable Half Interrupt Parameter enables the Dma Channel half major interrupt.
DMA_IP_CH_SET_CONTROL_DIS_AUTO_↔ REQUEST	[BOOLEAN] The Disable Automatic Request Parameter disables the Dma Channel automatic request.
DMA_IP_CH_SET_CONTROL_BANDWIDTH	[VALUE] The Bandwidth Control Parameter sets the Dma Channel bandwidth control.

Definition at line 161 of file [Dma_Ip.h](#).

6.3.4.5 Dma_Ip_LogicChannelInfoParamType

enum [Dma_Ip_LogicChannelInfoParamType](#)

This type contains the Dma Ip Logic Channel Information Parameters.

The Parameters get specific information.

Enumerator

DMA_IP_CH_GET_SOURCE_ADDRESS	[VALUE] The Source Address Parameter gets the Dma Channel source address.
DMA_IP_CH_GET_DESTINATION_ADDRESS	[VALUE] The Destination Address Parameter gets the Dma Channel destination address.
DMA_IP_CH_GET_BEGIN_ITER_COUNT	[VALUE] The Begin Iteration Count Parameter gets the Dma Channel begin iteration count.
DMA_IP_CH_GET_CURRENT_ITER_COUNT	[VALUE] The Current Iteration Count Parameter gets the Dma Channel current iteration count.
DMA_IP_CH_GET_MAJOR_INTERRUPT	[BOOLEAN] The Major Interrupt Parameter gets the Dma Channel major interrupt.
DMA_IP_CH_GET_HALF_MAJOR_INTERRUPT	[BOOLEAN] The Half Major Interrupt Parameter gets the Dma Channel half major interrupt.

Definition at line 204 of file [Dma_Ip.h](#).

6.3.4.6 Dma_Ip_ReturnType

enum [Dma_Ip_ReturnType](#)

This type contains the Dma Ip Return Type.

The Return Type give information for the execution of interfaces.

Definition at line 127 of file [Dma_Ip_Types.h](#).

6.3.4.7 Dma_Ip_HwChannelStateValueType

enum [Dma_Ip_HwChannelStateValueType](#)

This type contains the Dma Ip Channel State Value Type.

The Channel State type provides information about the channel's general state. The Reset State is present when the Dma Channel is uninitialized. The Ready State is present when the Dma Channel is initialized and without error. The Error State is present when the Dma Channel is initialized and with error.

Definition at line 145 of file [Dma_Ip_Types.h](#).

6.3.5 Function Reference

6.3.5.1 Dma_Ip_Init()

```
Dma_Ip_ReturnType Dma_Ip_Init (
    const Dma_Ip_InitType *const DmaInit )
```

This function initializes the Dma Ip Driver.

This service is a non reentrant function that shall initialize the Dma Ip driver.

Parameters

in	<i>DmaInit</i>	Pointer to the configuration structure.
----	----------------	---

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ResetState.

6.3.5.2 Dma_Ip_Deinit()

```
Dma_Ip_ReturnType Dma_Ip_Deinit (
    void )
```

This function deinitializes the Dma Ip Driver.

This service is a non reentrant function that shall deinitialize the Dma Ip driver.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the deinitialization finished ok

6.3.5.3 Dma_Ip_SetLogicInstanceCommand()

```
Dma_Ip_ReturnType Dma_Ip_SetLogicInstanceCommand (
    const uint32 LogicInst,
    const Dma_Ip_LogicInstanceCmdType Command )
```

This function sets Dma Ip Instance Command.

This service is a reentrant function that shall command the Dma Instance. The command shall trigger specific functionalities of the Dma Instance.

Parameters

in	<i>LogicInst</i>	Selection value of the Logic Instance.
in	<i>Command</i>	The command for the Logic Instance.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the command finished ok.

6.3.5.4 Dma_Ip_GetLogicInstanceStatus()

```
Dma_Ip_ReturnType Dma_Ip_GetLogicInstanceStatus (
    const uint32 LogicInst,
    Dma_Ip_LogicInstanceStatusType *const Status )
```

This function gets Dma Ip Instance Status.

This service is a reentrant function that shall get the Dma Instance status. The command shall read specific functionalities of the Dma Instance.

Parameters

in	<i>LogicInst</i>	Selection value of the Logic Instance.
out	<i>Status</i>	Pointer to the Dma Instance status.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the get status finished ok.

6.3.5.5 Dma_Ip_LogicChannelInit()

```
Dma_Ip_ReturnType Dma_Ip_LogicChannelInit (
    const uint32 LogicCh )
```

This function initializes the Dma Ip Logic Channel.

This service is a non reentrant function that shall initialize the Dma Ip Logic Channel.

Parameters

in	<i>LogicCh</i>	Selection value of the Logic Channel.
----	----------------	---------------------------------------

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ResetState.

6.3.5.6 Dma_Ip_LogicChannelDeinit()

```
Dma_Ip_ReturnType Dma_Ip_LogicChannelDeinit (
    const uint32 LogicCh )
```

This function deinitializes the Dma Ip Logic Channel.

This service is a non reentrant function that shall deinitialize the Dma Ip Logic Channel.

Parameters

in	<i>LogicCh</i>	Selection value of the Logic Channel.
----	----------------	---------------------------------------

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the deinitialization finished ok.

6.3.5.7 Dma_Ip_SetLogicChannelCommand()

```
Dma_Ip_ReturnType Dma_Ip_SetLogicChannelCommand (
    const uint32 LogicCh,
    const Dma_Ip_LogicChannelCmdType Command )
```

This function sets Dma Ip Logic Channel Command.

This service is a reentrant function that shall command the Dma Channel. The command shall trigger specific functionalities of the Dma Channel.

Parameters

in	<i>LogicCh</i>	Selection value of the Logic Channel.
in	<i>Command</i>	The command for the Logic Channel.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ReadyState.

6.3.5.8 Dma_Ip_GetLogicChannelStatus()

```
Dma_Ip_ReturnType Dma_Ip_GetLogicChannelStatus (
    const uint32 LogicCh,
    Dma_Ip_LogicChannelStatusType *const ChStatus )
```

This function gets Dma Ip Logic Channel Status.

This service is a reentrant function that shall get the Dma Channel status. The command shall read specific functionalities of the Dma Channel.

Parameters

in	<i>LogicCh</i>	Selection value of the Logic Channel.
out	<i>Status</i>	Pointer to the Dma Channel status.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the get status finished ok.

6.3.5.9 Dma_Ip_SetLogicChannelGlobalList()

```
Dma_Ip_ReturnType Dma_Ip_SetLogicChannelGlobalList (
    const uint32 LogicCh,
    const Dma_Ip_LogicChannelGlobalListType List[],
    const uint32 ListDimension )
```

This function sets Dma Ip Logic Channel Global List settings.

This service is a reentrant function that shall set the Dma Ip Logic Channel global parameters list. The list is composed of an array of Dma Ip Logic Channel global parameters settings. The settings list(array) is defined by the user needs: it contains the desired parameters to be configured, in any desired order.

How to use this interface:

1. Use the "Dma_Ip_LogicChannelGlobalListType" to create a list(array) with the desired paramaters to configure (see parameters: "Dma_Ip_LogicChannelGlobalParamType") The list can declared globally or locally: A. Global example: `Dma_Ip_LogicChannelGlobalListType global_Dma_Ip_ChannelGlobalList0[NUMBER_OF_PARAMETERS] = {...};` B. Local example: `Dma_Ip_LogicChannelGlobalListType Dma_Ip_ChannelGlobalList[NUMBER_OF_PARAMETERS]; Dma_Ip_ChannelGlobalList[PARAMETER0].Param =`

```
DMA_IP_CH_SET_EN_PREEMPTION_PRIORITY; Dma_Ip_ChannelGlobalList[PARAMETER0].Value
= TRUE; Dma_Ip_ChannelGlobalList[PARAMETER1].Param = ...; Dma_Ip_ChannelGlobalList[PARAMETER1].Value
= ...;
```

2. Call the "Dma_Ip_SetLogicChannelGlobalList()" interface: `Dma_Ip_SetLogicChannelGlobalList(LOGIC_CHANNELx, Dma_Ip_ChannelGlobalList, NUMBER_OF_PARAMETERS);`

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Id.
in	<i>List</i>	Pointer to the Global List Array.
in	<i>ListDimension</i>	Number of entries in the List.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ReadyState.

6.3.5.10 Dma_Ip_SetLogicChannelTransferList()

```
Dma_Ip_ReturnType Dma_Ip_SetLogicChannelTransferList (
    const uint32 LogicCh,
    const Dma_Ip_LogicChannelTransferListType List[],
    const uint32 ListDimension )
```

This function sets Dma Ip Logic Channel Transfer List settings.

-> This service is a reentrant function that shall set the Dma Ip Logic Channel transfer parameters list. -> The "Transfer List" loads the configuration directly into the Hardware TCD and disables the ScatterGather for the Hardware TCD. -> The list is composed of an array of Dma Ip Logic Channel transfer parameters settings. -> The settings array is defined by the user needs: it contains entries for each desired parameter to be configured, in any suitable order.

-> How to use this interface: <-

1. Use the "Dma_Ip_LogicChannelTransferListType" to create a list(array) with the desired parameters to configure (see parameters: "Dma_Ip_LogicChannelTransferParamType") The list can be declared globally or locally: Global example: `#define DMA_IP_TRANSFER_LIST0_DIMENSION ((uint32)2U) Dma_Ip_LogicChannelTransferListType global_Dma_Ip_ChannelTransferList0[DMA_IP_TRANSFER_LIST0_DIMENSION] = {...};` Local example: `#define DMA_IP_TRANSFER_LIST0_DIMENSION ((uint32)2U) Dma_Ip_LogicChannelTransferListType Dma_Ip_ChannelTransferList0[DMA_IP_TRANSFER_LIST0_DIMENSION]; Dma_Ip_ChannelTransferList0[PARAMETER0].Param = DMA_IP_CH_SET_VAL_SOURCE_ADDRESS; Dma_Ip_ChannelTransferList0[PARAMETER0].Value = &SourceBuffer; Dma_Ip_ChannelTransferList0[PARAMETER1].Param = DMA_IP_CH_SET_DESTINATION_ADDRESS; Dma_Ip_ChannelTransferList0[PARAMETER1].Value = &DestinationBuffer;`
2. Call the "Dma_Ip_SetLogicChannelTransferList()" interface: `Dma_Ip_SetLogicChannelTransferList(LOGIC_CHANNELx, Dma_Ip_ChannelTransferList0, DMA_IP_TRANSFER_LIST0_DIMENSION);`

-> Coding Example: <- -> The user shall create the desired configuration list for his specific application. "UserDefinedFileName.h" `#define DMA_IP_TRANSFER_LIST0_DIMENSION ((uint32)8U) #define DMA_IP_SET_TRANSFER_TYPE0(CHANNEL, DIMENSION, SADDR, SOFF, SSIZE, DADDR, DOFF, DSIZE, MINOR_SIZE, MAJOR_COUNT) \ Dma_Ip_LogicChannelTransferListType Dma_Ip_ChannelTransferList0[DIMENSION]; \ Dma_Ip_ChannelTransferList0[0U].Param = DMA_IP_CH_SET_VAL_SOURCE_ADDRESS; \ Dma_Ip_ChannelTransferList0[0U].Value = SADDR; \ Dma_Ip_ChannelTransferList0[1U].Param = DMA_IP_CH_SET_SOURCE_SIGNED_OFFSET; \ Dma_Ip_ChannelTransferList0[1U].Value = SOFF;`


```

\ Dma_Ip_ChannelTransferList0[2U].Param = DMA_IP_CH_SET_SOURCE_TRANSFER_SIZE; \ Dma_Ip_ChannelTransferList0[2U].Value = SSIZE; \ Dma_Ip_ChannelTransferList0[3U].Param = DMA_IP_CH_SET_DESTINATION_ADDRESS; \ Dma_Ip_ChannelTransferList0[3U].Value = DADDR; \ Dma_Ip_ChannelTransferList0[4U].Param = DMA_IP_CH_SET_DESTINATION_SIGNED_OFFSET; \ Dma_Ip_ChannelTransferList0[4U].Value = DOFF; \ Dma_Ip_ChannelTransferList0[5U].Param = DMA_IP_CH_SET_DESTINATION_TRANSFER_SIZE; \ Dma_Ip_ChannelTransferList0[5U].Value = DSIZE; \ Dma_Ip_ChannelTransferList0[6U].Param = DMA_IP_CH_SET_MINORLOOP_SIZE; \ Dma_Ip_ChannelTransferList0[6U].Value = MINOR_SIZE; \ Dma_Ip_ChannelTransferList0[7U].Param = DMA_IP_CH_SET_MAJORLOOP_COUNT; \ Dma_Ip_ChannelTransferList0[7U].Value = MAJOR_COUNT; \ Dma_Ip_SetLogicChannelTransferList(CHANNEL, Dma_Ip_ChannelTransferList0, DIMENSION);

```

```
"ApplicationFileName.c" void ConfigureDmaChannel(ConfigType * pxConfig) { uint32 MinorLoopSize = 2U; uint32 MajorLoopCount; if(pxConfig->MajorLoopCondition == TRUE) { MajorLoopCount = 8U; } else { MajorLoopCount = 24U; } DMA_IP_SET_TRANSFER_TYPE0(pxConfig->LogicChannel, DMA_IP_TRANSFER_LIST0_DIMENSION, pxConfig->SourceBuffer, 2U, DMA_IP_TRANSFER_SIZE_2_BYTE, &RegisterAddress, 0U, DMA_IP_TRANSFER_SIZE_2_BYTE, MinorLoopSize, MajorLoopCount); }
```

Parameters

in	<i>Channel</i>	Specifies the Logic Channel Id.
in	<i>List</i>	Pointer to the Transfer List Array.
in	<i>ListDimension</i>	Number of entries in the List.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ReadyState.

6.3.5.11 Dma_Ip_SetLogicChannelScatterGatherList()

```
Dma_Ip_ReturnType Dma_Ip_SetLogicChannelScatterGatherList (
    const uint32 LogicCh,
    const uint32 Element,
    const Dma_Ip_LogicChannelScatterGatherListType List[],
    const uint32 ListDimension )
```

This function sets Dma Ip Logic Channel Scatter/Gather List settings.

-> This service is a reentrant function that shall set the Dma Ip Logic Channel scatter/gather parameters list. -> The "Scatter/Gather List" configures Logic Elements belonging to the same Dma Logic Channel. -> The "Scatter/Gather List" loads the configuration into the Software TCD. The Software TCD has the Scatter/Gather Enable set (ESG bit) and the Next Software TCD Address already loaded during the configuration generation process. -> The "Scatter/Gather List" shall not be able to modify the Scatter/Gather Element linkage (reorder of elements in the chain). The linkage of the elements is set only during the configuration process. -> The settings array is defined by the user needs: it contains entries for each desired parameter to be configured, in any suitable order. -> This service does not load the Logic Element into the Hardware TCD. This functionality is covered by Dma_Ip_SetLogicChannelScatterGatherConfig.

How to use this interface:

1. Use the "Dma_Ip_LogicChannelScatterGatherListType" to create a list(array) with the desired paramaters to configure (see parameters: "Dma_Ip_LogicChannelTransferParamType") The list can declared globally or locally: Global example: `Dma_Ip_LogicChannelScatterGatherListType global_Dma_Ip_ChannelScatterGatherList0[NUMBER_OF_PARAMETERS] = {...}`; Local example: `Dma_Ip_LogicChannelScatterGatherListType Dma_Ip_ChannelScatterGatherList[NUMBER_OF_PARAMETERS];` `Dma_Ip_Channel`

```
ScatterGatherList[PARAMETER0].Param = DMA_IP_CH_SET_VAL_SOURCE_ADDRESS; Dma_Ip_ChannelScatterGatherList[PARAMETER0].Value = &SourceBuffer; Dma_Ip_ChannelScatterGatherList[PARAMETER1].Param = DMA_IP_CH_SET_DESTINATION_ADDRESS; Dma_Ip_ChannelScatterGatherList[PARAMETER1].Value = &DestinationBuffer;
```

2. Call the "Dma_Ip_SetLogicChannelScatterGatherList()" interface: Dma_Ip_SetLogicChannelScatterGatherList(LOGIC_CHANNELx, LOGIC_ELEMENTy, Dma_Ip_ChannelScatterGatherList, NUMBER_OF_PARAMETERS);

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ReadyState.

6.3.5.12 Dma_Ip_GetLogicChannelParam()

```
Dma_Ip_ReturnType Dma_Ip_GetLogicChannelParam (
    const uint32 LogicCh,
    const Dma_Ip_LogicChannelInfoParamType Param,
    uint32 *const Value )
```

This function gets the Dma Ip Logic Channel Parameter value.

This service is a reentrant function that shall get the Dma Channel parameters value.

Parameters

in	<i>LogicCh</i>	Selection value of the Logic Channel.
in	<i>Param</i>	Selection parameter.
out	<i>Value</i>	Pointer to the parameter value.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the get information finished ok.

6.3.5.13 Dma_Ip_SetLogicChannelScatterGatherConfig()

```
Dma_Ip_ReturnType Dma_Ip_SetLogicChannelScatterGatherConfig (
    const uint32 LogicCh,
    const uint32 Element )
```

This function configures the Dma Ip Logic Channel Scatter/Gather.

This service is a reentrant function that shall configure the Dma Channel scatter/gather functionality. The specified Logic Element (corresponding to a Software TCD) shall be loaded into the Dma Logic Channel's Hardware TCD. The Logic Elements (describing the Software TCDs) form a simple chained list and the "Element" parameter represents the lists's head.

Parameters

in	<i>LogicCh</i>	Selection value of the Logic Channel.
in	<i>Element</i>	Selection value of the Logic Element representing the list's head.

Returns

Dma_Ip_ReturnType DMA_IP_STATUS_SUCCESS is returned if the initialization finished ok. DMA_IP_STATUS_WRONG_STATE is returned if the Dma Ip Channel state is not Dma_Ip_Ch_ReadyState. DMA_IP_STATUS_ERROR is returned if the Dma Ip Channel contains an error.

6.4 FLEXIO IP Driver

6.4.1 Detailed Description

Enum Reference

- enum [Flexio_Mcl_Ip_TimerPolarityType](#)
- enum [Flexio_Mcl_Ip_PinPolarityType](#)
- enum [Flexio_Mcl_Ip_PinConfigType](#)
- enum [Flexio_Mcl_Ip_TriggerPolarityType](#)
- enum [Flexio_Mcl_Ip_TriggerSourceType](#)
- enum [Flexio_Mcl_Ip_TimerModeType](#)
- enum [Flexio_Mcl_Ip_TimerOutputType](#)
- enum [Flexio_Mcl_Ip_TimerDecrementType](#)
- enum [Flexio_Mcl_Ip_TimerResetType](#)
- enum [Flexio_Mcl_Ip_TimerDisableType](#)
- enum [Flexio_Mcl_Ip_TimerEnableType](#)
- enum [Flexio_Mcl_Ip_TimerStopType](#)
- enum [Flexio_Mcl_Ip_TimerStartType](#)

6.4.2 Enum Reference

6.4.2.1 Flexio_Mcl_Ip_TimerPolarityType

enum [Flexio_Mcl_Ip_TimerPolarityType](#)

Enumerator

FLEXIO_TIMER_POLARITY_POSEDGE	Shift on positive edge of Shift clock
FLEXIO_TIMER_POLARITY_NEGEDGE	Shift on negative edge of Shift clock

Definition at line 123 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.2 Flexio_Mcl_Ip_PinPolarityType

enum [Flexio_Mcl_Ip_PinPolarityType](#)

Enumerator

FLEXIO_PIN_POLARITY_HIGH	Pin is active high
FLEXIO_PIN_POLARITY_LOW	Pin is active low

Definition at line 131 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.3 Flexio_Mcl_Ip_PinConfigType

enum [Flexio_Mcl_Ip_PinConfigType](#)

Enumerator

FLEXIO_PIN_CONFIG_DISABLED	Shifter pin output disabled
FLEXIO_PIN_CONFIG_OPEN_DRAIN	Shifter pin open drain or bidirectional output enable
FLEXIO_PIN_CONFIG_BIDIR_OUTPUT	Shifter pin bidirectional output data
FLEXIO_PIN_CONFIG_OUTPUT	Shifter pin output

Definition at line 138 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.4 Flexio_Mcl_Ip_TriggerPolarityType

enum [Flexio_Mcl_Ip_TriggerPolarityType](#)

Enumerator

FLEXIO_TRIGGER_POLARITY_HIGH	Trigger is active high
FLEXIO_TRIGGER_POLARITY_LOW	Trigger is active low

Definition at line 171 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.5 Flexio_Mcl_Ip_TriggerSourceType

enum [Flexio_Mcl_Ip_TriggerSourceType](#)

Enumerator

FLEXIO_TRIGGER_SOURCE_EXTERNAL	External trigger selected
FLEXIO_TRIGGER_SOURCE_INTERNAL	Internal trigger selected

Definition at line 178 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.6 Flexio_Mcl_Ip_TimerModeType

enum [Flexio_Mcl_Ip_TimerModeType](#)

Enumerator

FLEXIO_TIMER_MODE_DISABLED	Timer Disabled.
FLEXIO_TIMER_MODE_8BIT_BAUD	Dual 8-bit counters baud/bit mode.

Enumerator

FLEXIO_TIMER_MODE_8BIT_PWM	Dual 8-bit counters PWM mode.
FLEXIO_TIMER_MODE_16BIT	Single 16-bit counter mode.
FLEXIO_TIMER_MODE_16BIT_DIS	Single 16-bit counter disable mode.
FLEXIO_TIMER_MODE_8BIT_DUAL	Dual 8-bit counters word mode.
FLEXIO_TIMER_MODE_8BIT_DUAL_PWM	Dual 8-bit counters PWM low mode.
FLEXIO_TIMER_16BIT_INPUT_CAPTURE_MODE	Single 16-bit input capture mode.

Definition at line 185 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.7 Flexio_Mcl_Ip_TimerOutputType

```
enum Flexio_Mcl_Ip_TimerOutputType
```

Enumerator

FLEXIO_TIMER_INITOUT_ONE	Timer output is logic one when enabled, unaffected by timer reset.
FLEXIO_TIMER_INITOUT_ZERO	Timer output is logic zero when enabled, unaffected by timer reset.
FLEXIO_TIMER_INITOUT_ONE_RESET	Timer output is logic one when enabled and on timer reset.
FLEXIO_TIMER_INITOUT_ZERO_RESET	Timer output is logic zero when enabled and on timer reset.

Definition at line 198 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.8 Flexio_Mcl_Ip_TimerDecrementType

```
enum Flexio_Mcl_Ip_TimerDecrementType
```

Enumerator

FLEXIO_TIMER_DECREMENT_CLK_SHIFT↔ _TMR	Decrement counter on FlexIO clock, Shift clock equals Timer output.
FLEXIO_TIMER_DECREMENT_TRG_SHIFT↔ _TMR	Decrement counter on Trigger input (both edges), Shift clock equals Timer output.
FLEXIO_TIMER_DECREMENT_PIN_SHIFT↔ PIN	Decrement counter on Pin input (both edges), Shift clock equals Pin input.
FLEXIO_TIMER_DECREMENT_TRG_SHIFT↔ _TRG	Decrement counter on Trigger input (both edges), Shift clock equals Trigger input.

Definition at line 207 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.9 Flexio_Mcl_Ip_TimerResetType

```
enum Flexio_Mcl_Ip_TimerResetType
```

Enumerator

FLEXIO_TIMER_RESET_NEVER	Timer never reset.
FLEXIO_TIMER_RESET_PIN_OUT	Timer reset on Timer Pin equal to Timer Output.
FLEXIO_TIMER_RESET_TRG_OUT	Timer reset on Timer Trigger equal to Timer Output.
FLEXIO_TIMER_RESET_PIN_RISING	Timer reset on Timer Pin rising edge.
FLEXIO_TIMER_RESET_TRG_RISING	Timer reset on Trigger rising edge.
FLEXIO_TIMER_RESET_TRG_BOTH	Timer reset on Trigger rising or falling edge.

Definition at line 216 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.10 Flexio_Mcl_Ip_TimerDisableType

enum [Flexio_Mcl_Ip_TimerDisableType](#)

Enumerator

FLEXIO_TIMER_DISABLE_NEVER	Timer never disabled.
FLEXIO_TIMER_DISABLE_TIM_DISABLE	Timer disabled on Timer N-1 disable.
FLEXIO_TIMER_DISABLE_TIM_CMP	Timer disabled on Timer compare.
FLEXIO_TIMER_DISABLE_TIM_CMP_TRG_↔ LOW	Timer disabled on Timer compare and Trigger Low.
FLEXIO_TIMER_DISABLE_PIN	Timer disabled on Pin rising or falling edge.
FLEXIO_TIMER_DISABLE_PIN_TRG_HIGH	Timer disabled on Pin rising or falling edge provided Trigger is high.
FLEXIO_TIMER_DISABLE_TRG	Timer disabled on Trigger falling edge.

Definition at line 227 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.11 Flexio_Mcl_Ip_TimerEnableType

enum [Flexio_Mcl_Ip_TimerEnableType](#)

Enumerator

FLEXIO_TIMER_ENABLE_ALWAYS	Timer always enabled.
FLEXIO_TIMER_ENABLE_TIM_ENABLE	Timer enabled on Timer N-1 enable.
FLEXIO_TIMER_ENABLE_TRG_HIGH	Timer enabled on Trigger high.
FLEXIO_TIMER_ENABLE_TRG_PIN_HIGH	Timer enabled on Trigger high and Pin high.
FLEXIO_TIMER_ENABLE_PIN_POSEDGE	Timer enabled on Pin rising edge.
FLEXIO_TIMER_ENABLE_PIN_POSEDGE_TRG_↔ _HIGH	Timer enabled on Pin rising edge and Trigger high.
FLEXIO_TIMER_ENABLE_TRG_POSEDGE	Timer enabled on Trigger rising edge.
FLEXIO_TIMER_ENABLE_TRG_EDGE	Timer enabled on Trigger rising or falling edge.

Definition at line 239 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.12 Flexio_Mcl_Ip_TimerStopType

```
enum Flexio_Mcl_Ip_TimerStopType
```

Enumerator

FLEXIO_TIMER_STOP_BIT_DISABLED	Stop bit disabled.
FLEXIO_TIMER_STOP_BIT_TIM_CMP	Stop bit is enabled on timer compare.
FLEXIO_TIMER_STOP_BIT_TIM_DIS	Stop bit is enabled on timer disable.
FLEXIO_TIMER_STOP_BIT_TIM_CMP_DIS	Stop bit is enabled on timer compare and disable.

Definition at line 252 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.4.2.13 Flexio_Mcl_Ip_TimerStartType

```
enum Flexio_Mcl_Ip_TimerStartType
```

Enumerator

FLEXIO_TIMER_START_BIT_DISABLED	Start bit disabled.
FLEXIO_TIMER_START_BIT_ENABLED	Start bit enabled.

Definition at line 278 of file [Flexio_Mcl_Ip_HwAccess.h](#).

6.5 FTM IP Driver

6.5.1 Detailed Description

Function Reference

- void [Ftm_Mcl_Ip_SelectCommonTimebase](#) (uint8 Instance, uint32 SyncList)

Implementation specific function to updates the Global Timebase bits of configured modules.

6.5.2 Function Reference

6.5.2.1 Ftm_Mcl_Ip_SelectCommonTimebase()

```
void Ftm_Mcl_Ip_SelectCommonTimebase (
    uint8 Instance,
    uint32 SyncList )
```

Implementation specific function to updates the Global Timebase bits of configured modules.

This function is used to set the global timebase bits for modules that support the global timebase feature. The function selects the module that gives the common timebase and the modules that are use this timebase (as bits in

syncList). Then it synchronizes the modules. example: syncList is 0x0003 - modules 0 and 1 use the timebase given by instance syncList is 0x0005 - modules 0 and 2 use the timebase given by instance

Parameters

in	<i>Instance</i>	FTM module id
in	<i>SyncList</i>	FTM module mask value

Returns

void

6.6 TRGMUX IP Driver

6.6.1 Detailed Description

Function Reference

- Trgmux_Ip_StatusType [Trgmux_Ip_Init](#) (const Trgmux_Ip_InitType *const pxTrgmuxInit)
This function initializes the Trgmux Ip Driver.
- Trgmux_Ip_StatusType [Trgmux_Ip_SetInput](#) (const uint32 LogicTrigger, const uint32 Input)
This function sets the Trgmux Ip Input.
- Trgmux_Ip_StatusType [Trgmux_Ip_SetLock](#) (const uint32 LogicTrigger)
This function locks the Trgmux Ip Trigger.

6.6.2 Function Reference

6.6.2.1 Trgmux_Ip_Init()

```
Trgmux_Ip_StatusType Trgmux_Ip_Init (
    const Trgmux_Ip_InitType *const pxTrgmuxInit )
```

This function initializes the Trgmux Ip Driver.

This service is a non reentrant function that shall initialize the Trgmux Ip driver.

Parameters

in	<i>pxTrgmuxInit</i>	Pointer to the initialization structure.
----	---------------------	--

Returns

TRGMUX_IP_STATUS_LOCKED The trigger is locked. TRGMUX_IP_STATUS_SUCCESS The initialization was successful.

6.6.2.2 Trgmux_Ip_SetInput()

```
Trgmux_Ip_StatusType Trgmux_Ip_SetInput (
    const uint32 LogicTrigger,
    const uint32 Input )
```


Module Documentation

This function sets the Trgmux Ip Input.

This service is a reentrant function that shall set the Trgmux Ip Input.

Parameters

in	<i>LogicTrigger</i>	The logic trigger id.
in	<i>Input</i>	The input value.

Returns

TRGMUX_IP_STATUS_LOCKED The trigger is locked. TRGMUX_IP_STATUS_SUCCESS The initialization was successful.

6.6.2.3 Trgmux_Ip_SetLock()

```
Trgmux_Ip_StatusType Trgmux_Ip_SetLock (  
    const uint32 LogicTrigger )
```

This function locks the Trgmux Ip Trigger.

This service is a reentrant function that shall lock the Trgmux Ip Trigger.

Parameters

in	<i>LogicTrigger</i>	The logic trigger id.
----	---------------------	-----------------------

Returns

TRGMUX_IP_STATUS_LOCKED The trigger is locked. TRGMUX_IP_STATUS_SUCCESS The initialization was successful.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.